Introduction to Weather Technology using MetPy

September 13, 2026

Contents

0.1 Introduction . . . 1
0.1.1 Why Python? . . . 2
0.1.2 Jupyter Lab Environment . . . 3
0.1.3 Linux Introduction . . . 4
0.1.4 Editing Files in Linux . . . 16
0.1.5 Hypertext Markup Language (HTML) . . . 18
0.1.6 Python Script from a Notebook File . . . 21
0.1.7 Create Looping GIFs . . . 22
0.2 Python Syntax . . . 23
0.2.1 Python Basics . . . 24
0.2.2 Python Program Control Flow . . . 33
0.2.3 Datetime Module . . . 39
0.2.4 Python Script . . . 43
0.2.5 Runtime Control with Scripts . . . 45
0.3 Meteorological Concepts, Data Analysis, and Visualization using MetPy . . . 47
0.3.1 Meteorological Python . . . 47
0.3.2 Surface Obs and Plotting . . . 48
0.3.3 Upperair Obs and Plotting . . . 60
0.3.4 Introduction to Hand Analysis . . . 74
0.3.5 Atmospheric Forces . . . 86
0.3.6 Gridded Data and Contour Plotting . . . 93
0.3.7 Atmospheric Force Balances . . . 110
0.3.8 Color-filled Plotting . . . 117
0.3.9 Life Cycle of Mid-Latitude Cyclones . . . 124
0.3.10 Overlay Observations and Contours . . . 135
0.3.11 Numerical Weather Prediction . . . 138
0.3.12 Derived Atmospheric Parameters . . . 142
0.3.13 Calculations with MetPy . . . 148
0.3.14 Skew-T Log-p Diagram . . . 167
0.3.15 Satellite Observations . . . 186
0.4 Appendix . . . 197
0.4.1 Common Errors and Solutions . . . 198
0.4.2 Accessing Surface Data . . . 200
0.4.3 Accessing Upperair Data . . . 202
0.4.4 Access Gridded Archive Data . . . 204
0.4.5 Working with NAM Data . . . 206
0.4.6 MetPy Declarative Syntax . . . 209

0.1 Introduction

This textbook serves as an introduction to using scientific computing in Python to analyze and visualize atmospheric data commonly used in synoptic meteorology. The book is organized into three main sections:

1. a brief introduction with material covering why we currently use Python, how to run Jupyter Notebooks, and how to get started using Python on your own machines.
2. an introduction to basic use of Python including variables, lists, dictionaries, and slicing as well as how to run Python scripts, program control elements, and the datetime module;
3. a comprehensive introduction to the declarative syntax that is part of the MetPy module with the context of introductory meteorological concepts.

Incorporated throughout the text there are links to additional resources to continue to expand your knowledge of and use of Python in analyzing and visualizing meteorological data.

0.1.1 Why Python?

There are many different computer programming languages that can do many things, so why Python to analyze and visualize weather data? We'll Python has been around since the late 1980s and grew in popularity through the 1990s. In the early 2000s the scientific community began to adopt the language through building complementary packages that supported the types of work they were doing. This led to the development of packages to handle numeric arrays and create high-quality graphs of data. Over the past fifteen years with the advent of easy to use Python package management, an explosion of niche packages has come along to make doing science easier by adding calculation, analysis, and visualization tools to the Python ecosystem and has largely become the default programming language for many disciplines, including atmospheric science.

However, Python can't solve everything. For example, the core of numerical weather prediction models are still based in Fortran or C code, as the advantages of those compiled languages and long-tested codes are the right tool for the task.

0.1.2 Jupyter Lab Environment

Since MetPy is a module within the scientific Python ecosystem we will use Jupyter Notebooks through the Jupyter Lab environment to create our desired analyses and maps over the course of this book. We'll begin, by describing the Jupyter Lab environment. To launch jupyter lab on a Linux system type,

```
jupyter lab
```

When the Jupyter Lab environment opens in your web browser you'll have two areas in the window: a file browser on the left, and a launcher tab on the right. The following video describes the Jupyter Lab interface. Whether you are running it locally or connecting through a JuptyerHub the structure of the environment will be the same.

Common Keyboard Commands:

> Run A Cell and move to next cell): <shift>-<return>
> Run A Cell and stay on the cell): <ctrl>-<return>
> To Undo Typing in Cell: <alt>-z

Cell Types:

- Code – executable Python code
- Raw – raw text, won't execute
- Markdown – formatted text output

It is always best to make a copy of an example notebook to a new place/name so that you can always get back to the original. If you copied that notebook to your met130 directory, you'll want to navigate to that directory in the file browser on the left side of the Jupyter Lab environment. The notebook will have a small orange icon associated with the name of the notebook. To launch that notebook simple double click on that file and it will open the notebook as a second tab in the right pane.

To learn more about about the Jupyter Lab web interface, visit the following website:
Jupyter Lab Overview

0.1.3 Linux Introduction

Many if not all of you are familiar with computers. You have grown up in an age of computers that are easily accessible. You can surf the web, stay connected with friends, or play endless hours of video games and watch YouTube videos. However, few of you have likely used a computer and its programming capability to answer scientific questions. This chapter will begin the process of introducing you to the computer as a scientific tool. Therefore, we will start the process by introducing you to the Linux operating system (OS).

You can access a Linux terminal through the Jupyter Lab environment through the "Launcher" page (Figure 1). The Linux (Unix) operation environment has come a long way over the years. Historically the Linux OS (or its predecessors) was a text-based only environment, which means that there was no 'Desktop', folder icons, etc. There wasn't even a mouse! Of course, with no "windows" there was no need.

Since the first computers were so big, and by big I mean literally room- size or larger, you had a few access terminals (consoles) to log onto the computer and do the work you needed to do. This terminology lives on today with the terminal. This chapter will introduce you to the commands to work within this text-based environment.

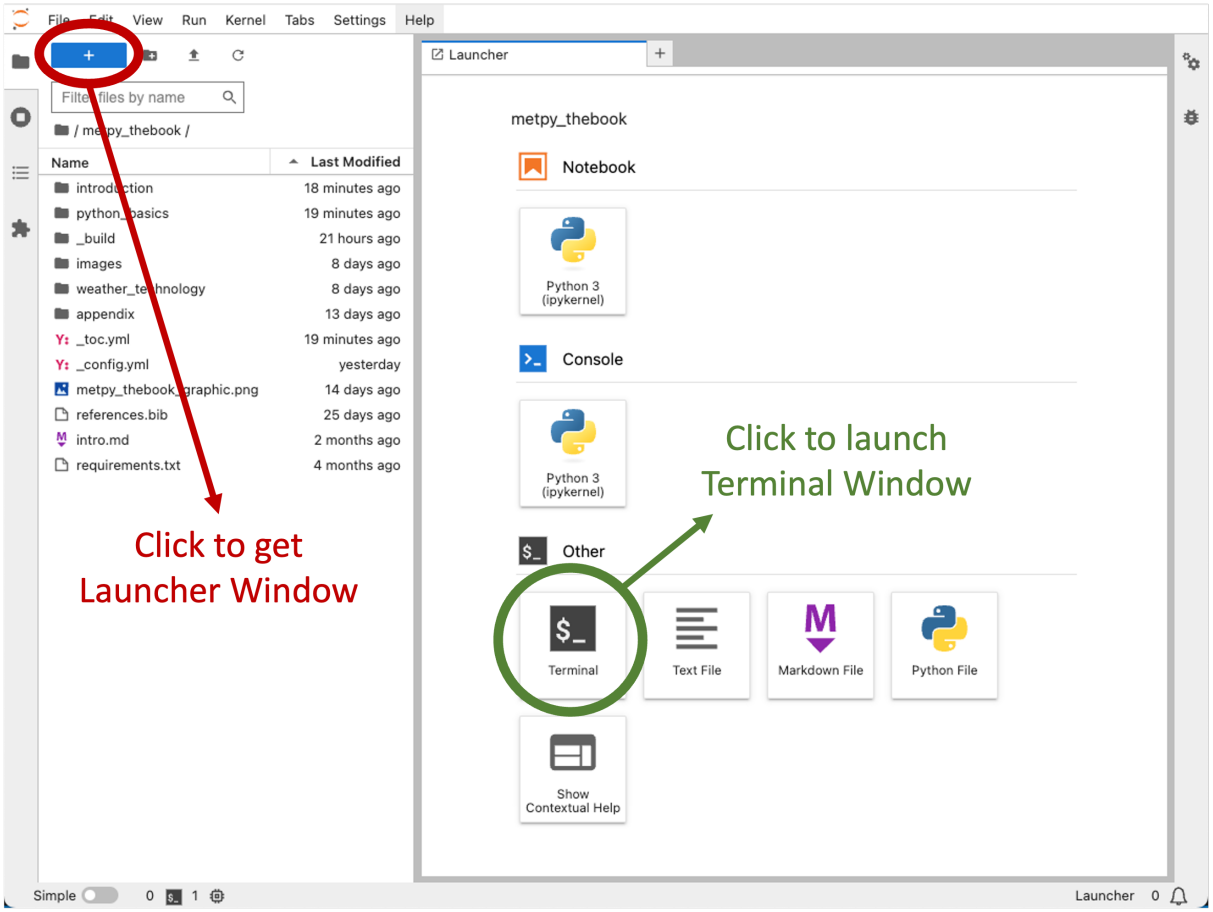


Figure 1: An annotated image illustrating what to click to launch a terminal window through th Jupyter Lab environment.

While the terminal is the "window" you will be working in, the language of the terminal is known as the **SHELL**. A shell is a command interpreter. It basically takes input from the user at the command prompt and instructs the computer to perform an operation. In other words, it is a program that allows the user to use the computer without having to deal directly with the "binary" code the computer uses. Hence in one sense, we can think of this as a "protective shell" that prevents the user and the operator from directly communicating with each other.

There are several different **shells** available. Our system is built on the **BASH** shell. Another common shell is the **TCSH**. While there are differences between the two shells, the commands that

we will be discussing today will work in both shells.

Linux Paths

Before we begin diving into the world of Linux, we need to discuss the concept of a path. Most of you have already dealt with paths in the windows world and just didn't know it. In general, a path is the route through a file system to get to a particular file. In Linux there are two kinds of paths: **absolute paths** and **relative paths**.

Absolute paths ALWAYS begin with a / and relative paths do not. It is that simple. The reason being is that an **absolute path** works no matter where you are in the file system. Thus, the directions must start from the "**root**" or beginning directory of the file system. This "**root**" directory is represented with a `/` . In Windows the root directory is C:\. (Please note that the direction of the "slashes" is correct.) In Windows all of the core system files (files needed by the operating system) are located in the C:\WINDOWS directory. The Linux equivalent of this is the `/usr/bin` directory. This is where the data used by the Linux operating system are stored.

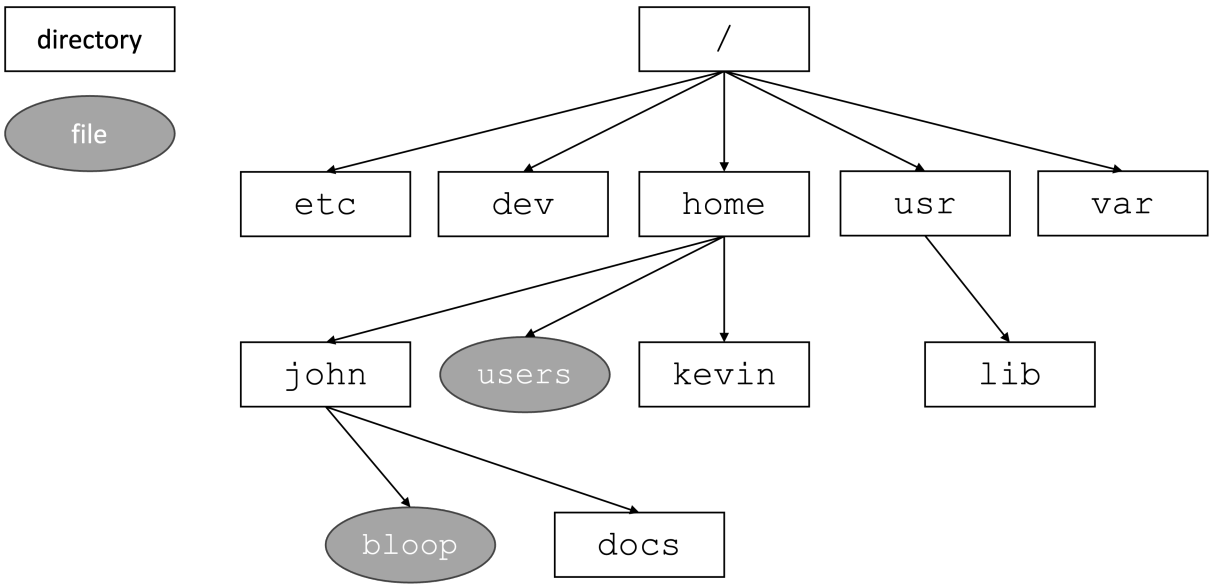


Figure 2: An example of a directory tree structure of a fictional Linux machine.

From the example in Figure Figure 2, here are some absolute and relative paths:

`/home/kevin/` (absolute path; directory)
`/usr/lib/` (absolute path; directory)
`john/bloop` (relative path from `/home`; filename with path)
`../var/` (relative path from `/usr/lib`; directory)

Where am I?

In a text-based environment, it is very important to know where you are, so how can we find out where we are in a text-based system? Well, there is a nice and simple command that will print the working directory, type `pwd`.

```
kgoebber@bergeron:~>pwd
```

You should see something similar to `/home/kgoebber` where `kgoebber` is your user name. This is your home directory, where you will start every terminal session. So now that we know where we are, we need to know *what* is in this directory.

What do I have here?

In Linux terminology we want to *list* the contents of the directory, type `ls` in a terminal window.

```
kgoebber@bergeron:~>ls
```

When you execute this command you will see the names of all of the files, directories, and links contained within your working directory. Is there anything there?

There are several variations on the `ls` command that provide additional useful information. For example:

`ls -a`: This lists all files and folders...including those that are hidden

`ls -F`: This lists all files and adds a / to the end of all directories

`ls -l`: This lists all files in "long" format.

`ls -ltr`: This lists all files in the long time reversed order (newest last)

There are a lot more options we can add to a `ls` command. In fact, there are more than we have time to cover in class.

Where's the manual for this thing?

This brings us to our next command: `man`. `man` is a short for manual page. If you type `man [command name]` you will receive the manual pages for that command. Now what about those instances where you do not know what the command is that you are looking to use. What then? Well, fortunately, `man` has a built-in search function. To use this, type `man -K [what you are searching for]` in the command prompt. This will pull up every manual page containing the word you searched. Nifty. For a full description of any of the commands and all of the options (i.e., what the `-a`, `-F`, and `-l` are for the list command) can be obtained by using the system *manual*.

```
kgoebber@bergeron:~>man ls
```

This will allow you to see the full documentation on the list command. Use *man* whenever you need to find out more information about a command or its uses.

How do I create/remove a directory?

Now, let's make a directory so that you can keep yourself organized using a Linux OS. It's really easy to make directories using the command `mkdir`, for example make a directory called wxtech.

```
kgoebber@bergeron:~>mkdir wxtech
```

To remove a directory, you first need to empty of the directory of any files or other directories then use the `rmdir` command to remove the directory.

How do I move to another directory?

Now let's move around the file system a little. If we want to move to a new directory there are two ways to do this: we can use the absolute path to where we wish to go, or we can navigate around by using relative paths.

Before we can even navigate around, we need to know what command will allow us to change to a new directory. In Linux parlance we will want to change directory. To do this we use `cd`. Change to the Download directory.

`kgoebber@bergeron:~>cd wxtech`

Now show the working directory and list the contents of this directory, use the `-a` option for listing.

From the listing in the "wxtech" directory you should notice that there are two relative paths that are contained in this (and every) directory. These two relative paths are `.` and `..`. The `.` always refers to the directory that you are currently working in. The `..` refers to the directory immediately above the one you are working in. This is often referred to as the **parent directory**.

So, to go back to your home directory you only have to go back one directory:

`kgoebber@bergeron:~>cd ..`

Print your working directory to confirm that you are in fact back at home.

There are also a few directories that are mounted to each machine that can also be accessed. One of the things that we will be doing in the terminal is running various weather programs to create and analyze various weather charts. So where is weather data located on the Bergeron computer system? The location is `/data/ldmdata/`, go to this location now.

> **Note**
> This brings me to an interesting dilemma. What are data? In science, the term data implies fact. In meteorology, we have very little fact. The facts we do have are observations. Thus, in meteorology, the only thing we should call **data** are observations. A lot of people like to look at meteorology computer models and say they are looking at model data. As we have just covered, this is a misnomer. Models do not produce **data**. The correct term for this is **model output**. In fact, **data is what we put into models and the models then produce a forecast – which is called model output.**

Take a look around this directory. Go into other directories, list the contents, see what is there and how it is organized.

Let's go back to our home directories. There are at least three different ways to do this in one command (many more if we only move one directory at a time). One way is to use the **absolute path**. The other two ways use various **relative paths**.

`cd /home/kgoebber` (absolute path)
`cd ~` (relative path)
`cd` (relative path)

The first two are fairly simple to understand. The last one may cause some problems (if you just look at it). The reason the last one works is that if you were to just type `cd` and hit enter you will always be taken back to your home directory. Think of the last command as if you were in the "Wizard of Oz", "It is your Ruby Red Slippers. It always takes you home."

Go back to your home directory through your favorite method described above.

How do I make a copy of something?

There are times you will want to reuse something you have written before or save a separate copy of an image so that you won't accidentally delete or overwrite the file. What you want to do is make a *copy* of something and we can use the copy command in Linux to complete this task. I have put a copy of the course syllabus, syllabus.doc, in /archive/valpo_courses/wxtech, copy it to your working directory or some other directory within your file system, the command is `cp`. If you wish to copy something between two different computers, then the command is `scp`. An structure of a copy command is:

```
kgoebber@bergeron:~>cp <filename 1><filename 2>
```

> **Note**
> The above example is a generic version of copying a file where you would need to input actual filenames and locations for a specific file that you are looking to copy.

A specific example may look something like:

```
kgoebber@bergeron:~>cp /data/ldmdata/surface/sao/2023080312.wmo .
```

*The "dot" at the end is **very** important.* That is telling the system to copy the file from the location specified and put it in your current working directory and call it the same thing. Of course, you can change the name of the new, copied, file by replacing the "dot" with the new name you want to give the file. (The above example is a case in which we have used a relative path, you could do the same thing using an absolute path, but it would just be more typing.) When copying a file you make a second file that is an exact replica of the original file.

If you complete a copy command in your own directory and list the contents of the directory you are working in you'll see the file that you copied to your directory.

How do I rename and/or move something?

When you copy something, what was once one are now two. Say instead of having two instances of a file you just wanted to rename the file. Without the window view we can't click on an icon and change the name, we need a program. In Linux this is called a move, such that we are moving one filename to another filename, the command is `mv`. The `mv` command requires two pieces of information. For example,

```
kgoebber@bergeron:~>mv <current filename><new filename>
```

If you rename a file there will still be only one file, just with a different name. If you move a file within your directory and you list the contents of your directory there will no longer be a file with the old filename, but there will be file new filename.

How do I remove a file?

But what if I created something and I didn't want it any more, I want to *remove* it from my file system and never, EVER, be able to retrieve it again, ever. The command to remove something from you file system is `rm`. For example, we can delete the test file we renamed,

```
kgoebber@bergeron:~>rm <filename>
```

Be very, very careful with remove as you will NOT be able to recover any file you remove!

What's in a file?

Finally, what if I want to read the contents of a file? Well, there are a couple of ways to read the file. We'll stick with some of the easy ways and learn how to write in a file next time. The three easiest ways to read the content of a file are `cat`, `less`, and `more`.

```
cat <filename>
more <filename>
less <filename>
```

Go ahead and cat a file in your directory to see what is contained within it.

> **Note**
> To get out of less or more hit `q`.

0.1.4 Editing Files in Linux

Welcome to vi

So how do we modify or create a text file via the command line? From the terminal you can create text files using a number of different programs, however, I will be introducing you to one of the more basic programs. You are free to use whatever text editor you wish; however, these labs will use vi as that is what the instructors know well. So here are some useful vi commands and features that are good to know if you choose to use vi. (More info in the recommended textbook beginning on page 159.)

Let's open a new terminal and get to work. To open a new file or old file type,

`vi <filename>` - opens the file in a vi editor, where `<filename>` is the name of the file you wish to open, whether new or old.

```
kgoebber@bergeron:~>vi test.txt
```

The previous command will open the file `test.txt` using the Linux program vi. In Linux the suffix (i.e., the stuff to the right of the "dot", such as `.txt`) does not matter. They are informative only for you. The type of file that it is, is set without adding a suffix, so you don't even need one. However, I usually find that it is a good habit to put some appropriate suffix on the file so that you can identify what the file is without opening it!

There are two different modes in vi, *insert* mode and *command* mode. When you open a file in vi you will ALWAYS be in *command* mode; there are multiple ways to get into insert mode.

`i` – will take you into insert mode wherever the cursor is.

`a` – will put you into insert mode at the character after your current cursor placement.

`o` – will put you into insert mode at the beginning of a new inserted line.

Other insert modes:

`R` – the capital R is a replace mode and will write over the text underneath the cursor until you hit escape to get out of insert mode

`r` – the little "r" is a replace mode for one character and after one character is input it will return to command mode

Once in insert mode, you can type in whatever you like. To get out of insert mode simply hit the **escape key**. When you are in doubt of whether or not you in insert mode or command mode hit the escape button a couple of times. You can always re-enter insert mode with one of the methods outlined above.

In command mode you can save and quit out of the program. Always need a colon first.

`:w`– write or saves the file with the filename initiated when vi was launched

`:q` – quits out of vi and takes you back to the unix shell

`:q!` – quits without saving and takes you back to the unix shell

`:wq` – writes and quits in sequence.

Moving Around in vi

While in command mode:

`crtl-f` – moves forward a page

`crtl-b` – moves backward a page

`shift-G` – will take you directly to the last line of the document

`1<shift-G>` – will take you back to the first line (same as :1<cr>)

`shift-$` – will take you to the end of the current cursor line

`shift-^` – will take you to the beginning of the current cursor line

`yy` – copy a line

`p` – put copy or deleted line on the line after the current cursor location

`dd` – deletes the current cursor line

`x` – deletes current character

All of the previous commands can be used with a number in front, such as:

`25dd` – will delete the next 25 lines including the line the cursor is on

Searching around vi

`/<word>` – searches forward for that word in the document

- while searching use n to go to the next instance of the word for which you are searching

`:<line number>` – will take you directly to that line number in the document, if it exists

Practice Tasks - vi

1. Create a simple text file that outlines the process of making a surface map. Make sure to note some of the common mistakes that you make at various points in the process.
2. Create a simple text file that contains your favorite Linux commands that you have used to this point and describe how each of them works.

0.1.5 Hypertext Markup Language (HTML)

How do we go about disseminating meteorological information? Well, there are actually a number of different mechanisms to do so including, television, radio, newspaper, fax machine, and the Internet. Since the advent of the Internet, dissemination of all types of meteorological information has gotten easier. Previously, difax (digital fax) maps were the primary mechanism to view weather data. We used to hang maps on the wall; it was a legacy from the era before high-speed Internet. With the Internet we can post many more images than what would be transmitted via fax, in a better quality, with color! So it is important to know at least the very basics of how to construct a simple html page and putting images and text on them for public consumption.

As part of your account on bergeron and us placing an Apache server on it, you all have the ability to create a public html page. This page should only be used for generally meteorological purposes and we'll use it for the remainder of the semester for our assignments and the semester project. First, we need to get everyone started, so we will make a simple webpage using the tag language of html. You'll be introduced to the most common tags that are used in creating an html page. You can create a webpage through other means, such as PHP, but we will keep it to the basics here.

> **Note**
> The following commands are going to be different depending on how and where you are creating a website. The HTML tags are applicable across systems. So you may need to create any HTML pages in a different location/manner, but the contents of a simple example webpage would be the same.

Step 1: Creating your homepage

The next step is to create an index file. Your homepage is the index.html file.

Go into your web directory:

```
kgoebber@bergeron:~>cd public_html\
```

Use vi to edit a file called index.html. Even though this file does not exist, editing it will create the file. To edit the file type:

```
kgoebber@bergeron:~>vi index.html
```

Step 2: HTML document essentials

HTML uses a series of "tags" or "switches" as programming code. Anything in between the tags will be displayed as text on the webpage. The text is modified depending on which tags are used. Tags are designated by being surrounded by < and > (for a start of a tag) and </tag> (for the end of a tag). For example, the tag for bold is <B>. To make the word "Hello" in bold on your webpage, you would enter the following code:

```
<B>Hello</B>
```

This code would display **Hello**. Notice how the start and end of a tag are different.

Any webpage must have, in the very least, the following code. Think of these tags as a basic skeleton that must go into all HTML files. Go ahead and write this into your index.html file:

```
<HTML>
<HEAD>
<TITLE>This is the page title</TITLE>
</HEAD>
<BODY>
</BODY>
</HTML>
```

Be sure to save your work often!

Step 3: Other important HTML tags

The <TITLE> tag designates the title of the webpage that is displayed at the top of the web browser. Go ahead and change this to what you would like.

Everything that is displayed in the main part of the webpage is contained within the <BODY> tag.

Go ahead and type some text after the <BODY> tag such as "Hello, welcome to my homepage!"

HTML does not recognize hitting the Enter key as a carriage return regardless of what it looks like on screen. A carriage return is designated by the code
, which stands for Break. Note: There is no
 for this tag.

Go ahead and put two carriage returns in your code.

Your code should now look like this:

```
<HTML>
<HEAD>
<TITLE>My Homepage</TITLE>
</HEAD>
<BODY>
Hello, welcome to my homepage! <BR><BR>
</BODY>
</HTML>
```

Step 4: Other HTML tags

To make a link to another webpage or a file that is contained in the web space, the <A HREF> command is used. In order to close this tag, use </A>. Here is a sample statement:

```
Here is a link to <A HREF="ca5.html">Computer Assignment #5</A>.
```

This code would look like the following in a web browser:

Here is a link to Computer Assignment #5.

Notice how the "linked" text to ca5.html was the text surrounded by the tags.

If the link were to a file or webpage not in your web directory (say to the NWS homepage), your code would look like this:

```
Here is a link to the <A HREF="http://www.weather.gov">National Weather Service</A>.
```

This code would look like the following in a web browser:

Here is a link to the National Weather Service.

Notice how the reference inside the quotation marks is the same as what you would type into an address bar of a web browser.

Go ahead and type that code into your sample webpage. Your final code should now look like this

```
<HTML>
<HEAD>
<TITLE>My Homepage</TITLE>
</HEAD>
<BODY>
Hello, welcome to my homepage! <BR><BR>
Here is a link to the <A HREF="http://www.nws.noaa.gov">National Weather Service</A>.
</BODY>
</HTML>
```

Another important tag is the image tag, which allows you to place images on your webpage. The image tag is <img src="sfc_1.gif"> if the sfc_1.gif image is in the same folder as the html file. But a more robust method is to put the entire url as the image source as follows:

```
<img src="http://bergeron.valpo.edu/~kgoebber/sfc_1.gif">
```

And you just have to put in the image tag, wherever you want to image to be displayed on the page.

Step 5: Pulling up your webpage

Save the .html file. Open a web browser and go to the following address:

`http://bergeron.valpo.edu/~<username>`

Obviously, replace *username* with your username. For example, my username is kgoebber, so my website address is:

`http://bergeron.valpo.edu/~kgoebber`

THAT'S IT!! EASY!

Now, you may want to make a separate folder to store all of your lab images, html pages, etc. To do this, use the `mkdir` command. Thus, if we wanted to create a directory for this class we could do the following:

`mkdir wxtech`

This would create a subdirectory of our current directory named `wxtech`. Don't believe me? Look for yourself. Do a `ls -F`. You should now see an entry that looks like `wxtech`. You can type **cd wxtech** and see that you are now working inside the directory you just created!

Practice Tasks - HTML

1. Create a surface image using the script that you got working from a previous lab and post it to your home page with some sort of commentary.

2. Create an upper-air map from your favorite model and do the same thing as the first task.

3. Customize your webpage. Use Google (or your favorite search engine) to figure out how to change the colors and do other amazing things with a webpage. The only way to learn about making a webpage is to try and do something with your own page.

4. Create a webpage that has information from the different assignments for this class.

0.1.6 Python Script from a Notebook File

Now let's use a Notebook file from when we created a surface map to start a script instead of writing one all from scratch.

Open the Jupyter Lab environment and open a notebook that contains some Python code. Once the notebook is open navigate to the *File >Save and Export As >Executable Script* in the Lab environment (Figure 3). This will either bring up a dialog box asking where you want to save the script or automatically save it to a download folder (e.g., `/home/<username>/Downloads`)

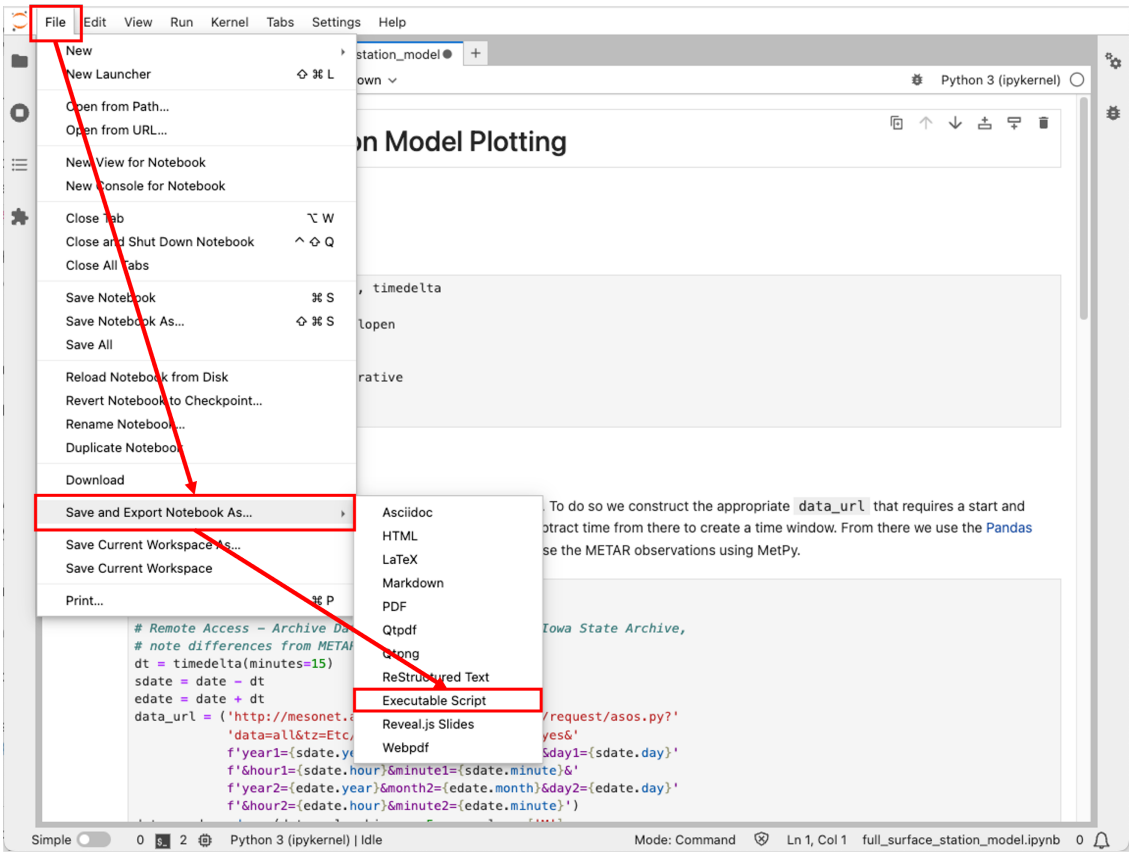


Figure 3: An illustration of the clicks to save a Jupyter Notebook to an executable script file.

Once you have saved the file, make sure you move it to your working directory (if it saved to your Downloads directory you'll want to use the Linux command to move your file to a different location).

Now open the script with vi via a terminal window and we'll edit it just a little before running the script.

1. Cleanup script from extraneous stuff

- Unfortunately, there is a lot of "gunk" that comes with going from a notebook to a script, we'll want to clean that out to make it all more readable.

2. Add a comment block at the very beginning of the script – do this so when you come back to it in April, you'll know what was supposed to be happening with this script

3. Modify any lines to get current data from today at 12 UTC

4. Make sure to save the image as a .png and do not show the image.

5. Add any other comments throughout the script so you know what is going on in each section.

Once all modifications have been made, go ahead and run the script.

What happened? Did you get any errors? If so, read the error and try to modify what wasn't working and run again.

0.1.7 Create Looping GIFs

Often we don't look at a single satellite image, but rather we want to look at a series of images to see how the atmosphere is changing over a period of time. There are a number of online tools that can help you make an animated GIF to have a "loop" of image for a presentation or website.

Any easy site to use is ezgif.com and here is the direct link: https://ezgif.com/maker

Simply browse for the series of images on your local machine, the click "Upload and make a gif". Once your images have uploaded, you'll be taken to a page that will allow you some control about how fast your loop will go and allow you to reorder the images in case they are not in the desired time order.

Quality Settings:

Delay = 100

Last image delay = 250 (or higher)

Loop Count = <leave empty>

Once you are satisfied with the animated GIF that you have created you can make any further refinements through the browser before saving to your local machine.

0.2 Python Syntax

This section will cover some basic fundamental aspects to the Python programming language to make reading and writing code that comes later in this book easier to understand and use. This is NOT a comprehensive introduction to the language and all of details of how it works. We'll cover only what is really needed and is more intended as a reference section. We'll cover the following concepts:

- Setting variables
- Basic Mathematical Operators
- Lists, Tuples, Dictionary, and Array Object Types
- Slicing of list, tuples, and array objects
- Datetime Module
- Program Control Elements
  - If/Then Statements
  - Looping
- Creating and running Python scripts
  - Interactive scripting
  - Using arguments

0.2.1 Python Basics

This section covers the basic syntax of Python, including setting variables, variable types, basic mathematical functions, basic Python objects, and subsetting of objects.

For more information on these foundational topics, please go to Project Pythia Foundations

Python Syntax - Basics #1

In this section we'll cover setting a variable with naming conventions and reserved words, the print function, and adding code comments. A good resource for additional information about the Python programming language in the atmospheric sciences is "Python Programming and Visualization for Scientists, 2nd Ed." by Alex DeCaria.

Variable Assignment In Python you set a variable using the equal sign and the variable will assume a type based on the value assigned to variable name.

```
variable_name = <value>

# Assumes type based on value given
# Following is a string (character) type
name = 'Kevin'

# Following is an integer type
temp_celsius = 20

# Following is a float type
temp_fahrenheit = 72.5
```

> **Note**
> Be careful when setting variable names. All names should begin with a letter and contain no spaces. After the first letter, any combination of letters and numbers can be used. To put a "space" in a name use an underscore (_). For example, the following is an okay variable name `first_name`. It is common practice to use only lower case letters in common variable names.

Reserved Words There are a number of words that you should not use to define a variable, becuase that words serves a very specific functionality within the Python language. The words to avoid are:

reserved	words	don't	use
`False`	`class`	`from`	`or`
`None`	`continue`	`global`	`pass`
`True`	`def`	`if`	`raise`
`and`	`del`	`import`	`return`
`as`	`elif`	`in`	`try`
`assert`	`else`	`is`	`while`
`async`	`except`	`lambda`	`with`
`await`	`finally`	`nonlocal`	`yield`
`break`	`for`	`not`	

To find out more about any element of a Python function, use the `help()` function.

```
help('keywords')
```

Print Function To print a variable, simply put the variable in the print statement.

```
print(name)
```

Output:

```
Kevin
```

To print a variable as part of a string, you can use an 'f-string'. Place an f before the beginning of string statement and use curly brackets (e.g., {}) to call the variable you wish to print in the statement.

```
print(f'My name is {name}')
```

Output:

```
My name is Kevin
```

```
# Example using the f -strings
value = 28
print(f'The varaiable has a value of {value}')
```

Code Comments It is imperative that you comment your code to ensure you remember what pieces of your code are doing. This will help when you come back to old code after a period of time or share your code with someone else. Comments can be made in two primary ways, through stand alone comments on one line or inline, either way you want to use the special character `#`, which is officially known as an octothorp or better known to you as a hashtag.

Any line that starts with a `#` will not be executed at runtime. Any line that has a `#` in it, nothing will be executed after that symbol.

```
# This is a comment and won't be executed
# The following line is executable code
print('Hello') # this is an inline comment
```

Output

```
Hello
```

```
# Setting a temperature value
tmpc = 28
print(f'The current temperature is {tmpc}C') # Printing out the current temperature
```

Python Syntax - Basics #2

In this section we'll cover setting a variable types, mathematical operations and calculations, and subsetting strings. A good resource for additional information about the Python programming language in the atmospheric sciences is "Python Programming and Visualization for Scientists, 2nd Ed." by Alex DeCaria.

Variable Types Since Python is a dynamically typed language, each variable will assume a type based on what the variable is assigned.

- integer

  ```
  a = 1
  ```

- floating point (real)

  ```
  b = 5.0
  ```

- string

  ```
  c = 'Valpo'
  ```

- boolean

  ```
  d = True
  e = False
  ```

You can check the variable type with the `type()` function.

```
a = 1
print(type(a))
```

Mathematical Calculations

Operators

- Addition (+)
- Subtraction (-)
- Division (/)
- Multiplication (*)
- Exponentiation (**)
- Floor division (//)
- Modulus/remainder (%)

Order of operations the same as well
Floats and Integers work as Reals and Integers in Fortran
NOTE: In Python, integer division will yield a real in return

```
5/9 = 0.5555555555555556
```

To get the integer of division, use floor division

```
5//9 = 0
```

```
print(5+5)
```

Converting Types

- If you read something from a file it will be input as a string
- May need to convert to integers and floats (real)

```
str(temp) makes temp a string
int(temp) makes temp an integer
float(temp) makes temp a real (float32)
```

- Integer to float

```
new_a = float(a)
```

- Not all conversions will work
  - a string that is not a number can't be converted to an integer for float value

```
# Demonstrate changing the variable type
tempf = '24'
print(type(tempf))
itmpf = int(tempf)
print(type(itmpf))
```

Variables and Math

```
TMPC = 24
TMPF = (9./5.)*TMPC + 32

# Output to the screen
print(TMPF)
print(f'The temperature in Fahrenheit is {TMPF}')
```

NOTE: We can also format the variable (TMPF) that we are printing out. Formats go after the variable and start with a colon. (e.g., {TMPF:.2f})

```
TMPC = 24
TMPK = TMPC + 273.15
print(f'The temperature in Kelvin is {TMPK}')
```

Subsetting Strings Each character within a string (including spaces) are one position. You can subset a string for a single or set of characters (positions) via its index value (address). One very important note is that Python uses a zero-based indexing scheme, so the first index value is zero (0), the second is one (1), and do on. You can also index from the end going backwards using negative numbers starting with -1, which will give you the last character.

```
dept = 'Geography'
print(dept[0])
print(dept[5])
print(dept[ -2])
```

Output

```
G
a
h
```

You can also subset for a range of indecies by using the construct of `start:stop:step`, where the subsetting is inclusive of the start value, but exclusive of the stop value.

```
print(dept[0:3])
```

Output

```
Geo
```

So we only get the values from the variable `dept` from the index values of `0`, `1`, and `2`.

```
name = 'Valparaiso'
print(name[0:4]+name[ -1])
```

Python Syntax - Basics #3

In this section we'll look at a couple of different Python objects that are commonly used. We'll cover a lists, tuples, and dictionaries.

Lists There are times when we want to be able to group together items and store them together. Python's most versitile construct for holding multiple items together is known as a list. Lists are mutable, meaning you can change the value of a item within the list. Lists can contain a single type of item (e.g., integers, floats, strings) or a mixed grouping.

There are two ways to define a list, 1) using square brackets or 2) using the list function `list()`. For example the following two are equivalent,

```
a = [1, 2, 3, 4, 5]

b = list(1, 2.5, 'Kevin', 4, 5)
```

Just like strings, we can subset a list to get a particular element using index values (addresses) of the desired elements.

```
print(a[2])
print(b[:3])
```

Output

```
3
1, 2.5, Kevin
```

Lists vs. Tuple Python doesn't have a true array object. The closest thing to an array is a Python List (which are sometimes called array-like objects). Many of their properties are the same, but there are some notable differences depending on what you are trying to do. There are two ways to define a list, 1) using square brackets or 2) using the list function `list()`. For example the following two are equivalent,

```
a = [1, 2, 3, 4, 5]

a = list(1, 2, 3, 4, 5)
```

Lists are mutable objects, meaning that you can redefine individual elements.

Another array-like object in Python is the Tuple. Tuples are different than lists in that they are immutable objects, meaning they can't be changed once assigned. Tuples can also be defined in two different ways, 1) using parentheses or 2) using the tuple function `tuple()`. For example, the following two tuple assignments are equivalent,

```
b = (10, 20, 30, 40, 50)

b = tuple(10, 20, 30, 40, 50)

# A list
a = [1, 2, 3, 4, 5]
print(a)

# Try a reassignment of a list object
a[1] = 10
```

```
# A tuple
b = ('Kevin', 'Emily', 'Adam')

# Try a reassignment of a tuple object
b[1] = 'Matt'
```

Both lists and tuples can contain different data types in the same variable. This can be advantageous at times.

List and Tuple Methods

List Object There are a number of methods that are available for list objects including: `append()`, `copy()`, `count()`, `index()`.

Python Docs: More on Lists

```
# List append method
a.append(100)
print(a)
```

Tuple Object There are only two methods available for the tuple object: `count()`, `index()`

Python Docs: Tuples and Sequences

```
# Tuple index method
indx = a.index(5)
print(indx)
```

Beware of Math with Lists and Tuples Math works VERY differently! A multiplication creates that number of copies within the list or tuple and concatenates them to the object.

```
# Multiple a list or tuple by 2
print(a*3)
```

Dictionaries Python has the ability to create a mappable data structure, which is called a dictionary. The main way you'll encounter them is through many different data read methods. The great thing about a dictionary is that you are able to reference data with a string (or key) as opposed to just an index value. This is very powerful for keeping track of a number of different arrays within a "dataset".

A dictionary is defined in one of two ways, 1) with curly brackets or 2) the `dict()` function where you link a value (or list of values) to a key (name).

```
hours = list(range(24))
stations = ['VPZ', 'GYY', 'ORD', 'MDW', 'RFD']
data = {'times': hours, 'stn': stations}
print(data['stn'])
```

Output

```
['VPZ', 'GYY', 'ORD', 'MDW', 'RFD']
```

There are a number of important methods attached to a dictionary object including: `clear()`, `copy()`, `keys()`, and `values()`

Python Docs: Dictionaries

```
units = {'tmpf': 'F', 'dwpf': 'F', 'pressure': 'hPa'}
long_names = {'tmpf': 'Temperature', 'dwpf': 'Dewpoint', 'pressure': 'Pressure'}

print(long_names['tmpf'])
```

Numpy Arrays A data object that is very import to scientists using computers is to have a true array, meaning one that we can do math on and have it automatically calculate element by element. Unfortunately, lists won't work as we have seen, so we need to bring in a third-party package (module) that contains this type of data object. Through the course of the semester we will interact with many more data objects that will be useful for our work in Earth Systems Science.

The Numpy module is a powerful package that includes not only a true array object, but a number of important calculations that can operate on these arrays in a time efficient manner. We'll dig deeper into all that Numpy as to offer in the next unit, but for now we want to introduce importing a third-party package and the development of a Numpy array.

Importing a Module There are many, many, many packages that have been developed to work off of the main Python syntax core, which is relatively small in and of itself. To gain access to those packages, they need to be imported to attach the methods contained within to your notebook/script. There are some common ways to import many packages and we'll use the most common methods over the course of this semester.

The main function for bringing in a package is `import`. There are variations on how to bring in a module or just a particular function, which we'll highlight as we go through the semester.

To import Numpy we'll use the import function and then give it an alias, or shorthand name, that we'll use to reference the particular components of that module. By doing so we'll protect ourselves against having two functions with the same name with one silently "overwriting" the other.

```
import numpy as np
```

This is the most common way you'll see the Numpy module imported into a notebook or script.

```
import numpy as np
```

Create simple array The function `array` can take a Python list that contains all of the same type (e.g., integers, floats, strings). To access the function we'll need to use the alias that we created when importing the module `np`.

```
a_array = np.array([1, 2, 3, 4, 5])
print(a_array[4])
```

Now with this "true" array, we can do math and get the expected output with it operating on each element and not doing some concatenation, which is what we had with native Python lists.

```
print(a_array*3)
```

0.2.2 Python Program Control Flow

So far we have learned computer programming that only works in a linear fashion. There are times when we may want to only have a program do certain tasks under certain conditions or do things repetitively with small variations. These programatic elements enhance our ability to make Python scripts that work for a wider array of circumstances. In this chapter we'll look at using if statements and looping to control the flow of Python scripts.

If-Then Statements

There will be times when we only want to do things only when certain conditions are met. In order to facilitate this within a programming language we need a way for the program to know the "truthiness" of a statement.

If statements are used for when you want your code to be able to make a choice based on some condition. This will allow your code to be more flexible to create a wider variety of outcomes from running that same code.

The manner in which these statements work is by testing a truth value of a statment. If the statement is true, then the code within the if statement will be executed.

Logical Operators In order test values we need a set of operators that can help the computer determine the "truthiness". Here are the list of the most common logical operators used in Python.

symbol	meaning
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal
==	equal to
!=	not equal to
`and`	
`or`	
`not`	
|	bitwise or
&	bitwise and

The non-bitwise `and`, `or`, and `not` evaluate the truthiness of the statements it is comparing, whereas the `&` and `|` operators work on the byte representations of any value to assess its truthiness. More likely than not, it will be best to use the non-bitwise versions.

NOTE: zero or empty objects are `False`, all other values and non-empty objects are `True`)

Let's test out the operators...

```
# Examine truthiness of statement
print(5 < 10)
```

If, Else If, Else Statements At a minimum you must include a single `if` call, but can then include any number of `elif` statements to check other possibilities, and then a single concluding `else` statement can be used as a catch-all for the if statement (with no need to have any `elif`s to have an `else`). Here is a full example of using `if`, `elif`, and `else` in a single if statement. Notice that at the end of each line contain a part of the if construct there is a colon (`:`), that is the important character for the Python interpreter to know it has reached the end of the logical statment and it should be evaluted to determine its truithiness. Also note that the `else` statement does not contain any logical condition as it only gets executed if each test before it results in a `False` designation.

```
a = 5
b = 10

if a < b:
    print(a)
elif a == b:
    print('a and b are equal')
else:
    print(b)
```

Output:

```
5
```

NOTE: Another key element of Python is that is is an indented language, so everything within the `if` statment and any `elif` or `else` statments must be indented the same amount. The standard indent is four spaces long.

if statment construct - stand alone Here is the construct of a simple if statment where you are looking to test a single conditional element (e.g., 5 <10) to then choose to take a specific action (whatever is in the code block indented with the if statement) if it is `True`.

```
if condition:
    [code block]

# If -Then Statement Example 1
from datetime import datetime, timedelta

current_date = datetime.utcnow()
print(f'Original Time: {current_date}')

minute = current_date.minute

if minute < 15:
    current_date -= timedelta(hours=1)
print(f'Time after If -Statement: {current_date}')
```

if-else construct If you want to test one conditional, but if it is `False`, always do something else, then the `if-else` construct is for you! Note that in the `else` there is no conditional statement as it is a catch-all and will always be executed if the initial conditional is `False`.

```
if condition:
    [code block]
else:
    [default code block]

hunger_level = 5
if hunger_level >= 6:
    print('Find something to eat, now!')
else:
    print('You can wait a little while to eat.')
```

if-elif-else construct The full complement of an if statement can contain one `if`, any number of `elifs`, and an `else` statment. Again, once a `True` condition is found it will run the code block associated with that condition and then move on in the program after the if statment.

```
if condition1:
    [code block one]
elif condition2:
    [code block two]
elif condition3:
    [code block three]
else:
    [default code block]
```

```
# if -elif -else statement

grade = 75

if grade >= 85:
    print('You got a great score')
elif grade >=70:
    print('You got a good score')
else:
    print("Let's work together to improve your score on a future assignment.")
```

Looping for Repetitive Tasks

There are many times when you'll want to do a repetative task (e.g., perform a calculation 100 or 1000 times) and using programming is a way to make that task more effecient. Iteration is a key component to any programming language due to the need to process data in a time effecient manner and Python offeres two different methods for looping (iterating) over a series of values: `for` loops and `while` loops.

For Loops In Python, the `for` loop is the most extensible method for iterating over an object. You are not relegated to just looping over a series of integer values, but can loop over a mixed list or other iterable object (e.g., dictionary keys). The construct of a `for` loop is:

```
for var in iter_obj:
    [code block]
```

Again, the syntatical structure of Python using the color at the end of the `for` loop line and everything that you want done with each cycle of the loop needs to be indented.

Common Iterable Objects:

- `range()` function
- List
- Tuple
- Dictionary Keys

> **Note**
> A common way to create a quick for loop over a set of sequential values is to set up a range of values using the built-in range function. One note is that the range function is inclusive of the first value and exclusive of the second.
>
> `range(0, 10) -> [0, 1, 2, 3, 4 ,5 , 6, 7, 8, 9]`

So how does the loop work?

For each element in the iterable object (e.g., `[0, 1, 2, 3, 4]`) the variable `var`, which is defined as the looping variable, assumes the values in sequence. So initially, `var` will have the value of `0`, it holds that while working through all of the code within the loop, then once you hit the end of the loop you go back to the beginning and `var` takes on the next value. This continues until all values in the iterable object are exhausted.

Let's explore an example to see how it works...

```
for var in [0, 1, 2, 3, 4]:
    print(var)
    if var < 1:
        print(f'{var} is less than 1')
    else:
        print(f'{var} is greater than or equal to 1')
print('Out of loop')
```

Output:

```
0
0 is less than 1
1
1 is greater than or equal to 1
2
```

```
2 is greater than or equal to 1
3
3 is greater than or equal to 1
4
4 is greater than or equal to 1
Out of loop

from datetime import datetime, timedelta

# For loop example 1

for i in range(0, 10):
    print(f'The value of i is {i}')

print()
print('I am now out of the loop!')

# For loop example 2

for time in [1215, 1245, 1315, 1345, 1415]:
    print(time)

# For loop example 3

for var in ['Kevin', 1859, 2.5, 'Keaton']:
    print(f'The varible is {var}')
```

While Loops There may be times you want to continue looping until a condition is met. Here we have a simple example that starts a loop at zero and goes to 11. Basically we are combining looping with testing the truth value of a statement. As long as the statement remains `True`, the loop will continue going. This is why it is important to ensure that the looping variable changes in some manner that can be eventually evaluated as `False`, so that you don't create an infinite loop!

```
# While loop example

# Set the starting loop value
inc = 0

# Check the value compaired to max value
while inc < 12:
    # Print current increment value
    print(inc)
    # Update value
    inc += 1
```

0.2.3 Datetime Module

When working with data, especially with meteorological data, it is a requirement to know and be able to work with dates. As a part of base Python, there is an important module called `datetime` that contains a number of useful pieces that make working with dates easier. In this chapter we'll look at some of the most relevant portions of the module that will be utilized throughout this book. One very important is that through this module we can allow the computer to set the current date (or really any date relative to today) for us. In addition, we'll learn about other tools contained within the module that will allow us to find and set dates as needed to work with various types of data.

Datetime Functionality

The datetime module is fully described in the Python online documentation that you can access at Python Docs: datetime . We'll begin by importing the three different portions of the module that we are going to use at various points throughout the book, `datetime`, `timedelta`, `time`, and 'UTC'.

```
from datetime import datetime, timedelta, time, UTC
```

datetime objects Now let's learn about and use the datetime objects from the datetime module. In meteorology we can't concern ourselves with every local time zone, especially when various locations have daylight savings times and other locations don't. As a result, meteorologists use a common time coordinate called Universal time (UTC or Zulu). This time never changes, there is no daylight savings time and it is the same at every location around the globe. To get the time in UTC

```
print(datetime.now(UTC))
```

The following should be similar to the output that you see on your screen:
`2020-01-01 21:53:38.422386`
This is the default print format of the datetime object,
`YYYY-MM-DD HH:MM:SS.ms`
By default in Python >=3.11, there is timezone information attached to the `datetime.now(UTC)`. This won't affect most operations, but at the end of this notebook the method to remove that timezone information will be reviewed to allow seamless use with some meteorological data.

Now let's save it to a variable name to be more useful:

```
now = datetime.now(UTC)
```

The new variable we created (called `now`) is really so much more than simply holding the time; it is an object that contains various methods for access information from our stored computer time. For example, we may not want to use all of the information given in the default output from the date command, like we had above. Using the various methods and/or formatting statements will allow us to specify what information we want (e.g., year, month, day) and get its numeric or character value. Here are a couple of common methods for getting numeric elements out of our datetime object:

```
now.day
now.hour
now.minute
now.second
```

```
print(now)
print(now.year)
```

To get a particular date format using the following list of format types (all begin with a percent sign; Python Docs: strftime-and-strptime-format-codes). Here we are going to do an example using the f-string (formatted string):

```
print(f'The current year is {now:%Y}')
```

You can combine the format statements to form strings, for example, write a Python print statement to get the year, month, and day in one continuous string (`YYYYMMDD`):

What is the result of the date function you wrote?

How about printing out the Month in a word abbreviation form?

Any of the Python statements given above can be assigned to a variable name to be used anywhere within your program. We'll continue to come back to aspects of the datetime objects throughout the semester.

Set your own time We are not beholden to only using the current date/time – we can set any date/time we want. For example, let's set a date of May 23, 2018 at 12 UTC, we can use the `datetime` module again by setting the the year, month, day, hour, minute, second.

> **Note**
> When setting your own time, the year, month, and day are required (in that exact order) and it no hour, minute, or second then they will be assumed to be zero.

```
# Set your own time
date = datetime(2018, 5, 23, 12)
print(date)
```

timedelta objects There will be moments where you'll want to do some math with the date (e.g., what time is 6 hours from now?) and timedelta objects makes that arithmatic very easy. Using the `timedelta` functionality we can add or substract days, seconds, microseconds, milliseconds, minutes, hours, or weeks.

Let's try this out by finding what was the datetime 12 hours ago (subtraction) and what will it be two weeks from now (addition):

```
# 12 hours ago
print(now - timedelta(hours=12))
```

So above, we used our previously defined current time, `now`, and substracted using `timedelta` twelve hours. It was seamless for the combination of the `datetime` and `timedelta` objects to know that it had to role back one date in addition to subtracting the correct number of hours.

```
future_time = now + timedelta(weeks=2)
print(future_time)
```

Here we saved our time calculation to a new variable, `future_time`, and printed it out. Again, we let the computer do the hard work for us.

time objects The time objects deal with only the local time (hours, minutes, seconds, etc.) and are a subset of the `datetime` objects. We can use this to help us set a datetime object more efficiently when we may want to get a relatively current date, but set the hour to a specific time.

Let's find the date of yesterday and set the time to 12 UTC.

```
# Use datetime and timedelta to get the date of yesterday
yesterday = datetime.now(UTC) - timedelta(days=1)
print(yesterday)

# Use time to set a 12 UTC time
hour = time(12)
print(hour)

# Use the combine method to set a new datetime object
new_date = datetime.combine(yesterday, hour)
print(new_date)
```

Removing timezone information from datetime object There are times where meteorological data do not store the timezone information, although we know that the data are stored in UTC. There is just a small bit of extra code that you need to be able to remove the timezone information from a datetime object.

```
yesterday
```

To remove the timezone information, we just need to use the replace method on the datetime object...

```
yesterday_notz = yesterday.replace(tzinfo=None)
print(yesterday)
```

Now the new object, `yesterday_notz`, is a timezone naive object, which will allow for time matching when timezone information is not a part of meteorological data.

0.2.4 Python Script

A Python script is essentially a list of statements entered into a text file that will be executed by the Python interpreter in the order in which they are written in the file. A key benefit of using a script is to define variables and not having to type them out every time that you want to perform a series of tasks to create a desired output in a notebook environment.

It is relatively easy to use some Python scripting to ease various tasks, including map making. During this section we will see how you can turn a notebook file into a script to make it easier to reproduce the desired output with less effort than opening a notebook file and running all of the cells.

Hello World Script

Let's start very simply by creating a simple Python script that prints the statement 'Hello World!'!

Open a new file using vi by typing the following in a new terminal window:

`kgoebber@bergeron:~>vi script1.py`

Now add the following code to your script, save the output, and quit the file.

```
# Print the following statement to standard output

print('Hello World')
```

Once you are back at the terminal prompt we now want to run our script. To run a python script we want to run our file through the Python interpreter which we can do by typing the following in the terminal:

`kgoebber@bergeron:~>python script1.py`

The output should be:

`Hello World!`

And that's it! You just wrote and ran your first Python script!

Now let's develop a bit more into a simple script.

Building a full Python script

Open a new text file from the terminal window (or the terminal from the Jupyter Lab Launcher window) using vi (or any other text editing tool you might know) and call it ***my_script.py***. Once the new file is open the first thing you will want to add some lines that describe the script and what it does. This is called a comment block. Recall that a comment in Python code begins with the pound (hashtag) sign (`#`). You may add something like the following for your initial comment block:

```
#############################################################
# my_script.py
# By: Kevin Goebbert
#
# This is a simple test script. It uses basic
# Python datetime commands to do a few simple
# tasks.
#
# Modifications:
#
#############################################################
```

All of the previous is just for your information, so that you can quickly understand what you were trying to do with this particular script. Now let's add something to the script to make it do something!

```
from datetime import datetime

# This is a comment and not executed
```

```
print("Hello, this is my first script.")

# Get current time from computer

now = datetime.now()

print(f"The current time is {now}")

# Get current time in UTC from computer

now_utc = datetime.utcnow()

print(f"The current time in UTC is {now_utc}")

print(f"The current month is {now_utc:%B}")

print("All done")
```

After you enter all of the above, *write the file and quit* to get back to your command prompt.

> **Note**
> Be careful with all of these quote marks. Be sure to have balance single/double quotes. If in one of the print statements we need to use nested quote marks. It doesn't matter which is used where, just that you use different ones within the nest from what was used on the outer quote marks.

Again, recall that in order to execute a Python script, type the following on your terminal window at the command prompt

`kgoebber@bergeron:~>python <yourfile>`

where `<yourfile>` in this name that you gave to your script file (e.g., *example_script.py*). Go ahead and run the file.

What happened? Run it again, did anything change?

0.2.5 Runtime Control with Scripts

There are numerous ways to control what and how Python scripts run and move through various tasks to generate the needed analysis/outcome that is coded within your program. There are many instances where you have essentially the same task that you want to perform, but just on a different set of data. Instead of going in and modifying a script to be able to read in a certain data file, a program can be set up to either ask for input or an argument could be added when going to run the script. Either way can make it easier to accomplish needed tasks without endlessly modifying individual pieces of code.

Interactive

One way to control a script at runtime to make it interactive (i.e., where it will ask you a question and record your response to run the script). Below is an example script that you can put into a new text file to demonstrate how interactive scripting can work.

The script first prints the question or statement, then awaits a response. The `input()` function contains the text that is printed to the screen variable and will ingest the value submitted from the command line that you type in and hit enter, finally setting the value of the variable to the left of the equal sign to whatever value was input. Create and run the script (name it `ex_interactive.py`) below and plug in your first name, for the first input, and your last name, for the second input to test how interactive scripting can work.

```
# Ask for input and store in variable
firstline = input('Go ahead: ')
print(f'You entered: {firstline}\n')

# Ask for second input and store in variable
secondline = input('Go ahead (2nd time): ')
print(f'You next entered: {secondline}\n')
```

Run the script from the Linux command prompt:
`kgoebber@bergeron:~>python ex_interactive.py`
Example Input/Output:

```
Go ahead: Kevin
You entered: Kevin

Go ahead (2nd time): Goebbert
You next entered: Goebbert
```

A practical way of using an interactive script for helping make a map is to have a script ask you which region you want to plot for a surface map.

Arguments

Another way to control script workflow is to give the script some information from the command line when you call the script. In the example using the while loop below we set the variable `total` using the first system argument value (i.e., `sys.argv[1]` in the code below) which allows the users to tell the script how many iterations we wanted to loop over at run time.

```
import sys

# Get value from runtime argument
total = int(sys.argv[1])

# Set initial value
```

```
inc = 0
while inc < total:
    print(inc)
    # increment variable value by adding 1
    inc += 1
```

Note
In order to use the system argument we need to import the sys library, which is part of the base Python language. The way in which the arguments are parsed are by separating the inputs via spaces.

Create a script using the code written above and call it `ex_while_argv.py`. Then to run the code you will need to supply one argument (a number) to allow the script to complete its work. Otherwise you will get an error. Try a couple of different numbers to see how the running of the program changes depending one the value used for the argument.

Run the script from the Linux command prompt:

`kgoebber@bergeron:~>python ex_while_argv.py 15`

You can also have more than one argument. The second argument would be the variable `sys.argv[2]` and the third argument `sys.argv[3]`, and so on. Essentially, you are accessing the contents of a list that contains everything that comes after the initial `python` call. So in the example running the `ex_while_argv.py` file above, the system argument list would be `['ex_while_argv.py', '15']`. Recall that list addresses begin with zero, so that the first "usable" argument will be at address one (i.e., `sys.argv[1]`). A practical situation for using an argument with a script for making weather maps would be to use them in conjunction with if-then statements to allow runtime control to choose different settings for a chosen upper-air maps level (e.g., 850, 500, 300 hPa).

0.3 Meteorological Concepts, Data Analysis, and Visualization using MetPy

0.3.1 Meteorological Python

MetPy is a Python module that contains domain specific functionality (e.g., plotting surface observations in the above describe station model format) for use in the Python programming language. This is a relatively new module and has just recently passed its 1.0 version (meaning it has reached at least a base-level of maturity). Throughout learning how to use MetPy you will be introduced to some of the basics of the Python programming language, but this is not strictly a course that will teach you Python. Primarily you'll use the declarative syntax from MetPy to plot various types of meteorological observations and model output. Remember that whatever you tell Python to do in your code, whether it was what you intended or not, is what it is going to do! So be sure to check what you have typed in – if you are not getting a map that you think you should be getting (or get an error), it is most likely a human input error (an i-d-10-t error, as they say).

Important Notes (before we get started):

MetPy Website: https://unidata.github.io/MetPy/latest/index.html

This website has all the documentation for the current version of MetPy. You will only be introduced to what small bits of MetPy can do here today; the reference guide will go into more detail about all of the different aspects of the module that can be used to plot observational data and model output.

OR

There will be ways to get some of the help information from the development environment you will use throughout the semester (jupyter notebooks). More on that as we go along!

As a **last resort** you can always ask someone who has a more experience using Python, Jupyter notebooks/lab, and MetPy, as they will probably have seen and dealt with many of the problems that you might be having.

0.3.2 Surface Obs and Plotting

Surface Data

The most basic of atmospheric measurements started at the surface. Before the advent of instruments like the thermometer and barometer (yes; there were times when we didn't have these basic instruments), people recorded whether the weather was warm or cold. Unfortunately, humans can only detect a minimum change in temperature of around 5řF. Since we live on Earth's surface, it is easiest to get our measurements there. But the more important question may be, what variables are important and *why*?

What do you think are the most important surface variables and why?

A goal for every meteorologist is to be able to explain the variations in atmospheric parameters that we observe. In this section we will focus on the surface variables and their spatial variation. There are many variables that are important and we have to somehow get all of them plotted on a single map, for multiple stations. Doing so will benefit a meteorologist immensely because there will be a wealth of information literally at their fingertips. The following section describes how we can compactly write the surface data in a concise format on a weather map.

Review of Surface Station Models

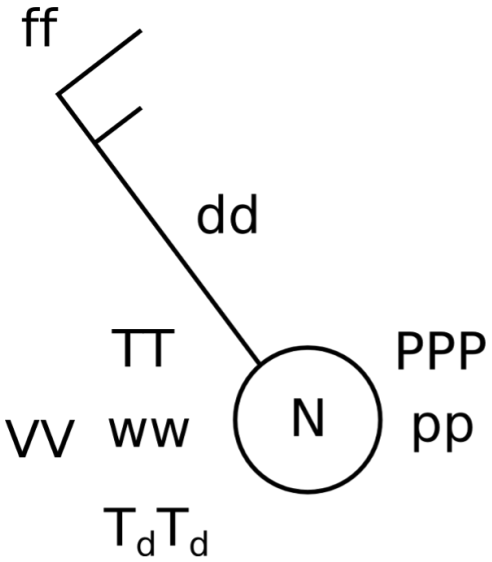


Figure 4: This is the example station model, each element is described below.

SURFACE STATION MODEL **ff** is the "flag". It represents the wind speed. A half stick represents 5 kts, a full stick is 10 kts, and a filled in triangle is 50 kts.

dd is the wind direction. It always points in the direction FROM which the wind is blowing

TT is the temperature. In the United States it is typically in řF while in every other part of the world it typically is řC (Assume řF unless otherwise specified for this class.)

VV is the visibility in statute miles.

ww is the present weather condition. This is typically a symbol. Please see later portions of this hand out for what these symbols are.

$\mathbf{T_dT_d}$ is the dewpoint temperature. It follows the same unit convention as the temperature.

PPP is the sea-level pressure to the nearest tenth of a unit. The units are typically hectopascals (hPa), which is equivalent to millibars (mb). You need to add either a 10 or a 9 to the front end of this given number (whichever beings the pressure closer to 1000).

pp is the pressure tendency over the previous 3 hours given in tenths of a unit. Typically a "slash" is present as well indicating if the pressure is rising (/) or falling (\).

N is the sky cover in oktas (eighths).

Clear = A circle with nothing in it.
1 Okta = A circle with a single vertical line in the middle
2 Oktas = A circle with the upper right quadrant filled in.
3 Oktas = A circle with a single vertical line in the middle AND the upper right quadrant filled in.
4 Oktas = A circle with the right half filled in
5 Oktas = A circle with a single horizontal line in the middle AND the right half filled in.
6 Oktas = A circle with the right half and bottom right quadrant filled in.
7 Oktas = A circle that is filled in EXCEPT a single vertical line in the middle.
8 Oktas = A circle that is filled in completely.
Obscured Sky = A circle with an X in it.

Please note that this is not all-inclusive. There is additional information that can be displayed on a surface station plot; however, this covers the basics.

Surface Observation Chart

Making and reading our surface charts are important, as it is essential to read them quickly to diagnose the constantly changing environment. A surface map can tell you a lot of information about the current environment. Observations near big bodies of water will likely have differing characteristics than stations just a little farther inland. Additionally, there will be differences between observations near cities versus out in the middle of cornfields. These variations in meteorological parameters could be very important to the future evolution of the weather at a given location. Therefore, it is important to completely understand how surface maps are created and be able to explain meteorological differences between different regions.

Example Surface Map

Here is an example of a surface map (Figure 5) containing many surface station models (Figure 4) representing surface variables using the MetPy declarative syntax. Notice how we can change the colors of elements of the station model to be able to more easily read the disparate pieces of information.

Figure 5: Midwest surface observations at 0000 UTC 23 February 2023.

Now that we have learned a little bit about surface station models, let's make our own plots using the MetPy declarative syntax.

Common Notebook Structure

In this section the main elements and structure of Jupyter notebooks to produce an analysis and visualization of meteorological data will be described. There are

Import Statements Python is the base language that is now commonly used within the broader scientific enterprise for conducting data analysis and visualization. By design, the base language is relatively small, providing intuitive control structures that are amenable to a large variety of computing problems. On top of this base is built an entire ecosystem of packages or modules that provide domain-specific methods for the calculation and visualization of data. In order to incorporate those modules into any Python script, one must import those elements.

datetime

- A built-in Python module (part of base Python) to help handle date and time issues. Both the datetime and timedelta portions of the module are brought in here.

metpy

- A module developed to add functionality specific for the atmospheric science/meteorology scientific community. Here we bring in three elements of the module, the METAR reader, the declarative plotting interface, and the units module.

other modules

- There can be any number of other imports that bring in functionality to read in data, perform data transformations, or alter plotting in some way. Additional import modules will be described the first time they are used in a particular section.

Read Data Before we can work with any data within our notebook, data must first be brought in. In this instance we are using MetPy's ability to read a file containing METAR codes (a long-serving surface data code developed for aviation purposes). Once we read in the data and have it saved to a namespace (in this case df) we can then use it at some later point in the notebook.

Calculate/Transform Data After data are read into the notebook, then the data are ready to be used in any calculations that might need to be performed or refactored in some way to make it useful for plotting and analysis. This section may not be needed if data are in the correct format and contain all necessary values to be plotted.

Plot Data This is the main part of the code where you get to describe what and how you want to plot some (or all) of the data that you have read into the notebook. Details of this declarative interface is given below. The result will be a map of observed data cover a part of the world.

Surface Data and Plotting using MetPy

This notebook demonstrates reading surface data and plotting it using MetPy's declarative syntax.

To make a plot of surface observations requires three elements

- `PlotObs()` to specify the observations to be plotted
- `MapPanel()` to specify the characteristics of the map to plot the observations on
- `PanelContainer()` to collect one or more panels.

These three elements are separate parts of the declarative module from MetPy. In Python we call these parts Classes and note that they contain MiXeD case names. These classes contain a number of attributes that can be set to specify what and how to plot the data in a figure. The descriptions of some of the basic elements of each element are given below.

Import Modules Here we'll import those modules that are needed to produce read in remote surface METAR observation data, process those observations, and plot the values on a map.

```
from datetime import datetime, timedelta, UTC
from io import StringIO
from urllib.request import urlopen

from metpy.io import metar
from metpy.plots import declarative
from metpy.units import units
import pandas as pd
```

Read Data The surface data are in METAR format and store the data files at this location for approximately two weeks.

The format of the filenames are `YYYYMMDDHH_sao.wmo` where `YYYY` is the year, `MM` is the month, `DD` is the day, and `HH` is the hour.

```
# Get the current date/time and remove the timezone information
date = datetime.now(UTC).replace(tzinfo=None)

# Remote Access - Archive Data Read with pandas from Iowa State Archive,
# note differences from METAR files
dt = timedelta(minutes=30)
sdate = date - dt
edate = date + dt
data_url = ('http://mesonet.agron.iastate.edu/cgi -bin/request/asos.py?'
            'data=all&tz=Etc/UTC&format=comma&latlon=yes&'
            f'year1={sdate.year}&month1={sdate.month}&day1={sdate.day}'
            f'&hour1={sdate.hour}&minute1={sdate.minute}&'
            f'year2={edate.year}&month2={edate.month}&day2={edate.day}'
            f'&hour2={edate.hour}&minute2={edate.minute}')
data = pd.read_csv(data_url, skiprows=5, na_values=['M'],
                   low_memory=False).replace('T', 0.00001).groupby('station').tail(1)
df = metar.parse_metar_file(StringIO('\n'.join(val for val in data.metar)),
                            year=date.year, month=date.month)
# Local Access
# data = f'/data/ldmdata/surface/sao/{date:%Y%m%d%H}_sao.wmo'
# df = metar.parse_metar_file(data, year=date.year, month=date.month)
```

Plot Data Here we seek to use the data that were just read in to plot the `air_temperature` variable in units of Fahrenheit. Since surface observations can be associated with a range of times, we can set

the `time_window` attribute to look over a range of times. These temperatures will be plotted on a map over a small geographic location and a title is added to the map.

```
# Plot desired data
obs = declarative.PlotObs()
obs.data = df
obs.time = date
obs.time_window = timedelta(minutes=30)
obs.fields = ['air_temperature']
obs.plot_units = ['degF']

# Panel for plot with Map features
panel = declarative.MapPanel()
panel.layout = (1, 1, 1)
panel.projection = 'lcc'
panel.area = 'in'
panel.layers = ['states']
panel.title = f'Air temperature on {date:%Y%m%d} at {date:%H}UTC'
panel.plots = [obs]

# Bringing it all together
pc = declarative.PanelContainer()
pc.size = (10, 10)
pc.panels = [panel]

pc.show()
```

Note

The use of formatted strings (f-strings) are used extensively in this book. Use of these strings allows the easy input of variables into a string value to dynamically generate the string. Variables can also be formatted (depending on type) to gain more control over how the values plot. In the above surface plot, f-strings are used to set the title where we use the variable `date` with date formatting to make a nice, reable title.

For more information on f-strings, see the Python documentation

Practice Tasks - Surface Observation

So, what did we set? Just the required and a few of the optional parameters to make a plot. Use the practice tasks below to explore more of the options.

1. Open a new notebook and make a plot of temperatures and dewpoints using different colors.
2. Plot the full surface station model (e.g., temperature, dewpoint, MSLP, current weather, and wind speed and direction) over Oklahoma.
3. Plot the temperature with temperatures equal to or less than 32 colored blue and greater than 32 colored red.

   Note: *You'll have to subset your DataFrame and plot to sets of observations. To subset a DataFrame it would look something like:*

   ```
   obs.data = df[df.air_temperature <= 0]
   ```

   which would only plot observations where the air temperature was less than or equal to 0.

Station Model Location Details

The layout of data around a station model is very important. At the beginning of this section we reviewed the typical layout for the display of meteorological variables around a surface station. In our plotting of surface observations (`PlotObs`) we can set everything in the same way. The following is the location where a variable will be placed (Figure 6), depending on the string at each location.

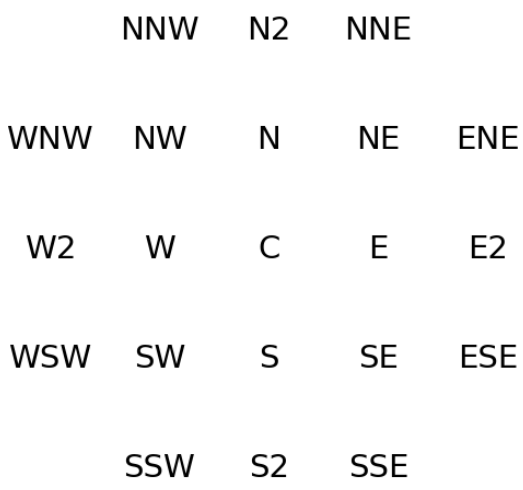


Figure 6: Graphic representation of the different possible locations around a surface station model using MetPy `PlotObs`.

The default location is C.

1. What variable should be plotted with the location of C (center)?
2. What about the W (west) location?
3. What about for a station model that you wanted to plot: temperature, dewpoint, mslp, current weather, and visibility?

Now let's work to plot a full suite of observation information for multiple stations.

Full Surface Station Model Plotting

Import Import the needed modules

```
from datetime import datetime, timedelta
from io import StringIO
from urllib.request import urlopen

from metpy.io import metar
from metpy.plots import declarative
from metpy.units import units
import pandas as pd
```

Read Data Here we gain access to surface data from the Iowa State Archive. To do so we construct the appropriate `data_url` that requires a start and end date. To create that we utilize the time we desire and add/subtract time from there to create a time window. From there we use the Pandas module method `read_csv()` to read in all of the data, then parse the METAR observations using MetPy.

```
date = datetime(2022, 1, 4, 12)

# Remote Access - Archive Data Read with pandas from Iowa State Archive,
# note differences from METAR files
dt = timedelta(minutes=15)
sdate = date - dt
edate = date + dt
data_url = ('http://mesonet.agron.iastate.edu/cgi -bin/request/asos.py?'
            'data=all&tz=Etc/UTC&format=comma&latlon=yes&'
            f'year1={sdate.year}&month1={sdate.month}&day1={sdate.day}'
            f'&hour1={sdate.hour}&minute1={sdate.minute}&'
            f'year2={edate.year}&month2={edate.month}&day2={edate.day}'
            f'&hour2={edate.hour}&minute2={edate.minute}')
data = pd.read_csv(data_url, skiprows=5, na_values=['M'],
                   low_memory=False).replace('T', 0.00001).groupby('station').tail(1)
df = metar.parse_metar_file(StringIO('\n'.join(val for val in data.metar)),
                            year=date.year, month=date.month)
df['date_time'] = date

df['tmpf'] = (df.air_temperature.values * units.degC).to('degF')
df['dwpf'] = (df.dew_point_temperature.values * units.degC).to('degF')
```

Plot Data Here we exploit the full range of observations to plot around our station model to get a "full" surface station plot. In order to plot cloud cover and the current weather, we must use the formats to specify the "font" to use, which is part of MetPy. We could go further and change colors, fontsize, etc., but that is left as an excercise for the reader.

```
mslp_formatter = lambda v: format(v*10, '.0f')[ -3:]

# Plot desired data
obs = declarative.PlotObs()
obs.data = df
obs.time = date
obs.time_window = timedelta(minutes=15)
obs.level = None
```

```
obs.fields = ['cloud_coverage', 'tmpf', 'dwpf',
              'air_pressure_at_sea_level', 'current_wx1_symbol']
obs.locations = ['C', 'NW', 'SW', 'NE', 'W']
obs.formats = ['sky_cover', None, None, mslp_formatter, 'current_weather']
obs.reduce_points = 0.75
obs.vector_field = ['eastward_wind', 'northward_wind']

# Panel for plot with Map features
panel = declarative.MapPanel()
panel.layout = (1, 1, 1)
panel.projection = 'lcc'
panel.area = 'in'
panel.layers = ['states']
panel.plots = [obs]

# Bringing it all together
pc = declarative.PanelContainer()
pc.size = (10, 10)
pc.panels = [panel]

pc.show()
```

Saving Images

Previously we have created maps, but they were only visible within the Jupyter environment. What if I want to save the image that I created? There is only one small addition to make to your cell where you create your image:

```
pc.save('test.png')
```

This will save the same image that is shown in your notebook to the local working directory (likely your working directory). It is saving to the name you give the file between the quote marks (in this case **test.png**). That's it. It really is that simple.

There are a few different types of images you can save as, the most common being the PNG (portable network graphic), but you can also save as a JPG, PDF, PS, and TIF. Just switch out the .png for the desired graphical output type.

Additionally, you can set other parameters about how the image will save. We'll highlight two of them here, but for all of the available options go the Matplotlib help pages for savefig

Common (Good) Options

dpi

- This setting is to allow you to specify a "dots per inch", which will give you a nicer looking image with a higher value. Common value to get a good image is 150.

bbox_inches

- This setting is set to 'tight' to remove excessive white space surrounding the image. It is best that this always be set to True (the default is false).

So, putting it all together, a good line for saving a file is:

```
pc.save('test.png', dpi=150, bbox_inches='tight')
```

Simply change the name within the quote marks to save to a different name (good habit is to name the image something unique each time so as not to accidentally overwrite something you really want to keep!).

Downloading Images To download an image to your local machine, find the file you wish to download (most likely an image file), right-click on the file and a dialogue box should appear and find the option to "Download". Depending on the settings of your web browser, you'll either be asked where to save the file, or it will save to your Downloads (or other designated) folder.

Map Making Summary

Summary of process for making a map:

1. Open Jupyter Lab
2. Open an existing or new notebook
3. Set up all parameters to read data and make a map
4. Run all cells to generate image
5. Close and shutdown notebooks when done for the day (cleanup)

Note: You may have multiple `PlotObs()` if you want to layer different things on the same map before adding them to a panel and a figure.

0.3.3 Upperair Obs and Plotting

At least twice a day, at stations all over the world, radiosondes are launched to report in situ measurements of temperature and moisture at different pressures in order to allow meteorologists to know what atmospheric conditions are above the surface. As a result of these launches there are a number of standard plots that are created. The maps represent the quasi-horizontal pressure surfaces (e.g., 850, 700, 500, 300, 250 hPa) where everywhere on the map the pressure is the same; however, the height at which the pressure is at varies greatly. Like the surface station model, we plot a lot of information around the upper-air station model.

Table 1: Decoding upperair station model

Parameter Reference	Description
FF	The wind speed. Typically reported in knots, but can be mph or m s^{-1}.
WD	The wind direction. Indicated by the stem, the wind is always described by where the wind is coming from. The depiction above would be a west wind.
TT	The temperature in Celsius (different from the surface model)
DD	The dewpoint depression or spread between the temperature and the dewpoint. For example at 500 hPa, DD = T(500) - Td(500) If the dewpoint is the same as the temperature (i.e., the atmosphere is saturated at 500 hPa), then the dewpoint depression (DD) is zero. Very Moist Air DD = 0 Very Dry Air DD >= 30
HHH	The height that you must go (referenced to mean sea level) up in order to reach the stated pressure level at the given location. This is in a code. See Table Figure 2 below for a description of how to interpret at different pressure levels.

Upperair Station Model

The upperair station model uses the same basic structure that is done for the surface, including where the mass and temperature fields go and wind speed and direction. However, there are a lot less data taken since we only collect temperature, moisture, and pressure values directly and infer wind speed and direction through GPS location differencing. The basic structure of the upperair station model is given in Figure 7 with full descriptions given in the table below the figure.

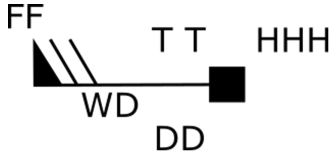


Figure 7: Graphic representation of the parameters plotted around the upperair station model.

To decode the three-digit height values that are plotted on an upperair map you can use your knowledge of common atmospheric conditions at different levels with this handy table to be able to add in the left off values to make the plotted observations more compact. Note how at the highest levels of the troposphere (e.g., 250 and 200 hPa) there are two missing values, one at the beginning and end of the plotted value for the station model.

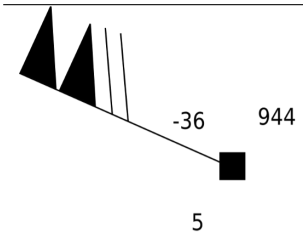
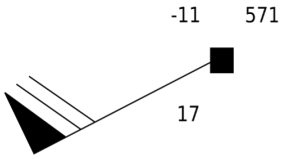
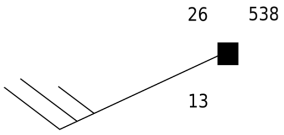
* Use the value that will bring the plotted value closest to 3000 meters.

Pressure (hPa)	Prefix to Plotted Value	Suffix to Plotted Value	Plotted	Height (m)
850	1	–	530	1530
700	2 or 3*	–	180	3180
500	–	0	582	5820
300	–	0	948	9480
250	1	0	028	10280
200	1	0	164	11640

Table 2: Table 1Decoding upperair heights

Practice Decoding Upperair Station Models

1. Decode the following upper-air station plots:

Observation 1	Observation 2	Observation 3
		
Temp: _____	Temp: _____	Temp: _____
Dewpoint Dep.: _____	Dewpoint Dep.: _____	Dewpoint Dep.: _____
Dewpoint: _____	Dewpoint: _____	Dewpoint: _____
Wind Speed: _____	Wind Speed: _____	Wind Speed: _____
Wind Direction: _____	Wind Direction: _____	Wind Direction: _____
Pressure Level: _____	Pressure Level: _____	Pressure Level: _____
Height: _____	Height: _____	Height: _____

Map Projections

There are a number of problems when it comes to representing a sphere on a two-dimensional surface. As a result, there are any number of ways that you can project a sphere onto a two-dimensional surface; each projection has positive and negative elements. In order to choose which projection is best for your needs, you'll have to make a compromise about something and the goal is to compromise on something that you are the least concerned about.

For example, plotting a meteorological map we are not concerning ourselves with being able to navigate using the map, so having true latitude and longitude lines is a luxury that we can afford to do without. Something more important for our purposes would be to represent the shape of landmasses as well as possible.

Plate Carrée This is a cylindrical projection that offers a simple way to view data that have geographic coordinates (e.g., latitude and longitude coordinates). The latitude and longitude lines intersect at right angles and produces rather large distortions of geographic area at the poles (similar but not quite as extreme as a Mercator projection). This is a reasonable default projection (Figure 8) when you are first inspecting data, but better choices can be made depending on your data visualization and analysis needs.

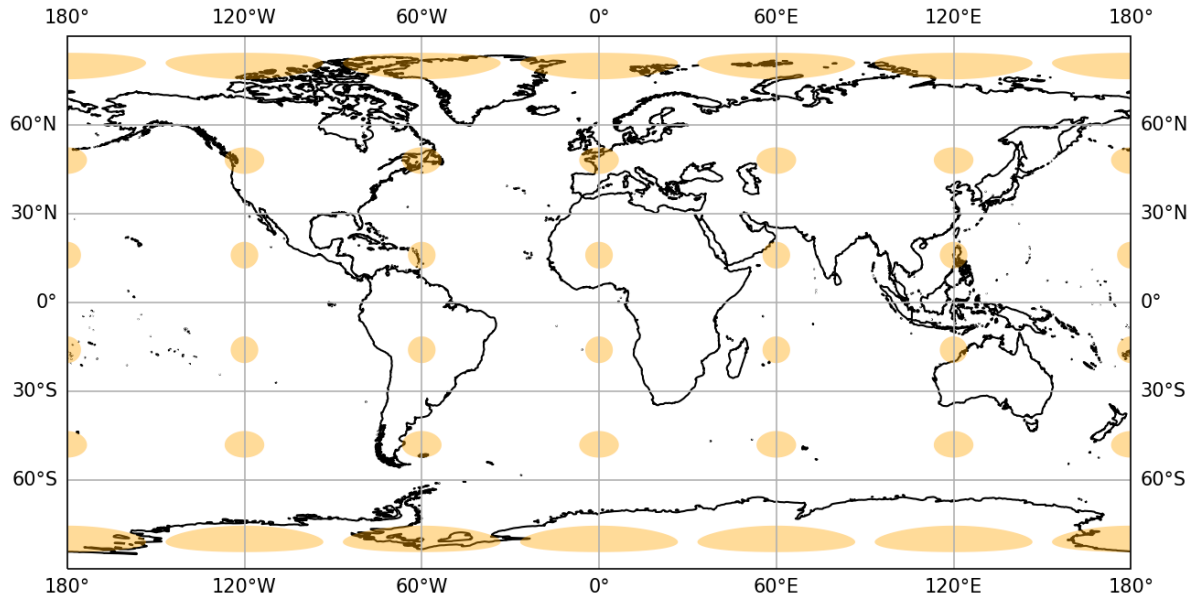


Figure 8: Representation of the Plate Carrée projection using Cartopy with the Tissot indicatrix plotted in orange showing the map distortion due to the projection.

Polar Stereographic Many early maps were focused on navigation and the countries in the mid-latitude regions of the Northern Hemisphere. As a result, certain areas of the globe were generally not well mapped. One of the more difficult areas to map is the Polar Regions of the two hemispheres. These areas are problematic as lines of longitude converge to a single point at the pole. These difficulties were overcome by projecting the features of the sphere onto a map with the center placed over the pole. A stereographic projection is an azimuthal projection coordinate and an example of the Northern Hemispheric stereographic projection is given below (Figure 9). Such that the middle of the map is the north pole and as you move away from that point you move toward the equator. A similar map can be made looking at the south pole. This is a great map if you are looking to represent an entire hemisphere that has considerably less distortion of area. An example of its usage in meteorology would be for a hemispheric plot of 300-hPa heights to determine the wavenumber of the hemispheric pattern.

For more information on the mathematics behind the polar stereographic projection, checkout the online article at Wolfram MathWorld.

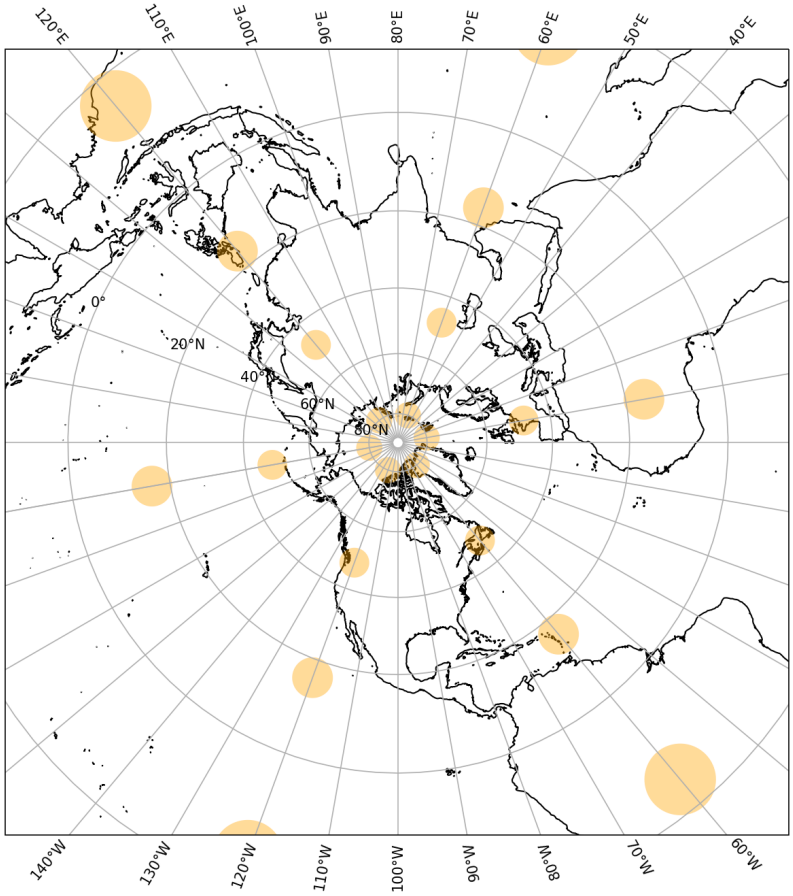


Figure 9: Representation of the Polar Stereographic projection focused on the North Pole with a central longitude of -100ŕE using Cartopy with the Tissot indicatrix plotted in orange showing the map distortion due to the projection.

Lambert Conformal So far we have briefly looked at the Plate Carree and Stereographic projections, which have their own strengths and weaknesses for meteorological purposes in the mid-latitude regions. A better projection for plotting data focused on mid-latitude regions is based on conic principles and has been widely used since the First World War (Figure 10). The primary conic projection used is called the Lambert Conic Conformal (LCC) Projection. The conic conformal projection, when constructed, essentially forms a cone around the globe, which results in minimal distortion through much of the mapped area. This can only be used for regional perspectives (e.g., the CONUS) as specification of a central longitude and parallel latitudes will determine the minimal distorted area of the map.

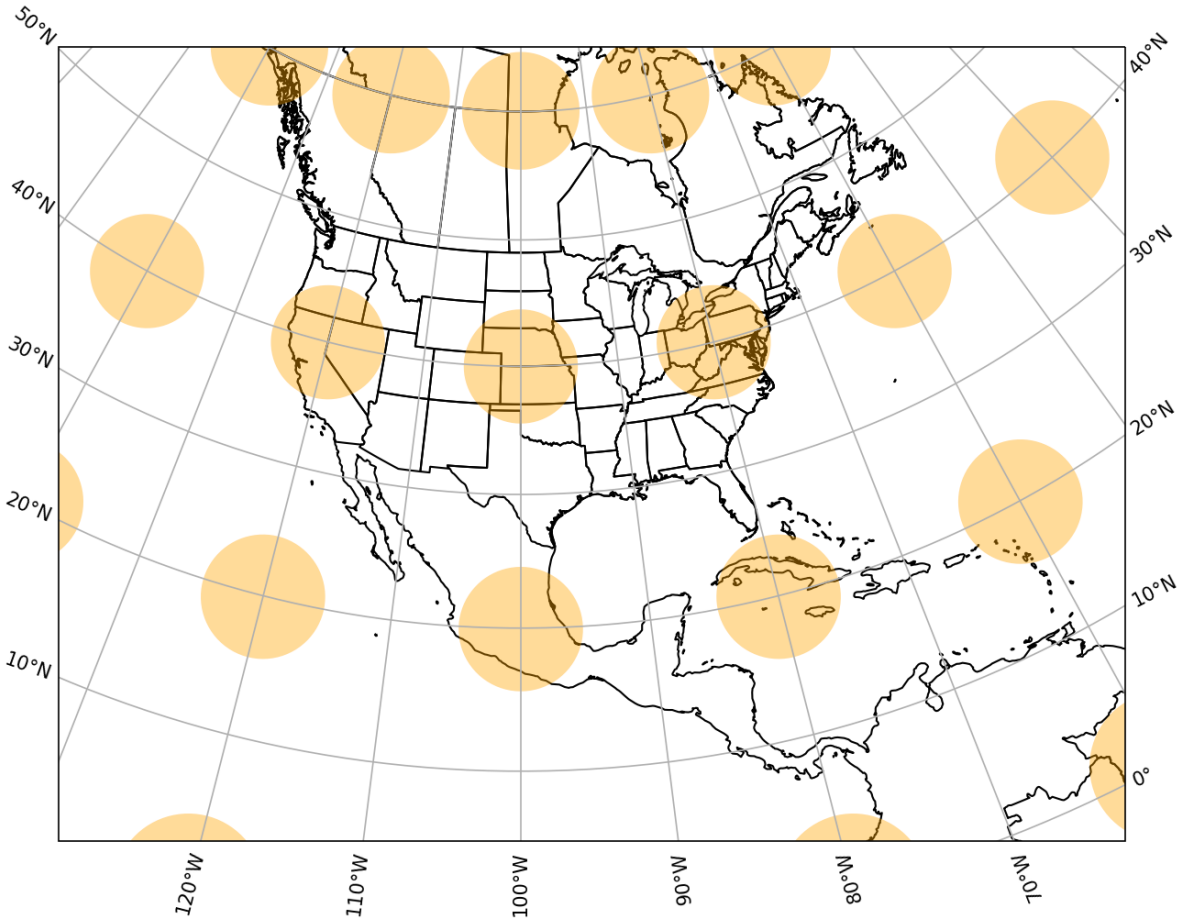


Figure 10: Representation of the Lambert Conformal projection with a central longitude of -100řE and parallel latitudes of 30řN and 60řN using Cartopy with the Tissot indicatrix plotted in orange showing the map distortion due to the projection.

There is a lot more information about map projections and their positive and negative features. There are a number of websites that have good information about various map projections and I found that this ESRI site was particularly good. For a full list of the projections available within the Cartopy package, visit this site.

Using Projections

There are three pre-defined projections that can be used, `'lcc'`, `'mer'`, `'ps'`, or `'area'`.

`lcc`: Lambert Conic Conformal

- central latitude of 40žN; central longitude -100žE; standard parallels of 30ž and 60žN

`mer`: Mercator

`ps`: Polar Stereographic

- central longitude -100žE

`area`: Uses the projection associated with the named area string

- To use this you *must* have a valid string in the `area` setting for `MapPanel()`.

Additionally, the projection can also be set to any valid Cartopy Projection...
Cartopy Projections: https://cartopy.readthedocs.io/stable/
To use a Cartopy Projection you need to add to your import list the following:

```
import cartopy.crs as ccrs
```

Then you can specify any projection (with arguments as needed). For example,

```
panel.projection = ccrs.PlateCarree()
```

or

```
panel.projection = ccrs.Miller()
```

This is especially useful if you are plotting a map panel over a given state or region that might be better to have a different rotation than what are in the preset projections.

Setting the Plot Area

The area can be set to a string using a common state or country postal abbreviation:

`panel.area = 'us'` or `panel.area = 'in'`

or a special name of a pre-defined area, such as:

`panel.area = 'uslcc'`

and you can add a plus (`+`) or minus sign (`-`) to the abbreviation to zoom in or out, respectively.

Additionally, you can set the area with a list of longitude and latitude values to get a more precise area for what you are attempting to plot. The list goes in the order of the west longitude, east longitude, southern latitude, northern latitude. For example, to plot over the continental United States (CONUS) you might prefer:

```
panel.area = [-124, -72, 24, 50]
```

A full list of valid areas is contained within the MetPy documentation. You can see that list and a basic visualization of the area using the default projection for that area at: https://unidata.github.io/MetPy/latest/api/areas.html

Upperair Observation Plotting

This notebook demonstrates reading surface data and plotting it using MetPy's declarative syntax.

To make a plot of surface observations requires three elements

- `PlotObs()` to specify the observations to be plotted
- `MapPanel()` to specify the characteristics of the map to plot the observations on
- `PanelContainer()` to collect one or more panels.

So we are using the same method to plot upperair observations that we used to plot surface observations: `PlotObs()`. If you need a refresher on how to set up the parameters for `PlotObs`, please refer to detailed description here.

One of the main differences will be that we'll need a different data source.

Import Modules Here we import two new pieces to help us read and process upperair observation data. A new function to help get us latitude and longitude points for each upperair observation station. This helper function comes from the I/O module in metpy and is named `add_station_lat_lon`. We'll also use a new library, Siphon, to be able to access and read remote upperair observation data.

```
from datetime import datetime

from metpy.io import add_station_lat_lon
from metpy.plots import declarative
from metpy.units import units
from siphon.simplewebservice.iastate import IAStateUpperAir
```

Read Data There is an easy way to access upperair data from Iowa State that has data from 1974 to yesterday that is easily accessible in a Python framework. This requires a different Python module called Siphon (also developed by Unidata). With the use of functionality from Siphon, all we need to provide is the date and time (in UTC) for the observations we wish to obtain. So we can use our favoriate `datetime` module to specifically specify the year, month, day, and hour of our observations.

> **Note**
> For upperair observations we *at most* have them only twice in a given day, at 00 UTC and 12 UTC. So you'll only want to put in an hour of `0` or `12` when accessing upperair observations.

```
# Set date for desired UPA data
date = datetime(2021, 12, 31, 0)

# Request data using Siphon request for data from Iowa State Archive
df = IAStateUpperAir.request_all_data(date)
```

Now we need to process our data just a little bit. First we use our helper function to add station latitude and longitude values to our DataFrame, then we take out the station called 'KVER' because there is not an upperair station at that location (it is misidentified within the dataset).

```
# Add lat/long data and drop stations for which we don't have lat/lons
df = add_station_lat_lon(df, 'station').dropna(subset=['latitude', 'longitude'])

# Eliminate the station KVER
df = df[df.station != 'KVER']

# Print the list of variables available from the dataset.
print(list(df.columns))
```

Note
Assumed Upperair Observation Units: `{'pressure': 'hPa', 'height': 'meter', 'temperature': 'degC', 'dewpoint': 'degC', 'direction': 'degrees', 'speed': 'knot', 'u_wind': 'knot', 'v_wind': 'knot', 'station': None, 'time': None}`

Calculate Dewpoint Depression Dewpoint depression (DD) is a quantity that needs to be calculated from the temperature and dewpoint at each observation point. The calculation is simple,

DD = Temperature - Dewpoint

and then we just need to save the difference to a new variable name and store within our DataFrame.

```
df['dewpoint_depression'] = df['temperature'] - df['dewpoint']
```

Plot Upperair Map Now that we have our data, we are ready to go about plotting it on a map.

Here we demonstrate our use of the Lambert Conic Conformal projection that has been incorporated into the declarative syntax, `'lcc'`. Additionally, we'll use a `lambda` to format the heights into the appropriate three-digit format and set an appropriate extent in the for the map we are desiring to plot.

Formatting Heights As you learned earlier in this packet, upperair geopotential height observations are plotted in a particular format that changes depending on what level is being plotted. To plot the height value in the appropriate format you'll need to specify a format. Here are the code snippets for each common level:

925, 850, or 700 hPa: `hght_format = lambda v: format(v, '03.0f')[1:]`

500 or 300 hPa: `hght_format = lambda v: format(v, '03.0f')[:-1]`

250 hPa and above: `hght_format = lambda v: format(v, '03.0f')[1:-1]`

Note
You'll need to choose the appropriate `height_format` variable depending on which level you are plotting. Two examples are given in this notebook.

Filtering Observations The `reduce_points` parameter is used to thin the data out, so that you can read the observations that are plotted without interference from other observations. Setting filter to a positive value will reduce the number of overlapping observations, with higher numbers produce less chance of an overlap to occur. By default, the attribute is set to zero, which will cause no filtering of observations.

Note
For an upperair map, since there are so few observations, `reduce_points` is typically set to zero.

Setting Titles Setting some sort of title is a very important to creating a good, self-describing plot. By setting the title you specify to others what and who plotted this map. Any of the three options can be set with a string (or formatted string). A generic title will be generated if this attribute is not set. The three options allow you to place titles in up to three different locations (center, left, and right). Below contains a demonstration of using a combination of `right_title` and `left_title` to make a more readable set of titles for the map.

The Payoff Plot

```
# Set a format specifier for the geoptential height
# This formatter takes a value 9300 -> 930
# Good for 500 -hPa and 300 -hPa Observations
height_format = lambda v: format(v, '.0f')[: -1]

# Set a format specifier for the geoptential height
# This formatter takes a value 1576 -> 576
# Good for 700 -hPa, 850 -hPa, and 925 -hPa Observations
# height_format = lambda v: format(v, '.0f')[1:]

# Plot desired data
obs = declarative.PlotObs()
obs.data = df
obs.time = date
obs.level = 500 * units.hPa
obs.fields = ['temperature', 'dewpoint_depression', 'height']
obs.locations = ['NW', 'SW', 'NE']
obs.formats = [None, None, height_format]
obs.vector_field = ['u_wind', 'v_wind']

# Panel for plot with Map features
panel = declarative.MapPanel()
panel.layout = (1, 1, 1)
panel.projection = 'lcc'
panel.area = [ -124, -72, 24, 50]
panel.layers = ['states', 'coastline']
panel.left_title = f'{obs.level.m} -hPa Observations'
panel.right_title = f'Valid: {date:%Y -%m -%d %H00} UTC'
panel.plots = [obs]

# Bringing it all together
pc = declarative.PanelContainer()
pc.size = (15, 12)
pc.panels = [panel]

#pc.save(f'{obs.level.m}hPa_observations_{date:%Y%m%d%H}.png', dpi=150, bbox_inches='tight')
pc.show()
```

Practice Tasks - Using Projections

1. Open a notebook and create a map using the Lambert Conic projection by using:

   `panel.projection = 'lcc'`

   - What are qualities of this map that you like and don't like?

2. Open a second notebook create another map using the Mercator projection by using:

   `panel.projection = 'mer'`

   Compare the Lamber Conic Conformal projection with the on made using the Mercator projection.

   - What are the qualities of this map that you like and don't like?

3. Open a third another notebook create another map using a projection of your choice by setting it with a valid Cartopy projection.

   - Compare with the other maps, what do you like? What don't you like?

Practice Tasks - Upperair Maps

1. Create a 500-hPa map displaying the information in the proper format in a Jupyter Notebook.
2. Create the same maps as in 1 but do so using a Python script.
3. Plot 850 and 300-hPa observations using two new scripts derived from the 500-hPa script you made in 2.

0.3.4 Introduction to Hand Analysis

Map analysis is still a large part of what a meteorologist does every single day. This larger concept of "map analysis" includes the drawing of contours of temperature, pressure, and height as well as analyzing fronts, troughs, ridges, etc. The guidelines contained in this chapter are a compilation of good rules of thumb and techniques for identifying many different meteorological phenomena on weather maps. You may not always physically draw out the contours or analyze fronts, but you will always need to be able to identify all of the different aspects of the weather to be able to conduct a weather discussion or doing research. Therefore, developing the skills of map analysis will serve you well throughout your meteorological career.

This set of guidelines will start with a description of hand analysis (e.g., the drawing of contours) on surface and upper-air weather maps. Then the discussion will turn to analyzing maps that are already contoured with the basic parameters, whether hand or computer drawn.

This chapter provides you with an introduction to the methods of synoptic map analysis. It is by no means an all-inclusive guide, but serves as a good outline on the proper techniques of analysis. You will be expected to refer to this information for general synoptic analysis techniques throughout your career.

Tools

There are a number of tools that are useful for conducting a successful hand analysis.

- Pencil
- Fine tip pens (black, blue, green, red, yellow, brown, purple, and orange)
- Colored pencils (blue, brown, green, red, purple, yellow)
  - including multiple shades of blue, green
- Good eraser
- Lots of patience

General Tip

The most important thing to remember when analyzing maps is that it requires **time** to do a good job. You are contouring values in a fluid; actual values may change quickly, but the gradient of values change more gradually.

Step 1: Draw Contours In Pencil

Begin by drawing your lines in light pencil and then examine this preliminary analysis.

- Where are the wiggles that can't be justified by the data?
- Draw in new smoother lines where the wiggles are not justified
  - (consistent with the data of course!).
- Smooth out any sharp changes in gradients.
- Even the most experienced synopticians mess up occasionally, so make
  - use of that pencil and eraser.

Drawing Contours ***How do I draw a contour?*** *Answer:* Interpolation

When drawing contours of a field (also called isopleths), the technique of interpolation should be favored over extrapolation. Thus, a contour is usually placed closer to the data point whose value is nearer that of the contour (Figure 11). However, other factors, such as internal consistency and the possible presence of inaccurate or 'bad' data must also be considered. In these cases, an exact interpolation between data points in your analysis is not always the best way to go. Particularly in data sparse regions, you should utilize wind information to help you with your pressure or height analysis.

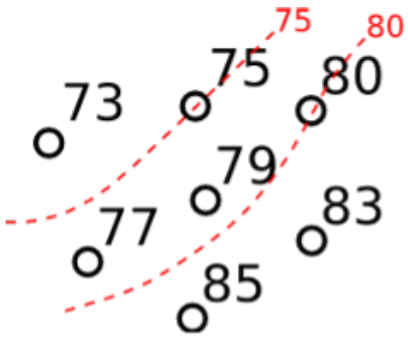


Figure 11: Example of isotherms. Go through values that appear and place the line at the approximate location that the value of the isopleth would occur.

Missing Data ***No Data?*** *Answer:* No contours!

Good synopticians do not extrapolate data contours in large data void regions (such as the Gulf of Mexico and the Atlantic and Pacific Oceans on all maps, the Rockies at 850 mb, and underground areas on isentropic charts), as there is no data there. If the fields you are analyzing appear coherent and on either side of the data void region, you may connect your contours through the region using a series of dots in the appropriate color. This indicates that there is a significant amount of uncertainty in this area and the dots represent a rough estimate. But one should be cautious about how you draw any lines through a data void region.

Under no circumstances should contours be connected by circling around the boundaries of a map, over oceans, or other extremely large bodies of water.

Bad Data ***Is the observation bad?***

Be on the lookout for bad data points. If you question a particular value, check neighboring points. If they are significantly different, you *may* want to ignore the point. Also examine the same station's data at other available pressure levels. If they are also spatially inconsistent, you may be visiting Bad Dataville. In problem areas, look at analyzing the available information several different ways and then disregard any data point(s) that result in an unrealistic analysis. Remember, disregarding a data point is a judgment call (as it may be due to a real process), but your analysis should make sense!

Be acutely aware of the presence of thunderstorms, as their effects can introduce noise into a synoptic scale analysis. At the surface, look for thunderstorms, lightning, heavy precipitation, gust

fronts, meso-highs, etc. In synoptic analyses, these effects can often be treated judiciously as bad data. Remember that thunderstorms often affect the analysis at several levels.

Step 2: Meteorological Consistency

Make your analysis meteorologically consistent.

Your analysis should be consistent in both time and space. Remember that the atmosphere is a fluid in which events evolving upstream/downstream, above/below and at previous times all influence the current weather situation and this fact needs to be reflected in your analyses. Examine maps from different pressure levels to make sure everything is consistent (e.g., a typical cyclone will tilt to the west with height). If you have a time series of maps, do features evolve relatively smoothly or does the pattern take an unexplained 'jump'? If you have maps from several times at the same level and/or several levels at the same time, you can use these charts to help with a particular analysis.

Except for the near surface, meteorological fields are continuous and usually continuously differentiable. This means that your analysis should be (for the most part) **smooth**, particularly as you move up through the atmosphere. Height and pressure contours should curve to represent the large-scale pattern, *without a lot of wiggles present.* Spacing and direction of these contours should be consistent with the wind information. If the height field aloft suggests the presence of a short wave, check to see if the wind field agrees.

Meteorological Relationships

- Geostrophic wind balance
  - requires wind speeds to be greatest in regions of tightest gradient
- Gradient wind balance
  - requires a stronger wind for a given contour spacing around an anticyclone (supergeostrophic) than around a cyclone (subgeostrophic)
- Troughs tend to tilt toward cold air
- Ridges tend to tilt toward warm air
- Jet streaks over low-level temperature gradients

Keep in mind where the gradient/geostrophic assumption is good and where it is bad and where the ageostrophic wind is important. Examples include frictionally induced cross-contour flow at the surface, an isallobaric wind component where there are rapid changes in the pressure or height fields, and transverse circulations around jet streaks aloft.

Step 3: Labeling contours

After drawing your contours and making them consistent you'll need to label your contours appropriately.

- Label your contours at both ends on the map boundaries according to the methodology appropriate for each field; leave a space to label closed contours.
- Use decameters on upper-level maps (e.g., 5400 m is labeled 540)
- Use two digits for pressure on surface maps (e.g., 1004 mb is labeled 04)
- Label colors should correspond to match the contours coloring.
- DO NOT PUT UNITS ON LABELS!

Step 4: INK the contours

Once you have a finalized set of contours draw in pencil, that you have refined to be meteorologically consistent, then you are ready to make them permanent by making one final pass of your contours with extra fine tip markers. You'll also need to complete the following:

- Make the analysis look "clean", erase stray pencil marks
- Complete any shading

Upperair Map Contour Parameters

The tables below outlines the most common parameters to be contoured on a upperair maps for 850, 700, 500, and 300 hPa. Pay particular attention to the contour intervals at the different levels, which would be good to commit to memory and should be used whether a human or a computer is drawing the contour. In particular, pay close attention to the geopotential height parameter.

	Parameter	Contour Interval	Color	Style
850 hPa	Height	3 dm centered on 150 dm (e.g., 138, 141, 144, 147, 150, 153, 156)	Black	Solid
	Temperature	Every 5řC on the 10's and 5's	Red	Dashed
	Mixing Ratio	Every 2 g/kg	Green	Dashed

Shading on 850 hPa maps:

Mixing ratio (or other moisture variable) can be shaded with progressively darker green colors as the moisture content increases (e.g., higher values of mixing ratio).

	Parameter	Contour Interval	Color	Style
700 hPa	Height	3 dm centered on 300 dm (e.g., 288, 291, 294, 297, 300, 303, 306)	Black	Solid
	Temperature	Every 5řC on the 10's and 5's	Red	Dashed
	Dewpoint Depression	Every 5řC	Green	Dashed

Shading on 700 hPa maps:

Dewpoint Depression (or other moisture variable) can be shaded with progressively darker green colors as the moisture content increases (e.g., lower values of dewpoint depression).

	Parameter	Contour Interval	Color	Style
500 hPa	Height	6 dm centered on 540 dm (e.g., 528, 534, 540, 546, 552)	Black	Solid
	Temperature	Every 5řC on the 10's and 5's	Red	Dashed
	Wind Speed	Every 20 kts on the 10's starting at 30 kt (or)Every 10 m/s on the 5's starting at 25 m/s	Blue	Dashed

Shading on 500 hPa maps:

Shade isotachs with progressively darker shades of blue to purple pencil beginning at 50 kt. In other words, shade the area between the 50 and 70 knot isotach in light blue, between 70 and 90 knots in medium blue, from 90 to 110 knots in dark blue, between 110 and 130 knots in light purple, and so on (analogously, 25-35 m/s, 35-45 m/s and 45-55 m/s, 55-65 m/s, etc.). You may use orange (or some color other than blue or purple) for the strongest wind.

---

300 hPa

Parameter	Contour Interval	Color	Style
Height	12 dm centered on 900 dm (e.g., 876, 888, 900, 912, 924)	Black	Solid
Temperature	Every 5řC on the 10's and 5's	Red	Dashed
Wind Speed	Every 20 kts on the 10's starting at 50 kt (or)Every 10 m/s on the 5's starting at 25 m/s	Blue	Dashed

Shading on 300 hPa maps:

Shade isotachs with progressively darker shades of blue to purple pencil beginning at 50 kt. In other words, shade the area between the 50 and 70 knot isotach in light blue, between 70 and 90 knots in medium blue, from 90 to 110 knots in dark blue, between 110 and 130 knots in light purple, and so on (analogously, 25-35 m/s, 35-45 m/s and 45-55 m/s, 55-65 m/s, etc.). You may use orange (or some color other than blue or purple) for the strongest wind.

Practice Tasks - Upperair Hand Analysis

1. Practice drawing 500-hPa heights every 60 m.
2. Practice drawing 300-hPa heights every 120 m.
3. Practice drawing 300-hPa wind speed every 20 kts.

Surface Maps Contour Parameters

We previously introduced hand analysis when we looked at the difference between an isobaric (pressure) surface and a constant height surface. In this lab you will complete a hand analysis of a surface map. The surface map can be more difficult because we cannot use the same atmospheric relationships, such as geostrophic or gradient wind balance, to aid in the drawing contours at the surface. This is the result of need to account for the friction that is present at and near the surface. However, it is through the analysis of one-year worth of surface data that the Bergen School was able to develop the early theories and life cycles of mid-latitude cyclones.

Surface Analysis

Contouring is accomplished in just the same way as it would be for upper-level maps, separating warmer air from colder air or higher pressure from lower pressure. At the surface there are three main parameters that re analyzed by contouring: mean sea-level pressure (MSLP), temperature, and dewpoint temperature. The details of the contours are summarized below and are also part of the initial handout regarding map analysis.

Surface

Parameter	Contour Interval	Color	Style
Mean Sea Level Pressure (MSLP)	4 mb centered on 1000 mb (e.g., 984, 988, 992, 996, 1000, 1004, 1008)	Black	Solid
Temperature	Every 10řF on the 10's or 5řC on the 10's and 5's	Red	Dashed
Dew Point	Every 10řF on the 10's or 5řC on the 10's and 5's	Green	Dashed

Shading on surface maps:

Precipitation is shaded in *light green* pencil, fog in *yellow*. When shading avoid a polka dot look when shading areas of precipitation. Thunderstorm symbols should be circled in *red* pen. Occasionally, the clear regions are delineated by scalloped *blue* lines.

There are additional features that are important to mark while analyzing a surface map. Generally, you want to indicate areas of precipitation, shading areas in light green. You may wish to differentiate different precipitation types with different colors. For example, you might shade snow in light blue, rain in light green, and mixed precipitation types in pink. The shading is done to help the analyzer better understand the current weather situation present on the map.

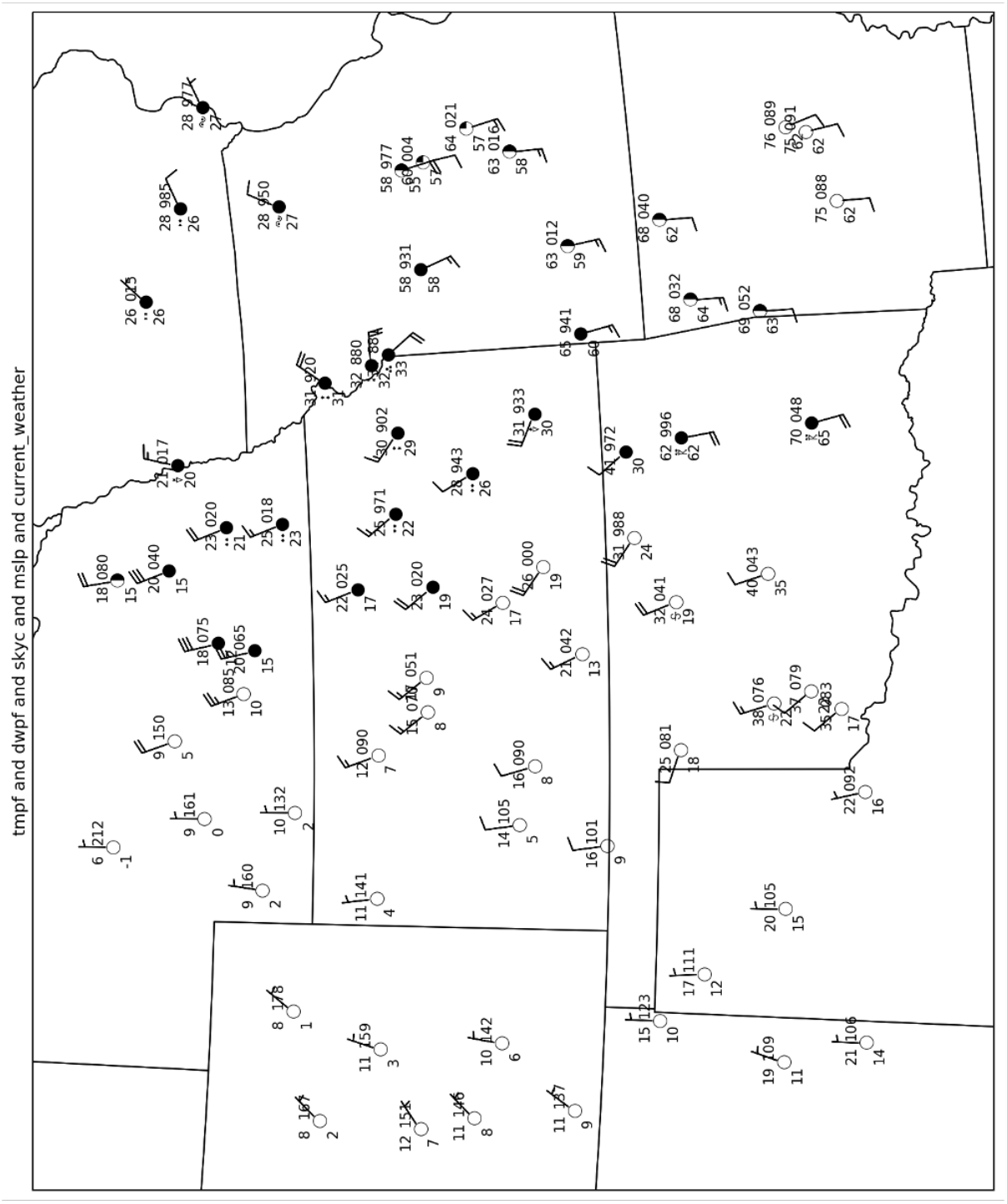
Drawing Fronts

Identifying fronts is not a trivial task, yet they are fundamental parts of our cyclone model since the time of Bjerknes and Solberg [1922]. Therefore, it is important to identify and correctly analyze the location of fronts at the surface. There are times that a front will practically jump off the map, but more often than not it will take time to discern the actual location of a front. Sanders and Doswell [1995] document well the difficulty of identifying fronts. For a given case even experienced meteorologists will often disagree with the placement of fronts. The following are some key factors to consider when attempting to locate fronts.

1. *strong temperature gradients*
    - Fronts generally lie on the warm side of temperature gradients. However, beware of diurnal effects, cloudy vs. sunny regions, snow cover line, lake effects, etc.
2. *wind shifts*
    - Winds normally veer with a frontal passage: cold fronts - S ahead, NW behind; warm fronts - SE-NE ahead, SE-SW behind, stationary fronts – winds parallel to temperature gradient
3. *troughs in the MSLP field*
    - Fronts are often found in pressure troughs.
4. *pressure tendencies*
    - Look for pressure rises behind cold fronts, falls ahead of warm fronts
5. *moisture discontinuity*
    - Drier air behind cold front, moist air ahead of front
6. *upper-air charts*
    - Look especially for 850 mb temperature gradients.
7. *1000-500 mb* thickness
    - Thickness is proportional to the mean temperature in the layer.
8. *current weather*
    - Light to moderate stable precipitation ahead of warm fronts; convection ahead of cold fronts.
9. *time continuity*
    - How is the temperature gradient changing in time?
10. *local/topographic effects*
    - Wind blowing through river valleys, etc.

Practice Tasks - Surface Contouring

Using the map below, draw mean sea-level pressure contours every 4 mb and temperature contours every 10řF, then analyze the cold and warm front.



0.3.5 Atmospheric Forces

In this chapter you will be introduced to the forces governing atmospheric motions that will be important for describing important for synoptic scale motions with force balances. A common theme throughout all of meteorology is balance. The non-uniform heating from the sun across Earth's surface creates an imbalance in temperature, which creates a localized difference in pressure. The atmosphere does not like the imbalance (in temperature or pressure or anything), so the reaction is to undo the temperature/pressure difference. This is where we get weather from, an attempt to regain balance in the atmosphere. Luckily, the atmosphere is never able to achieve the balance (because it is always being pulled out of balance) otherwise we wouldn't have anything to forecast!

Pressure Gradient Force

There are a number of forces (accelerations) that govern atmospheric flows. The most important is the pressure gradient force (PGF). ***The PGF is the only force that can actually cause the wind to blow***. All other forces act upon the already moving air in the atmosphere. Mathematically we can write a pressure gradient (PG) in algebraic form as,

$$PG = \frac{\Delta p}{\Delta n} \tag{1}$$

where Δp is a change in pressure and Δn is a change in distance. This is not quite our pressure gradient force. Pressure has units of $N\ m^{-2}$, which is force per unit area, whereas the PG has units of $N\ m^{-3}$, which is a force per unit volume. To get it into our pressure gradient *force* that we use in meteorology we need to divide by density, ρ:

$$PG = -\frac{1}{\rho}\frac{\Delta p}{\Delta n} \tag{2}$$

By convention, we place the negative sign in front of the PGF so that the force ***always*** points toward low pressure where the units of PGF are $m\ s^{-2}$ (an acceleration). **Air will *ALWAYS* move from higher pressure to lower pressure.**

Looking at this concept graphically we can inspect a 500-hPa plot of geopotential heights (Figure 12). Where lines of geopotential height are closest together indicates where the strongest height gradients are located. In Figure 12, we can see that the red dashed contours are located where the black geopotential height contours are most closely packed, thus indicating the areas on this particular map where the strongest height gradients are located.

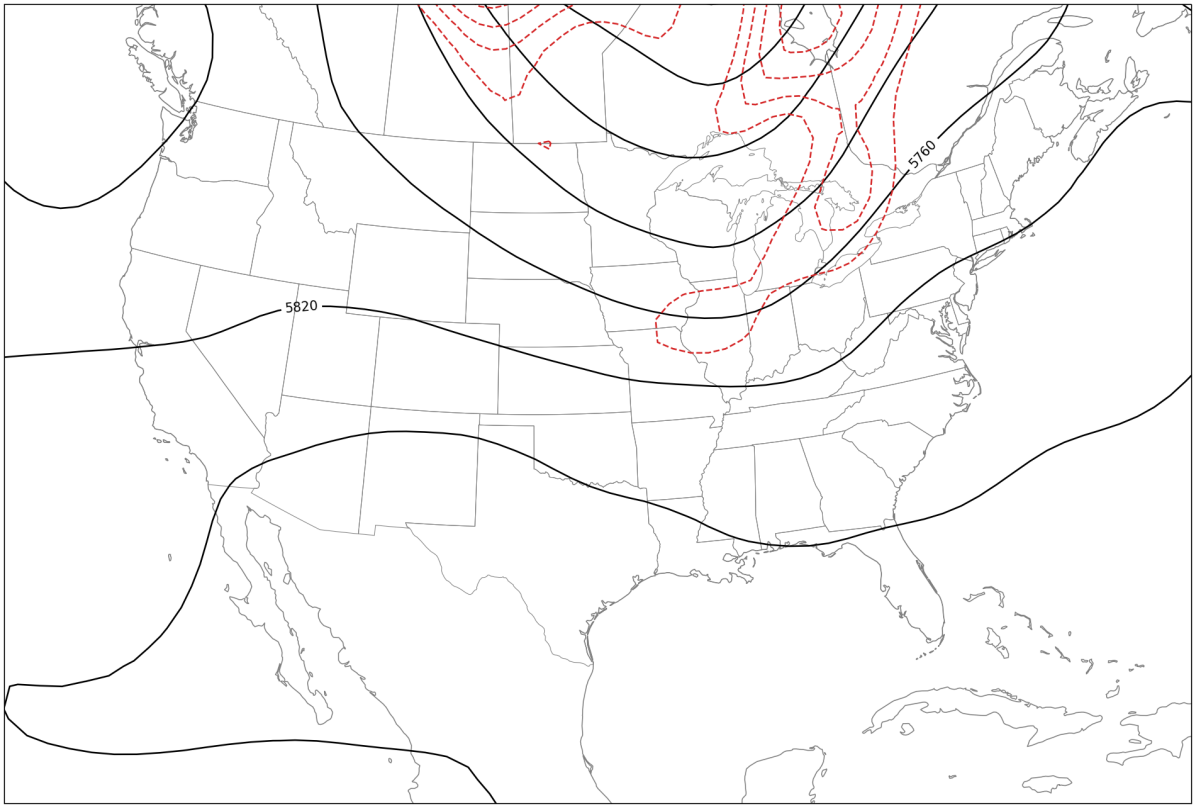


Figure 12: This map shows 500-hPa heights in solid black contours and the magnitude of the height gradient in dashed red contours.

Coriolis Force

The fact that Earth rotates once every day on its axis results in a force acting on the wind. This force is called the Coriolis Force (CF), which is named after the scientist who quantified the relationship. In order for the CF to impact atmospheric motions, the flow must be going over large time and/or spatial scales. For example, the CF will NOT affect water draining out of your sink or bathtub because it will drain too quickly and is not draining over a large enough spatial scale. We can quantify the Coriolis parameter, f, as follows,

$$f = 2\Omega sin(\phi) \tag{3}$$

where ϕ is the latitude and is the Coriolis constant, which is $7.292 \cdot 10^{-5} s^{-1}$. The strength of the Coriolis parameter increases with increasing latitude (Figure 13) and is maximized at the poles (not shown).

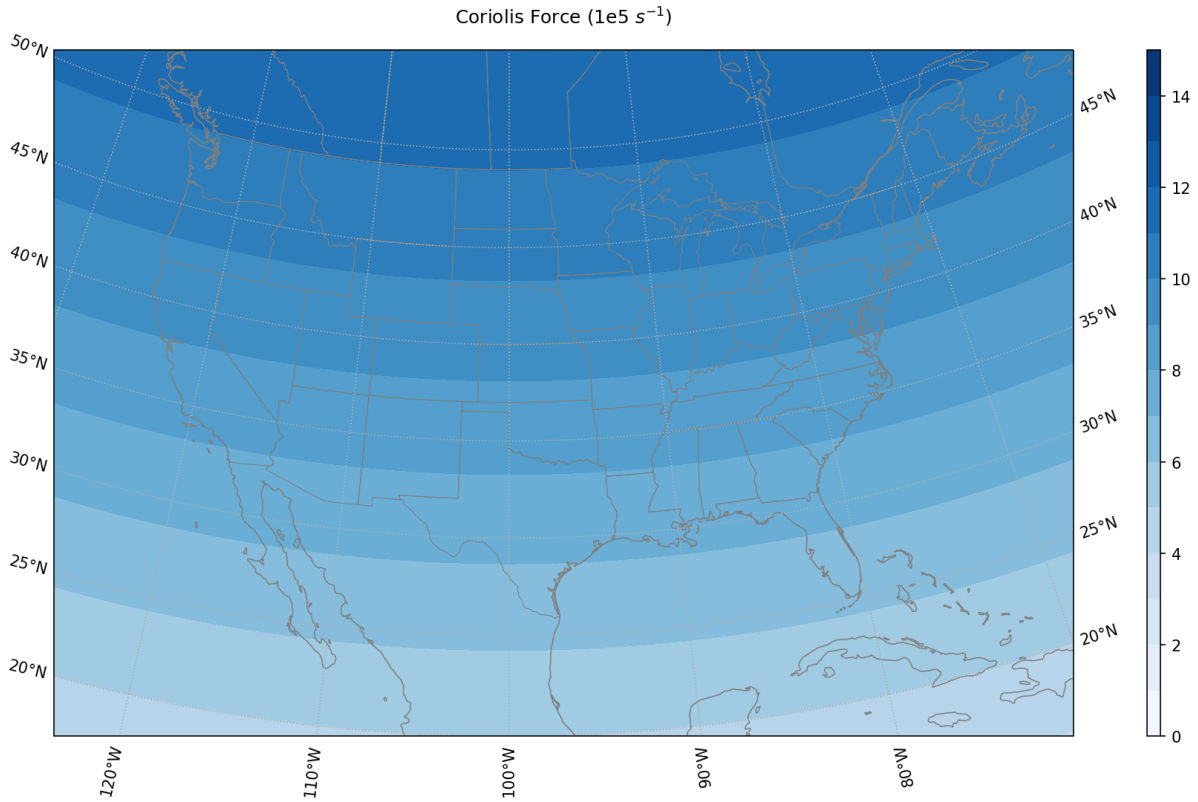


Figure 13: This map shows the value of the Coriolis parameter, which is directly proportional to the latitude.

The Coriolis force acts perpendicular to the wind at all times, it pulls the wind to the right in the Northern Hemisphere and to the left in the Southern Hemisphere. The CF in the Northern Hemisphere can be written in a bulk sense as,

$$CF = -fV \tag{4}$$

where V is the wind speed. The CF will only act on the wind, it cannot create the wind. If there is no wind, the Coriolis force is zero, because the wind is zero! Similarly, if you are at the equator, the Coriolis Force will also be zero because the Coriolis Parameter is zero! It shouldn't be a surprise that the CF has the same units as the PGF, $m\ s^{-2}$.

Centrifugal Force

If the flow is curved, as in troughs and ridges, another force must be considered, the centrifugal force (CENTF). This force is also known as the centripetal acceleration. The centrifugal force acts to push outward from the center of the turn. For example, if you are in a car and take a corner holding constant speed, you will feel like you are being pushed away from the turn, that is the centrifugal force at work. The centrifugal force can be written in algebraic form as,

$$CENTF = \frac{V^2}{R} \tag{5}$$

where R is the radius of curvature. The radius of curvature, R, is positive for cyclones and negative for anticyclones in the Northern Hemisphere. Therefore, in low pressure, the centrifugal force acts opposite the direction of the PGF, but in high pressure it acts in the same direction. Again, unit analysis indicates that the centrifugal force has the same units as PGF and CF and is $m\ s^{-2}$.

Friction Force

Close to the surface there are many things blocking the flow of air from high to low pressure, such as trees, buildings, mountains, etc. We term this force, friction, and it always acts to slow the wind. Friction can be written algebraically as,

$$FF = -kV \tag{6}$$

where k represents the coefficient related to the roughness of Earth's surface and will vary depending on the characteristics of the surface and has units of s^{-1}. Again, the units will be $m\ s^{-2}$.

Gravity

Finally, the last force to consider is the gravitational force. It's not only a good idea, but also the law! Gravity always acts downward and keeps things from accelerating in the vertical and is assumed to equal 9.81 $m\ s^{-2}$.

All of these forces can now be combined into a number of different force balances, which can explain a large number of different flows in the atmosphere. We will consider these force balances and the assumptions that form these atmospheric balances during a later class period.

Practice Tasks - Forces

Given the following 300-hPa map (Figure 14), answer the following questions:

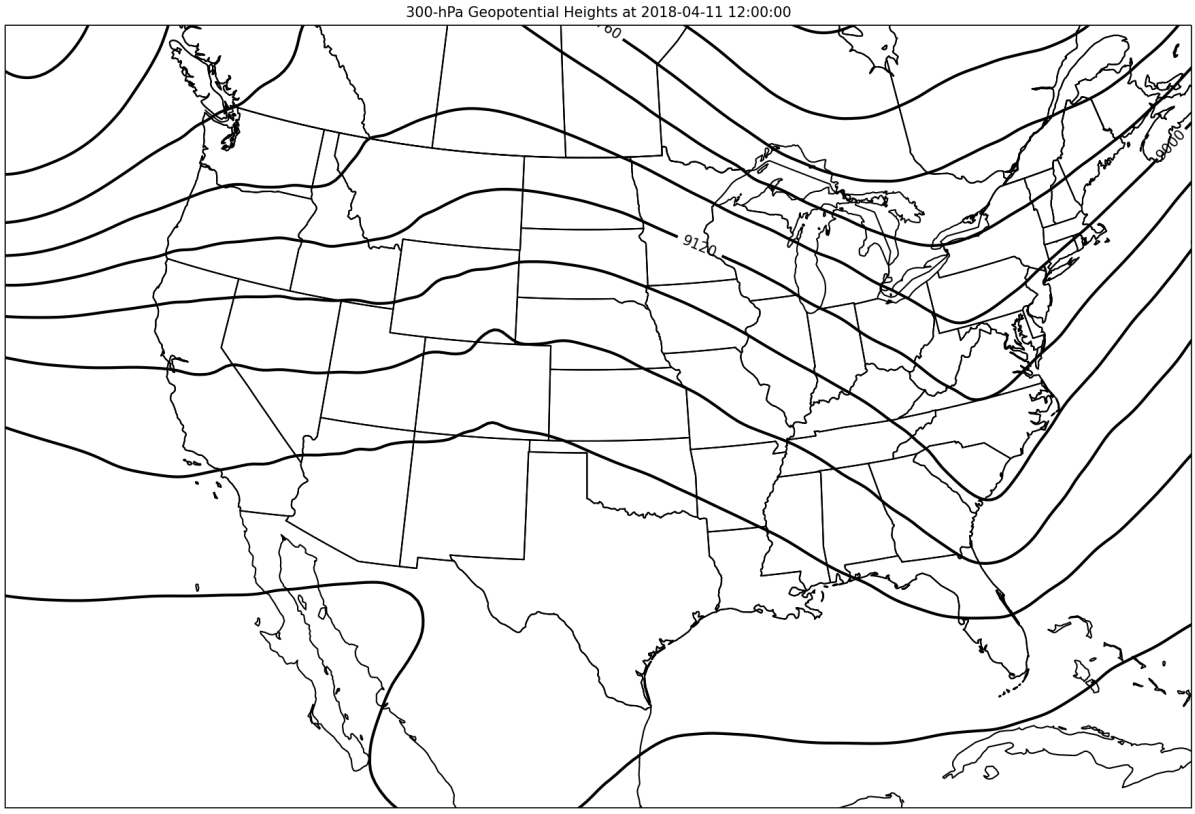


Figure 14: This map shows 300-hPa heights in solid black contours for 11 April 2018 at 12 UTC.

1. Describe geographically where the strongest height gradient is located.
2. Over what state(s) would the centrifugal force be the strongest. Explain your choice.
3. Would friction play a large role in slowing the winds down at this level? Explain.
4. Would the Coriolis Force be greater over South Dakota or Indiana under the conditions depicted on the map? Explain.

0.3.6 Gridded Data and Contour Plotting

So far this semester we have used real data, both surface and upper-air data, which come from instrumentation deployed throughout the atmosphere. We now turn to plotting model output using Python, which is on a defined grid. Previously we created maps using `PlotObs()`, which uses observational data that are NOT gridded; it plots data at observation points, which are not on a standardized grid. Model output, on the other hand, is gridded with points at regularly spaced intervals in the horizontal as well as the vertical (Figure 15).

With gridded data (e.g., model output) we can put the computer to work to generate contours that will conform to the data instead of having to do all that work by hand analysis from our point observations. This allows meteorologists to make hundreds of analyzed maps quickly that can then be inspected to determine what is happening now and if there are forecast output, then into the future as well!

Our goal in this chapter is to be able to find, access, and understand what data are contained within gridded files and be able to create contour plots analyzing the data.

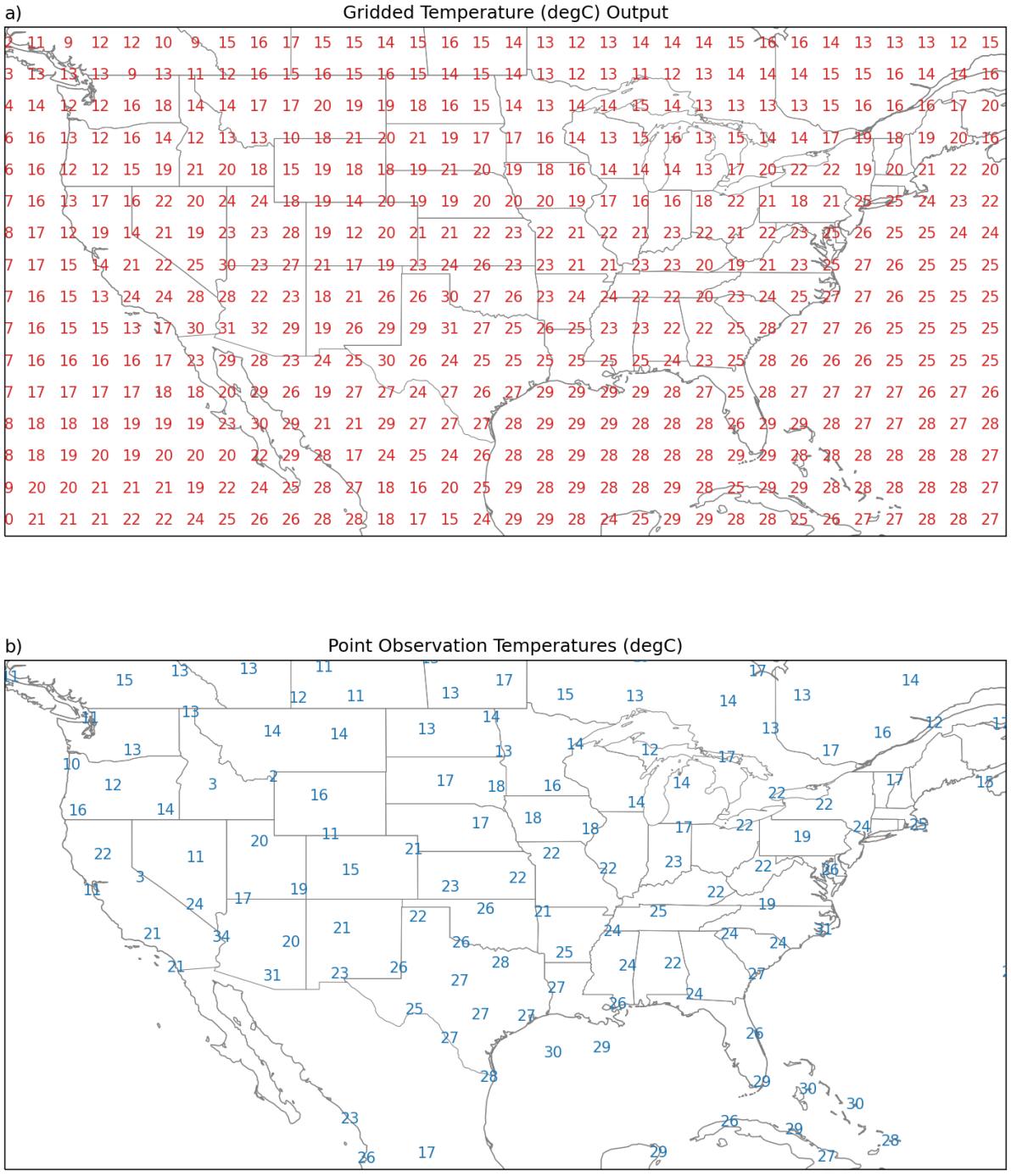


Figure 15: a) gridded data on regularly spaced grids and b) observations from discrete locations from where those particular observations came from.

Gridded Data Files

Model output has grown a lot bigger over the last decade and are becoming cumbersome to push those big files around to everyone. Instead, we can access the files remotely (meaning the files are on a computer somewhere in the world) and only bring back the small parts of what we need to do the analysis or draw the contours of the map we are making.

Current Model Output - Remote Access For current model output, the best remote site is:
https://thredds.ucar.edu/thredds/catalog/idd/forecastModels.html

Below are the URL filename structures for the most common synoptic models available through the Unidata THREDDS server. There are at least two weeks of current model output available via this access method. For the links below, replace the end of the URL (catalog.html) with the filename using the given format.

- GFS One Degree (OPeNDAP)
  - https://thredds.ucar.edu/thredds/catalog/grib/NCEP/GFS/Global_onedeg/catalog.html
    Filename Format: GFS_Global_onedeg_YYYYMMDD_HHMM.grib2
- GFS Half Degree (OPeNDAP)
  - https://thredds.ucar.edu/thredds/catalog/grib/NCEP/GFS/Global_0p5deg/catalog.html
    Filename Format: GFS_Global_0p5deg_YYYYMMDD_HHMM.grib2
- NAM CONUS 12km from CONDUIT (OPeNDAP)
  - https://thredds.ucar.edu/thredds/catalog/grib/NCEP/NAM/CONUS_12km/conduit/catalog.html
    Filename Format: NAM_CONUS_12km_conduit_YYYYMMDD_HHMM.grib2
- Rapid Refresh CONUS 13 km (OPeNDAP)
  - https://thredds.ucar.edu/thredds/catalog/grib/NCEP/RAP/CONUS_13km/catalog.html
    Filename Format: RR_CONUS_13km_YYYYMMDD_HHMM.grib2

If you visit the main website above you'll see that there are a whole lot of different model output sources, feel free to try and use them to make maps for tasks.

Current Model Output - Local Files Many institutions (or individuals) keep (download) a small set of local model output in netCDF (or other) format, which are easily read into Jupyter Notebooks. In a terminal window (or tab) locate where your netCDF model output is located.

Note

At Valparaiso University, local data from LDM can be found in the `/data/ldmdata/` folder on our server. From there you can find were the model data are located using your terminal window, note the absolute path and the filename to be able to use in your code.

Record the absolute path to model data for each of the model types listed below:
NAM:
GFS:
All of the file names are of a standard format.
YYYYMMDDHH_<model>.nc
YYYY – 4-digit Year
MM – 2-digit Month
DD – 2-digit Day
HH – 2-digit Model Initialization Hour
<model>is the model name: gfs, nam (in lowercase)

Model Output Contents

On surface maps we plotted temperature, dewpoint temperature, mean sea-level pressure, wind speed and direction, and current weather symbols, which are all typical values recorded by Automated Surface Observing Stations (ASOS) and are standardized. For model output, there are not a common set of data or naming convention used for variables between different types of data (e.g., GFS, NARR, NCEP-NCAR Reanalysis). There are small differences between the different models and for that reason we need a method to find out what variables are available in contained within each file. This is particularly easy in the Jupyter Notebook environment if we use the `xarray` module.

Here we'll demonstrate how to gain access to a gridded file and inspect the contents of the file (e.g., what variables are contained within the file), all of which can be completed in a standard Jupyter Notebook.

```
from datetime import datetime, time, timedelta, UTC

import xarray as xr

yesterday = datetime.now(UTC) - timedelta(days=1)
date = datetime.combine(yesterday, time(0))

# Access the 1 Degree GFS from the Unidata THREDDS server
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib'
                     f'/NCEP/GFS/Global_onedeg/GFS_Global_onedeg_'
                     f'{date:%Y%m%d}_{date:%H%M}.grib2')
```

List File Contents We can quickly and easily list the variable names that are contained within our model output files. This listing will contain both dimension variables (e.g., time, latitude, and longitude) as well as the variables that we might wish to plot (e.g., Temperature_isobaric). Non-dimension variables are capitalized if accessing data through a THREDDS server (a very common way for accessing these data).

```
list(ds.variables)
```

What did you find?

Access Variable Specifics Let's look at a single variable: Geopotential_height_isobaric, which is the variable for heights of pressure surfaces (i.e., what we typically plot on maps). Type the following into a new cell and run it.

There are two methods to access our variables:

Dot Method:

```
ds.Geopotential_height_isobaric
```

or

Dictionary-like Method:

```
ds['Geopotential_height_isobaric']
```

Both methods are equivalent and as long as there are no spaces or dashed in the name of a variable, either method will work. If there is a space or a dash, then only the dictionary method will work.

For example, the wind components (e.g., u-component_of_wind_isobaric) must use the dictionary method because of the dash in the name of the variable.

The output in a Jupyter Notebook will have an HTML representation that you can interact with. For example, click on the triangle icons to expand or minimize information about different aspects of the data contained within that file for that variable.

```
# Information about the Geopotential_height_isobaric variable
ds.Geopotential_height_isobaric
```

What units of Geopotential Height?

How many available forecast hours are there?

How many vertical levels are there?

```
# Information about the Geopotential_height_isobaric variable
# This example uses the dictionary -like method for accessing
# the varaible.
ds['u -component_of_wind_isobaric']
```

You will ALWAYS want to check the gridded file to know what different variables are called and know some of the basic information about the variable (e.g., how many vertical level, what units they are in, etc.). Use the information on this page to help you out!

Contour Plotting

This notebook demonstrates reading and plotting gridded data including model output. The following code will contour around every grid point and does this without "thinking". So, don't worry if not all contours are as smooth as you would like. We'll discuss how to smooth the data in future chapters.

Import Packages Here we bring in the needed Python modules to be able to plot contours using the computer. Note that there are fewer needed for this particular plotting scenario.

```
from datetime import datetime, timedelta, time, UTC

from metpy.plots import declarative
from metpy.units import units
import xarray as xr
```

Get Data Here we are using current data to demonstrate contour plotting. If you have a different data source you would like to use, simply change out the call to remote data in the `open_dataset` function to be appropriate for the local or remote data you would like to access. Future chapters will demonstrate different data sources that can be commonly used. Here we'll use the GFS data from the Unidata THREDDS server.

```
# Set the date/time of the model run
# The following code will get you yesterday at 12 UTC
yesterday = datetime.now(UTC) - timedelta(days=1)
model_run_date = datetime.combine(yesterday, time(12))

# Remote access to the dataset from the UCAR site
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib'
                     f'/NCEP/GFS/Global_onedeg/GFS_Global_onedeg_'
                     f'{model_run_date:%Y%m%d}_{model_run_date:%H%M}.grib2')

# Subset data to be just over the U.S. for plotting purposes
ds = ds.sel(lat=slice(70,10), lon=slice(360 -140, 360 -60))
```

Plot Data Here we demonstrate the use of `ContourPlot()` for having the computer draw contours of a variable. Only a small subset of atttributes are used in this example, but your plot can be further customized using a host of different attributes, which you can read about in the MetPy documentation.

MetPy `ContourPlot()` Documentation

```
# Set the plot time with forecast hours
plot_time = model_run_date + timedelta(hours=0)

# Set attributes for plotting contours
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'Geopotential_height_isobaric'
cntr.level = 500 * units.hPa
cntr.time = plot_time
cntr.contours = range(0, 10000, 60)
cntr.clabels = True

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
```

```
panel.area = [ -125, -74, 22, 52]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'500 -hPa Geopotential Heights at {plot_time}'
panel.plots = [cntr]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Contour Plotting with Wind Barbs

This notebook demonstrates plotting gridded data wind components as wind barbs along with contours. We'll continue to use 500-hPa as an example level for plotting purposes.

Import Packages Here we bring in the needed Python modules to be able to plot contours using the computer. Note that there are fewer needed for this particular plotting scenario.

```
from datetime import datetime, timedelta, time, UTC

from metpy.plots import declarative
from metpy.units import units
import xarray as xr
```

Get Data Here we are using current data to demonstrate contour plotting with wind barbs. If you have a different data source you would like to use, simply change out the call to remote data in the `open_dataset` function to be appropriate for the local or remote data you would like to access. Future chapters will demonstrate different data sources that can be commonly used. Here we'll use the GFS data from the Unidata THREDDS server.

```
# Set the date/time of the model run
# The following code will get you yesterday at 12 UTC
yesterday = datetime.now(UTC) - timedelta(days=1)
model_run_date = datetime.combine(yesterday, time(12))

# Remote access to the dataset from the UCAR site
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib'
                     f'/NCEP/GFS/Global_onedeg/GFS_Global_onedeg_'
                     f'{model_run_date:%Y%m%d}_{model_run_date:%H%M}.grib2')

# Subset data to be just over the U.S. for plotting purposes
ds = ds.sel(lat=slice(70,10), lon=slice(360 -140, 360 -60))
```

Plot Data Here we demonstrate the use of `BarbPlot()`, in addition to still plotting contours of geopotential height, for plotting a vector from its components. For more details about all of the settings for plotting gridded data wind barbs checkout the attribute descriptions or head to the MetPy `BarbPlot` Documentation

```
# Set the plot time with forecast hours
plot_time = model_run_date + timedelta(hours=0)

# Set attributes for plotting contours
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'Geopotential_height_isobaric'
cntr.level = 500 * units.hPa
cntr.time = plot_time
cntr.contours = range(0, 10000, 60)
cntr.clabels = True

# Add wind barbds
barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = plot_time
```

```
barbs.field = ['u -component_of_wind_isobaric',
               'v -component_of_wind_isobaric']
barbs.level = 500 * units.hPa
barbs.skip = (3, 3)
barbs.plot_units = 'knot'

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 22, 52]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'500 -hPa Geopotential Heights at {plot_time}'
panel.plots = [cntr, barbs]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Plotting Grid-Relative Wind Barbs

This notebook demonstrates plotting grid-relative wind components as wind barbs along with contours. We'll continue to use 500-hPa as an example level for plotting purposes.

Grid-Relative Wind Barbs Unlike observation data, wind components can be stored in two different ways: 1) Earth relative or 2) grid relative. The most common way for the wind components to be stored would be as Earth relative, meaning that the winds are stored in what would be considered the "common" way on a rectilinear grid and would be comparable tp standard observation data. Grid-relative means that the compnents are stored relative to a particular grid projection. Currently the two datasets that have wind components often stored in this grid-relative manner are the North American Model (NAM) and the North American Regional Reanalysis (NARR), whose grid was based off of a historic NAM grid. However, they may not always be stored that way, so you should ALWAYS pay close attention to the meta data associated with your dataset and ensuring the plotted winds conform to known meteorological relationships (e.g., geostrophic/gradient wind balance).

This notebook will demonstrate how MetPy can convert grid-relative winds to be appropriate for your chosen map projection.

Import Packages Here we bring in the needed Python modules to be able to plot contours using the computer. Note that there are fewer needed for this particular plotting scenario.

```
from datetime import datetime, timedelta, time, UTC

from metpy.plots import declarative
from metpy.units import units
import xarray as xr
```

Get Data Here we are using current data to demonstrate contour plotting with wind barbs. If you have a different data source you would like to use, simply change out the call to remote data in the `open_dataset` function to be appropriate for the local or remote data you would like to access. Future chapters will demonstrate different data sources that can be commonly used. Here we'll use the NAM data from the Unidata THREDDS server to demonstrate a dataset that stores grid-relative winds.

```
# Set the date/time of the model run
# The following code will get you yesterday at 12 UTC
yesterday = datetime.now(UTC) - timedelta(days=1)
model_run_date = datetime.combine(yesterday, time(12))

# Remote access to the NAM dataset from the UCAR site
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib'
                     f'/NCEP/NAM/CONUS_12km/NAM_CONUS_12km_'
                     f'{model_run_date:%Y%m%d}_{model_run_date:%H%M}.grib2')
```

Plot Data Here we plot the 500-hPa geopotential height and wind barbs for the model initialization time. Once the map in created, look closely at the winds.

```
# Set the plot time with forecast hours
plot_time = model_run_date + timedelta(hours=0)

# Set attributes for plotting contours
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'Geopotential_height_isobaric'
cntr.level = 500 * units.hPa
```

```
cntr.time = plot_time
cntr.contours = range(0, 10000, 60)
cntr.clabels = True

# Add wind barbds
barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = plot_time
barbs.field = ['u -component_of_wind_isobaric',
               'v -component_of_wind_isobaric']
barbs.level = 500 * units.hPa
barbs.skip = (15, 15)
barbs.plot_units = 'knot'

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 22, 52]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'500 -hPa Geopotential Heights at {plot_time}'
panel.plots = [cntr, barbs]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Questions

- What do you notice about the winds?
  - The winds should follow geostrophic/gradient wind balance at this level.
- If you are noticing any issues, look toward the upper-left portion of the map...should the winds be crossing the geopotential heights?

Converting Grid-Relative Winds In the declarative syntax of MetPy, there is a simple attribute that can be reset to convert the native grid-relative winds to be converted to the appropriate winds for your projection. The parameter on the wind barb class is `earth_relative` and is a Boolean, which for grid-relative data needs to be set to `False`. For more details about the settings for plotting gridded data wind barbs checkout the attribute descriptions or head to the MetPy `BarbPlot` Documentation

```
# Set the plot time with forecast hours
plot_time = model_run_date + timedelta(hours=0)

# Set attributes for plotting contours
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'Geopotential_height_isobaric'
cntr.level = 500 * units.hPa
cntr.time = plot_time
```

```
cntr.contours = range(0, 10000, 60)
cntr.clabels = True

# Add wind barbds
barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = plot_time
barbs.field = ['u -component_of_wind_isobaric',
               'v -component_of_wind_isobaric']
barbs.level = 500 * units.hPa
barbs.skip = (15, 15)
barbs.plot_units = 'knot'
barbs.earth_relative = False

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 22, 52]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'500 -hPa Geopotential Heights at {plot_time}'
panel.plots = [cntr, barbs]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Notice that in the above image, the wind barbs correspond much, MUCH better to geostrophic/gradient wind balance. Even though there is not a lot of change in the middle of the project (this is the result of the projection of the data and the map not being too different), that small difference gets bigger when the projection is more highly curved, which happens toward the edge of the map.

Direct Wind Barb Comparison Let's take a look at the two version of the wind barbs, the uncorrected wind barbs are in red in the image below and the re-gridded barbs are plotted in blue.

```
# Set the plot time with forecast hours
plot_time = model_run_date + timedelta(hours=0)

# Set attributes for plotting contours
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'Geopotential_height_isobaric'
cntr.level = 500 * units.hPa
cntr.time = plot_time
cntr.contours = range(0, 10000, 60)
cntr.clabels = True

# Add wind barbds
```

```
barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = plot_time
barbs.field = ['u -component_of_wind_isobaric',
               'v -component_of_wind_isobaric']
barbs.level = 500 * units.hPa
barbs.skip = (15, 15)
barbs.plot_units = 'knot'
barbs.earth_relative = False
barbs.color = 'blue'

# Add uncorrected wind barbds
barbs2 = declarative.BarbPlot()
barbs2.data = ds
barbs2.time = plot_time
barbs2.field = ['u -component_of_wind_isobaric',
               'v -component_of_wind_isobaric']
barbs2.level = 500 * units.hPa
barbs2.skip = (15, 15)
barbs2.plot_units = 'knot'
barbs2.color = 'red'

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 22, 52]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'500 -hPa Geopotential Heights at {plot_time} with correct wind barbs (blue), u
panel.plots = [cntr, barbs2, barbs]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Overlay Multiple Contours

But what if we want to overlay more than one set of contours? We just need to add another set of settings for plotting contours. Then we just need to add both contours to our panel and viola!

Here is some example pseudo-code to demonstrate how this would work:

```
cntr = ContourPlot()
...
cntr2 = ContourPlot()
...
panel = MapPanel()
...
panel.plots = [cntr, cntr2]

pc = PanelContainer()
pc.plots = [panel]
pc.show()
```

When plotting two sets of contours, you'll want to pay particular attention to the color and linestyle of each of the contours so that anyone reading your map will be able to easily distinguish between the two sets of lines.

Let's see how this works using some real data and plotting two different sets of contours.

Plotting Multiple Contours

This notebook demonstrates plotting multiple contours from gridded data. Here we'll plot 850-hPa geopotential heights with 850-hPa temperature in Celisus.

Import Packages Here we bring in the needed Python modules to be able to plot contours using the computer. Note that there are fewer needed for this particular plotting scenario.

```
from datetime import datetime, timedelta, time, UTC

from metpy.plots import declarative
from metpy.units import units
import xarray as xr
```

Get Data Here we are using the same current data that we have used in this chapter from the one degree GFS to demonstrate our plotting.

```
# Set the date/time of the model run
# The following code will get you yesterday at 12 UTC
yesterday = datetime.now(UTC) - timedelta(days=1)
model_run_date = datetime.combine(yesterday, time(12))

# Remote access to the dataset from the UCAR site
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib'
                     f'/NCEP/GFS/Global_onedeg/GFS_Global_onedeg_'
                     f'{model_run_date:%Y%m%d}_{model_run_date:%H%M}.grib2')

# Subset data to be just over the U.S. for plotting purposes
ds = ds.sel(lat=slice(70,10), lon=slice(360 -140, 360 -60))
```

Plot Data Here we demonstrate how to overlay multiple contours on a single plot (with wind barbs still plotted). You'll want to be sure you use two unique variable names for each set of contours and set appropriate attributes to have each contour plot just a little differently (e.g., different colors and/or linestyles).

```
# Set the plot time with forecast hours
plot_time = model_run_date + timedelta(hours=0)

# Set attributes for plotting geopotential height contours
hght = declarative.ContourPlot()
hght.data = ds
hght.field = 'Geopotential_height_isobaric'
hght.level = 850 * units.hPa
hght.time = plot_time
hght.contours = range(0, 10000, 30)
hght.clabels = True

# Set attributes for plotting temperature contours in Celsius
tmpc = declarative.ContourPlot()
tmpc.data = ds
tmpc.field = 'Temperature_isobaric'
tmpc.level = 850 * units.hPa
tmpc.time = plot_time
tmpc.contours = range( -40, 41, 5)
```

```
tmpc.clabels = True
tmpc.linestyle = 'dashed'
tmpc.linecolor = 'tab:red'
tmpc.plot_units = 'degC'

# Add wind barbds
barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = plot_time
barbs.field = ['u -component_of_wind_isobaric',
               'v -component_of_wind_isobaric']
barbs.level = 850 * units.hPa
barbs.skip = (3, 3)
barbs.plot_units = 'knot'

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 22, 52]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'850 -hPa Geopotential Heights and Temperature (degC) at {plot_time}'
panel.plots = [hght, tmpc, barbs]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Practice Tasks - Contour Plotting

1. Create a 500-hPa map with height contoured in solid black and temperature in Kelvin contoured in dashed red.

2. Add winds to your map.

3. Create a map that contains derived "surface" quantities from the model with mean sea-level pressure in solid black, and temperature in Fahrenheit in dashed red. (You'll need to convert the temperatures to the appropriate scale, which would work similarly to what we did for surface observation plots.)

4. Make other maps with a variety of variables being plotted.

0.3.7 Atmospheric Force Balances

One way to describe the wind is to construct force balance diagrams to visualize all of the different forces acting on a "parcel" of air. This goes back to the Newtonian idea of force equaling mass times acceleration:

$$\vec{F} = m\vec{a} \tag{7}$$

Where $\vec{F}$ is the force vector, m is the mass in kg, and $\vec{a}$ is the acceleration in $m\ s^{-2}$. A force balance would mean that the sum of forces acting on a particular body sum to zero. Therefore, a body in balance will not be experiencing any acceleration,

$$\frac{\sum \vec{F}}{m} = 0 \tag{8}$$

There are four main force balances that we will consider plus a 'balance of balances'. The force balances presented here are: hydrostatic, geostrophic, gradient, Guldberg-Mohn, and the thermal wind balance. Between these balances we can describe most of the flows that occur in the atmosphere, especially the horizontal wind that dominates most atmospheric motion.

Note
When talking about vectors, whether a force vector or a wind vector, it is important to remember that they have both a magnitude and direction. This is especially useful when thinking about explaining how the wind will blow. Any vector can be broken down into its components from which the magnitude (e.g., speed of the wind) and direction can be calculated.

Hydrostatic Balance

We begin with the hydrostatic balance, which is a force balance between the vertical pressure gradient force and gravity. There is always a vertical pressure gradient force and it is highest at the surface and decreases exponentially with increasing altitude. This balance can be written algebraically as,

$$-\frac{1}{\rho}\frac{\Delta p}{\Delta z} = g \tag{9}$$

On average, the atmosphere can be approximated to be in hydrostatic balance (i.e., there is little vertical motion) as average vertical motions are on the order of centimeters per second. However, there are certain atmospheric phenomenon that 'violate' hydrostatic balance, resulting in considerable vertical motion ($> 1\ m\ s^{-1}$).

What atmospheric phenomenon (that you can see) violates hydrostatic balance?

Hydrostatic balance can also be visualized with a force balance diagram (Figure 16) that depicts the different forces and how they are acting on an air parcel in the vertical dimension.

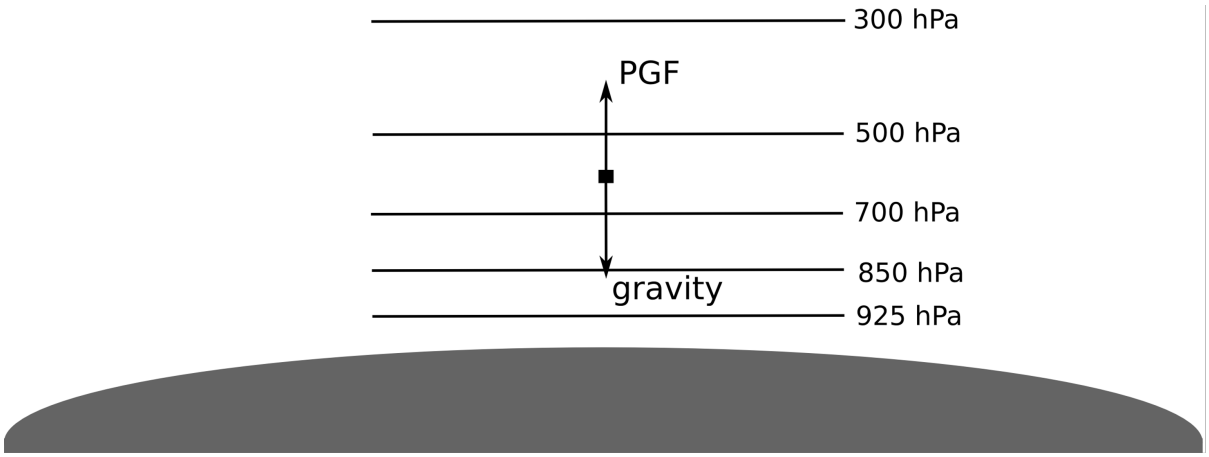


Figure 16: A force balance diagram illustrating the forces in hydrostatic balance.

Geostrophic Balance

One very common horizontal force balance is the geostrophic balance. Flows in the atmosphere are often over large enough temporal and spatial scales that bring the Coriolis force into play. In the upper-levels (i.e., generally above 700 hPa) when the flow is basically straight (i.e., not strongly curved) the atmosphere can be approximated to be in geostrophic balance. The balance is between the pressure gradient (height gradient) force and the Coriolis force (Figure 17) and can be written algebraically for a constant height surface as,

$$ 0 = -\frac{1}{\rho}\frac{\Delta p}{\Delta x} + f v_g
0 = -\frac{1}{\rho}\frac{\Delta p}{\Delta y} - f u_g $$

where the subscript g signifies the geostrophic wind. Note how we also choose to separate the wind into u_g and v_g components, symbolizing west/east and south/north flow respectively. We can rewrite the right-hand side of the force balance for geopotential height as,

$$ 0 = -g\frac{\Delta z}{\Delta x} + f v_g
0 = -g\frac{\Delta z}{\Delta y} - f u_g $$

where $g\Delta z = \Delta\Phi$ and is known as geopotential height. Solving for v_g and u_g yields an equation for the geostrophic wind on a constant pressure surface,

$$ v_g = \frac{g}{f}\frac{\Delta z}{\Delta x}
u_g = -\frac{g}{f}\frac{\Delta z}{\Delta y} $$

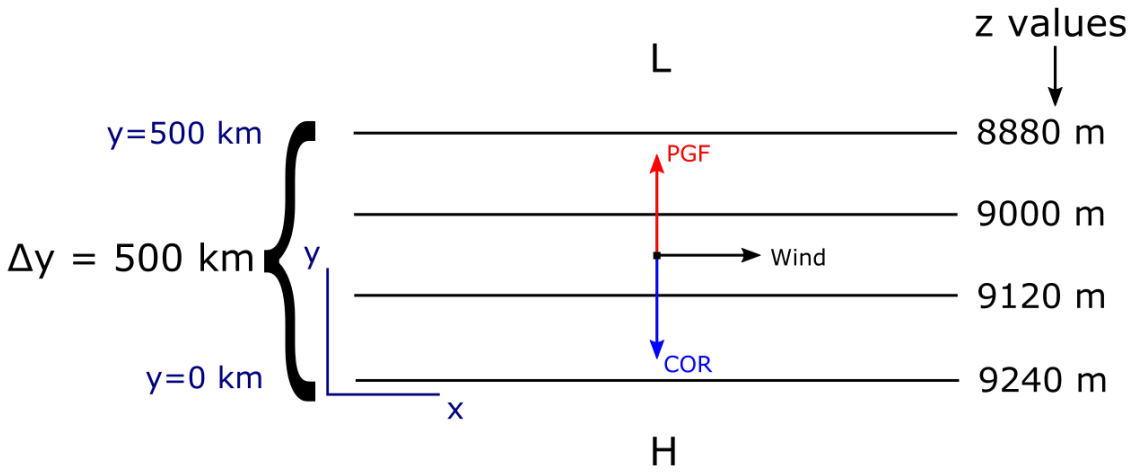


Figure 17: Demonstration of geostrophic balance with example force balance diagram and annotated figure elements.

Example Geostrophic Wind Calculation Having these equations are great, but what can we do with them? We can use them to calculate the values of the wind components by using the geopotential height gradient, the Coriolis parameter, and the value of gravity. We'll use the values from the illustration in Figure 17 to obtain our values and complete our calculation.

> **Note**
> Be careful when setting up calculations to be sure that you are doing your differencing in the same way. If you don't, then you can end up with a sign error (e.g., a negative when you should be getting a positive).

u_g Calculation

$$\Delta z \text{ (y-direction)} = 8880\ m - 9240\ m = -360\ m \tag{10}$$

$$\Delta y \;=\; 500 \cdot 1000\ m \;-\; 0\ m \;=\; 500000\ m \tag{11}$$

$$g = 9.81\ m\ s^{-2} \tag{12}$$

$$u_g = -\frac{9.81\ m\ s^{-2}}{10^{-4}\ s^{-1}} \frac{-360\ m}{500000\ m} = 70.6\ m\ s^{-1} \tag{13}$$

v_g Calculation

$$\Delta z \text{ (x-direction)} \;=\; 9000\ m \;-\; 9000\ m \;=\; 0\ m \tag{14}$$

$$v_g = \frac{9.81\ m\ s^{-2}}{10^{-4}\ s^{-1}} \frac{0\ m}{500000\ m} = 0\ m\ s^{-1} \tag{15}$$

This is because there is no change in the geopotential height values in the x-direction.

Gradient Balance

In curved flow, geostrophic balance does not apply because of the addition of the centrifugal force. Therefore, the gradient wind balance is a three-way balance between the pressure gradient force, Coriolis force, and the centrifugal force (Figure 18). This balance applies in upper-level troughs and ridges and can be written algebraically as,

$$ 0 = -\frac{1}{\rho}\frac{\Delta p}{\Delta x} + f V + \frac{V^2}{R}$$
$$0 = -\frac{1}{\rho}\frac{\Delta p}{\Delta x} - f U + \frac{U^2}{R} $$

This equation can be solved using the quadratic formula. If the pressure gradient were equal around both a high and a low, gradient wind balance would dictate that winds around a high would be supergeostrophic (faster than the geostrophic balance would predict) and subgeostrophic (slower than the geostrophic balance would predict) around a low.

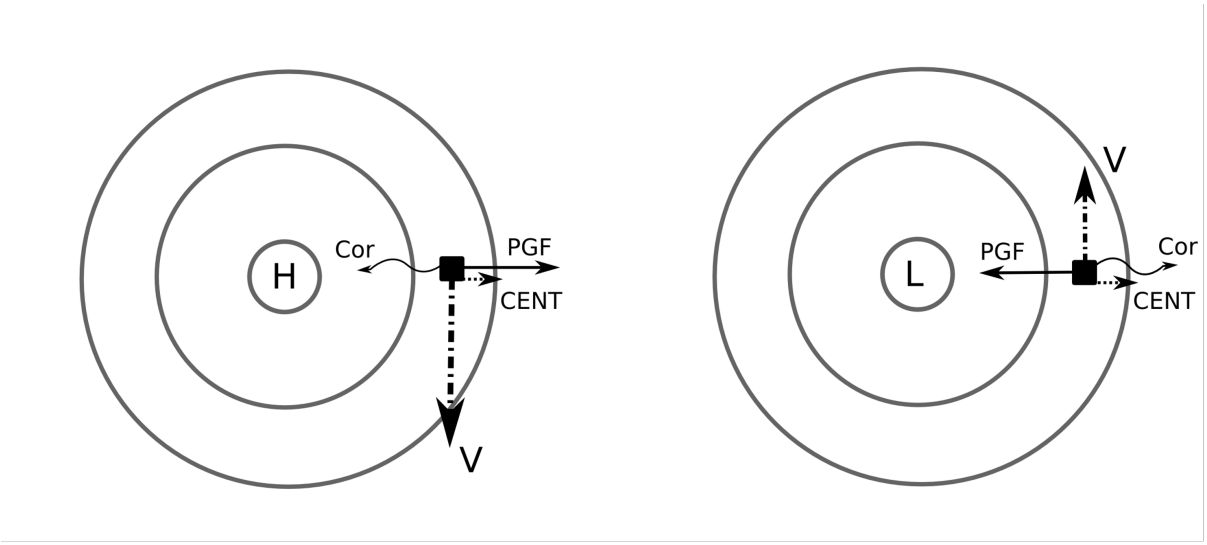


Figure 18: A force balance diagram illustrating Gradient wind balance.

Note

When using the quadratic formula to solve for the gradient wind, to make it easier, you'll want to think about how you can define your different values of a, b, and c from the prototypical quadratic formula.

If a typical quadratic equation can be written as,

$$av^2 + bv + c = 0 \tag{16}$$

then you can use the following quadratic formula to solve for v,

$$v = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{17}$$

Guldberg-Mohn Balance (Surface Balance)

At and near the surface, friction begins to affect atmospheric flows by slowing the wind. Because friction acts to slow the wind, the Coriolis force subsequently does not turn the wind as much and therefore the wind will flow across isobars. This is known as the Guldberg-Mohn force balance and can be written algebraically as,

$$ 0 = -\frac{1}{\rho}\frac{\Delta p}{\Delta x} + f V - k U

0 = -\frac{1}{\rho}\frac{\Delta p}{\Delta y} - f U - k V $$

The greater the roughness of the surface, the stronger the friction, the greater the angle that the flow will cross the isobars. In Figure 19 notice that near the center of low pressure just north of Nova Scotia, CN, there are a number of wind barbs that are crossing the isobars and heading into the low center. The friction near the surface is causing the speed of the wind to slow down, thus reducing the overall Coriolis force deflection and yielding a wind that is not parallel to the isobars and ultimately goes into a low.

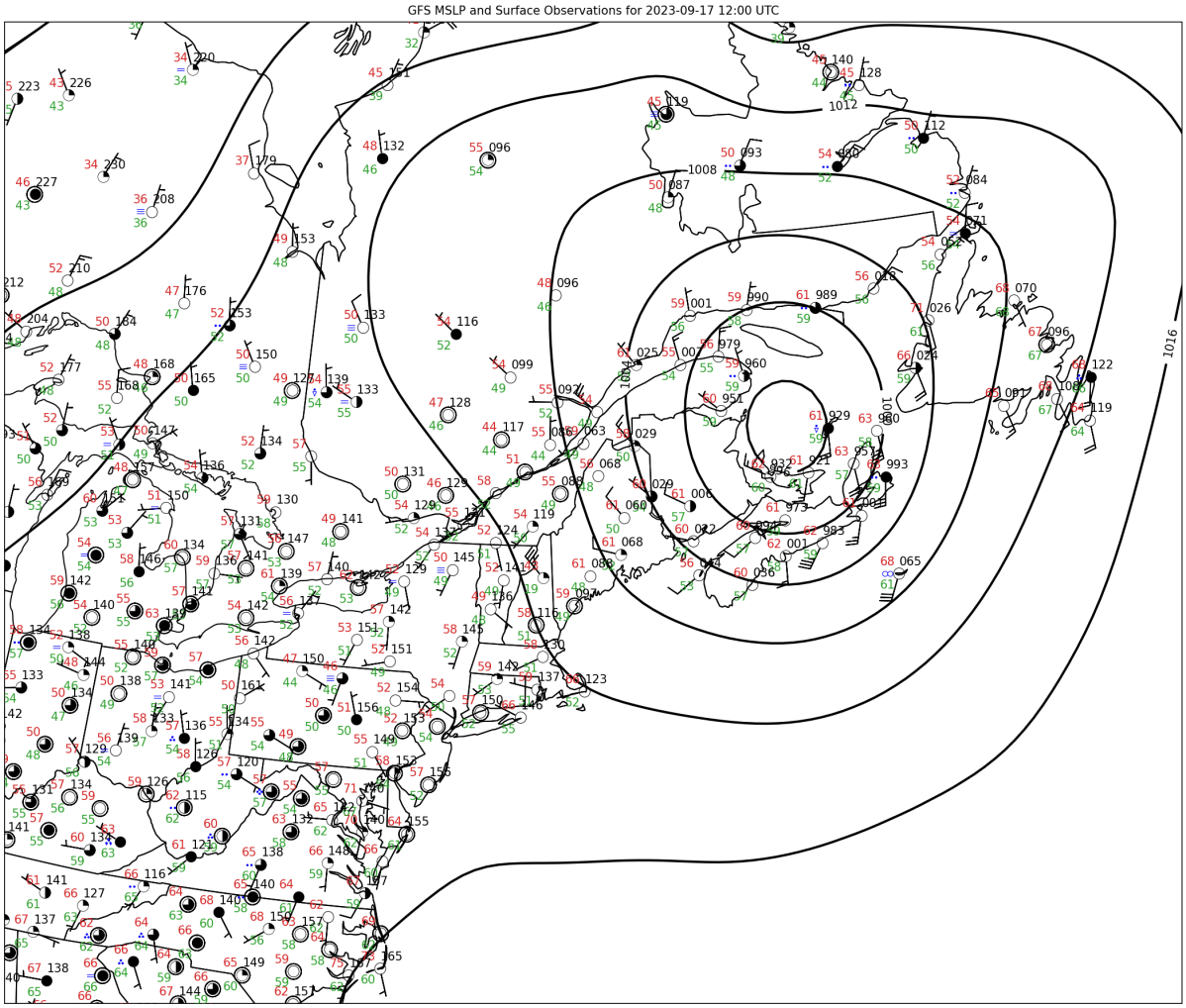


Figure 19: Demonstration of Guldberg-Mohn surface balance with actual observations and GFS MSLP from 17 September 2023 at 12 UTC.

Thermal Wind Balance

The thermal wind is a combination of the hydrostatic balance and geostrophic balance and explains why winds typically intensify with increasing height (stronger winds from the west) in the mid-latitude regions (Figure 20). This balance brings together the relationship between temperature and winds. Cooler surface temperatures mean a shorter vertical distance to a given pressure level, whereas higher surface temperatures would mean a longer vertical distance to a given pressure level. This results in strong pressure gradients aloft, which is in association with the strong jet stream winds observed in the upper troposphere.

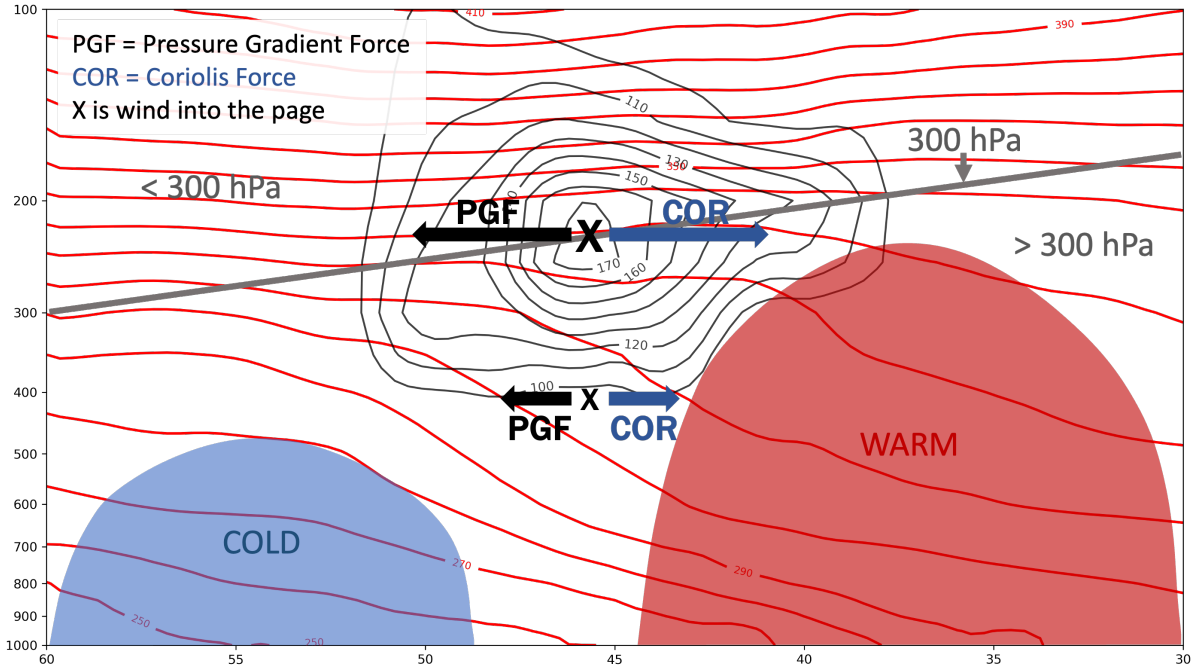


Figure 20: Visualization of thermal wind balance where in the colder air pressure decreases more reapidly that in warmer air creating a strong pressure gradient force that increases with height and is maximuzed near the top of the troposphere (near 300 hPa).

0.3.8 Color-filled Plotting

We are not relegated to only plotting contours of values, we can color-fill a variable for our plots. There are a couple of different options we have including color-filled contour, image plots, and raster plots. The choice will depend on what you are looking to plot and the aesthetic of the output image. We'll demonstrated color-filled plots using `FilledContourPlot()` and `RasterPlot()` here and you can look to examples on the web for others as desired.

Color-filled Plotting

Plotting filled contours with `FilledContourPlot()` works very similarly to `ContourPlot()`, except that contours are filled using a colormap between contour values. All attributes for `ContourPlot()` work for `FilledContourPlot()` plots, except for linestyle, linecolor, and linewidth. Additionally, there are the following attributes that work for the various color-filling plot types:

Specific Color-fill Attributes

- colormap
  - This attribute is used to set a valid colormap from either Matplotlib or MetPy:
    - Matplotlib Colormaps
    - MetPy Colormaps
- colorbar
  - This attribute can be set to 'vertical' or 'horizontal', which is the location the colorbar will be plotted on the panel.
- image_range
  - A set of values indicating the minimum and maximum for the data being plotted. This attribute should be set as (min_value, max_value), where min_value and max_value are numeric values.

Raster plot style use `RasterPlot()` for coloring in a cell with a given color for a value, creating a rasterized image (think color-filled pixels the size of a grid cell). This is a good alternative to color-filled contouring, especially as the size of the grid gets smaller.

Example of `RasterPlot()` use here.

One can also plot an image (e.g., satellite imagery), which uses the `ImagePlot()` class. This plot style is also a type of rasterized image. This will be used for plotting satellite imagery, which will be demonstrated later but could be used for gridded output from a model as well.

Armed with this palette, we can now use scripting and create very nice images using a suite of model output!

Color-filled Contouring

This notebook creates an example of overlaying color-filled contours and regular contours over the CONUS.

Import Packages

```
from datetime import datetime

from metpy.plots import declarative
from metpy.units import units
import xarray as xr
```

Get Data Here we use data available from the NCAR Geoscience Data Exchange (GDEX) from the quarter degree GFS.

You can find other data available at this resource: https://gdex.ucar.edu

```
# Set the date/time of the model run
date = datetime(2017, 2, 24, 12)

# Remote access to the dataset from the GDEX Archive
ds = xr.open_dataset('https://tds.gdex.ucar.edu/thredds/dodsC/files/g/d084001/'
                     f'{date:%Y}/{date:%Y%m%d}/gfs.0p25.{date:%Y%m%d%H}.f000.grib2')

# Subset data to be just over the U.S. for plotting purposes
ds = ds.sel(lat=slice(70,10), lon=slice(360 -140, 360 -60)).metpy.parse_cf()
```

Plot Data Here we demonstrate the use of `FilledContourPlot()` for having the computer draw color-filled contours of a variable. Many of the attributes are the same as `ContourPlot()`, but there are also other specific attributes for setting things like colorbars and a colormap. Only a small subset of atttributes are used in this example, but your plot can be further customized using a host of different attributes, which you can read about in the MetPy documentation.

MetPy `FilledContourPlot()` Documentation

```
# Set attributes for plotting contours
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'Pressure_reduced_to_MSL_msl'
cntr.level = None
cntr.time = date
cntr.contours = range(0, 2000, 4)
cntr.clabels = True
cntr.plot_units = 'hPa'
cntr.smooth_field = 4 # Smooth the contours

# Set attributes for plotting filled contours
cfill = declarative.FilledContourPlot()
cfill.data = ds
cfill.field = 'Temperature_height_above_ground'
cfill.level = 2 * units.m
cfill.time = date
cfill.contours = range( -38, 103, 2)
cfill.colormap = 'coolwarm'
cfill.colorbar = 'horizontal'
```

```
cfill.plot_units = 'degF'

# Set attributes for plotting wind barbs
barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = date
barbs.field = ['u -component_of_wind_height_above_ground',
               'v -component_of_wind_height_above_ground']
barbs.level = 10 * units.m
barbs.skip = (2, 2)
barbs.plot_units = 'knot'

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 22, 52]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.left_title = f'MSLP (hPa), 2 -m TMPF, 10 -m Wind Barbs (kts)'
panel.right_title = f'Valid Time: {date:%d %B %Y %H UTC}'
panel.plots = [cntr, cfill, barbs]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Smoothing Data Fields

There are times you might want to smooth the raw values from your gridded output to generate contours that better reflect what you would produce through a hand analysis. There are a couple of functions that you can use to accomplish this goal in MetPy, but we'll look at one here.

One method of smoothing involves using points surrounding a given grid point and averaging them in a thoughtful way to produce a grid of values that vary in a more gradual manner. This *n-point smoothing* available in MetPy can use either 5 or 9 points and can be run successively one or more times. The more times smoothing is done, the less nuance will appear in the contours, eventually not well representing reality. There are two different options to smooth the look of plotted variables, `smooth_field` and `smooth_contour`.

Smooth Field

Use the smooth_field smoothing for data with finer resolutions (e.g., smaller grid spacings with a lot of grid points). This will produce smoothed geopotential heights for all levels available in the DataArray using the nine-point smoother twice or the number of iterations given as an integer.

MetPy `smooth_field` Documentation

Here is an example (Figure 21) of using `smooth_field`, notice how the black contours, which have had no smoothing applied, are not smooth over the Rocky Mountain region whereas the red contours, which have been smoothed through four iterations, are noticeably smoother over the mountain region.

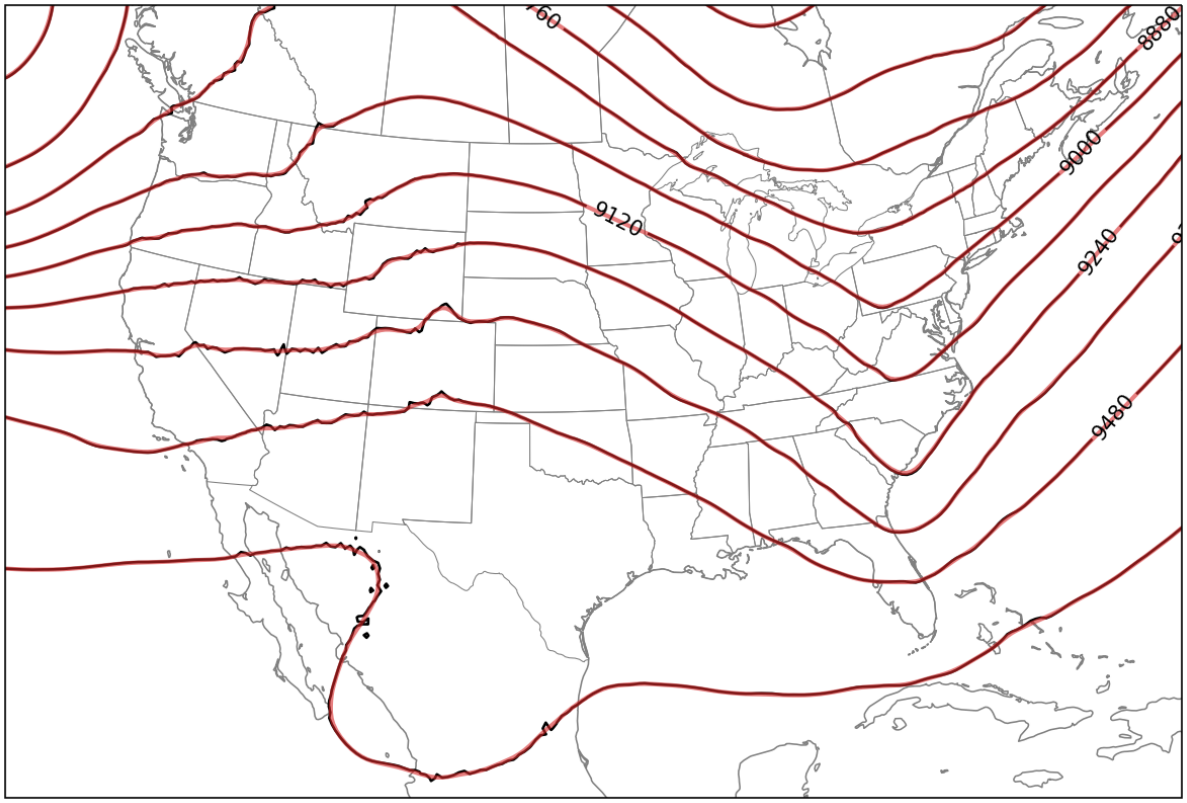


Figure 21: This map shows 300-hPa heights in solid black contours and smoothed contours using the smooth_field attribute in red.

Smooth Contour

The attribute is best used to smooth contours when contouring a sparse grid (e.g., when your data has a large grid spacing). This will use a spline interpolation.

MetPy `smooth_contour` Documentation

Here is an example (Figure 22) of using `smooth_contour`, notice how the black contours, which have had no smoothing applied, have clear vertex points (e.g., angles) to the contours whereas the red contours, which have been refitted with a zoom factor of four, are much smoother.

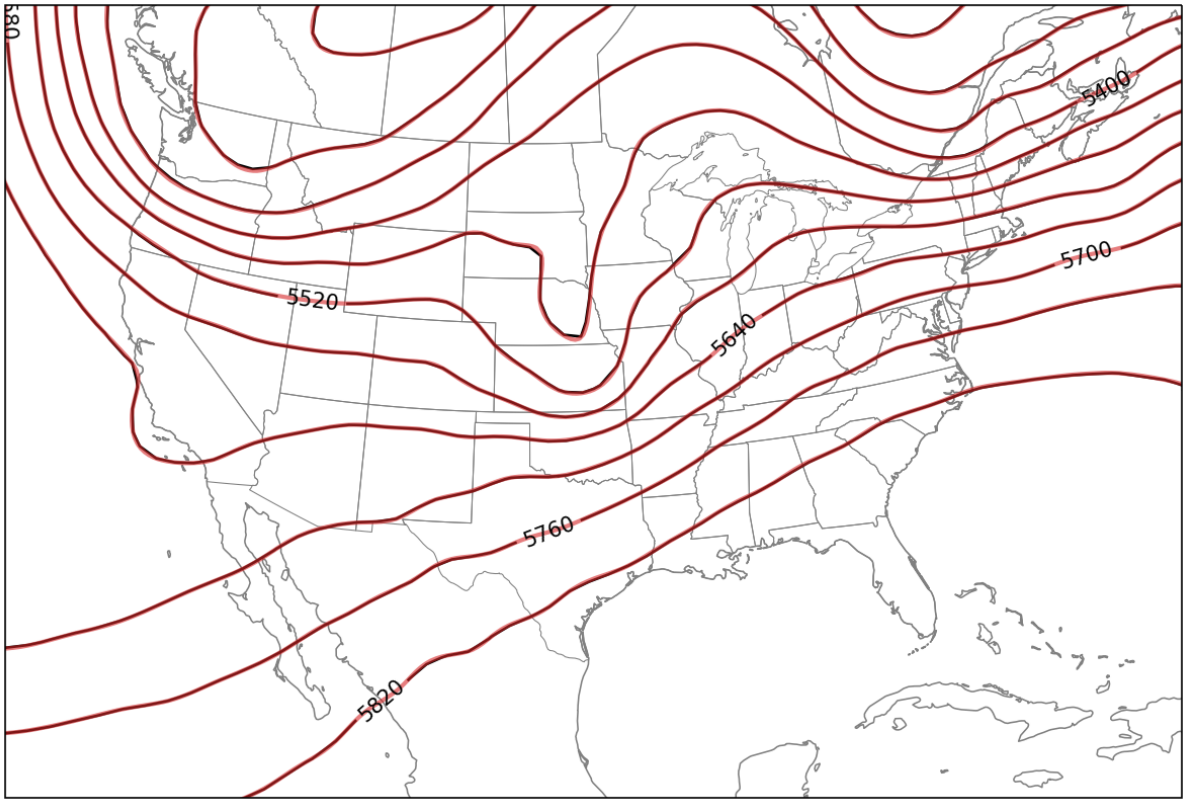


Figure 22: This map shows 500-hPa heights in solid black contours and smoothed contours using the smooth_contour attribute in red using a zoom factor of 4.

Practice Tasks - Color-filled Plotting

1. Create a 500-hPa map with height contoured in solid black, and temperature in Celsius in color-filled contours.

2. Add winds to your map.

3. Create a map 850-hPa geopotential heights in solid black and temperature in Celsius color filled. (You'll need to convert the temperatures to the appropriate scale, which would work similarly to what we did for the example color-filled plot.)

4. Make the above map, but use `RasterPlot` instead.

0.3.9 Life Cycle of Mid-Latitude Cyclones

Introduction

Before the publication of Bjerknes and Solberg [1922], the meteorological lexicon did not have cold or warm fronts, or the idea that mid-latitude cyclones evolved through a life cycle. In many different scientific disciplines one can learn about the life cycle of cells, animals, stars, etc. This advancement in meteorological knowledge did not come out of nowhere. Throughout the latter parts of the 19$\hat{\text{th}}$ĉentury and the beginning of the 20$\hat{\text{th}}$, many meteorological researchers were slowly modifying the way we view the atmosphere, but it was the work of the Bergen School that solidified this evolving view.

The revelation that all mid-latitude cyclones were moving and evolving through a roughly structured life cycle had large ramifications for forecasting. With the advent of the life cycle of cyclones one could use that life cycle to predict how the low-pressure system would change based on its current state. While we have progressed and modified the simple life cycle presented in 1923, the foundations of using maps to understand how weather patterns evolved have only grown with time. This lab handout will build upon the ideas of Bjerknes and Solberg [1922] to develop a complete basic conceptual model of the life cycle of mid-latitude cyclones.

Life Cycle of Cyclones

In the time since the Bergen School published the first theories of cyclone life cycles, meteorologists have learned that it is vitally important to understand the evolution of the upper-air pattern in association with the mean sea-level pressure pattern. We will start with understanding the evolution of the characteristics of surface values of temperature and mean sea-level pressure and its association with a 500-hPa trough.

Pre-Environment Stage

Most cyclones form over an area that has at least some kind of surface temperature gradient. As you will learn about later in your meteorology coursework, a surface temperature gradient is not absolutely needed to form a cyclone. However, cyclones will preferentially form over regions that have some weak temperature gradient, as it is easier for the atmosphere to concentrate a weak temperature gradient rather than forming a temperature gradient where there was none.

The pre-environment stage is defined by a weak temperature gradient (if one exists at all), where there may or may not be defined a stationary boundary (Figure 23). In general, there is little temperature advection near the surface and only a small depression of lower pressure. In general, there is also a moisture gradient between the two different airmasses. In the Northern Hemisphere to the north is the dry continental airmass and to the south the moist tropical airmass.

Associated with the temperature gradient at the surface there is a need for an upper-level trough to initiate the cyclogenesis stage (Figure 23). This additional requirement differs from the classic Norwegian Cyclone Model (NCM) because of now having upper-level observations. The upper-level trough will initiate appropriate divergence and subsequent rising motion to the *east* of the trough axis. This will eventually result in less mass within the column, which will mean the surface pressure will decrease, initiating our mid-latitude cyclone.

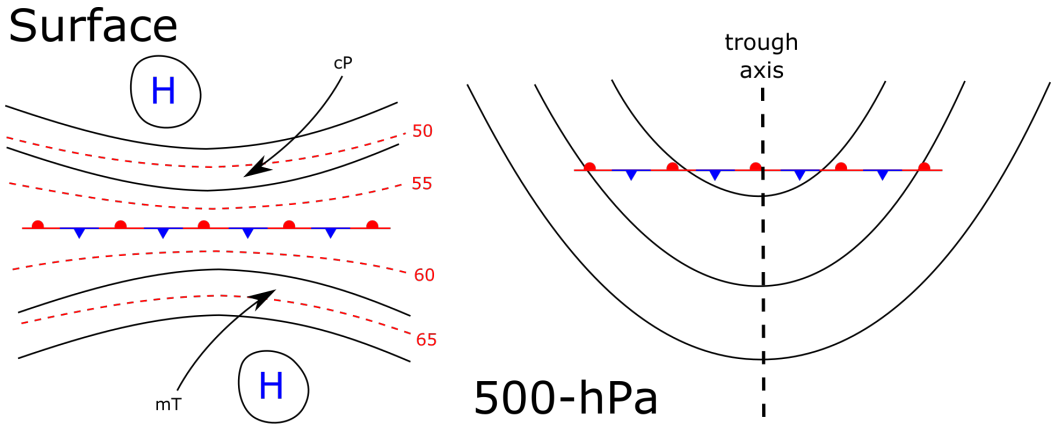


Figure 23: A schematic diagram of the pre-environmental conditions over the region where cyclogenesis will take place at the surface and 500 hPa with location of low-level fronts for spatial reference.

Cyclogenesis Stage

For cyclogenesis to occur at the surface, a wave in the upper-levels must come along to provide the necessary support to strengthen the low-level temperature gradient. This stage is signified by an increasing temperature gradient, the beginning of cyclonic flow at the surface, and a small decrease in the mean sea-level pressure (Figure 24). There are now fronts defined with a warm front typically east of the cyclone center and a cold front south and west of the cyclone center. The pressure troughs in association with the fronts begin to develop.

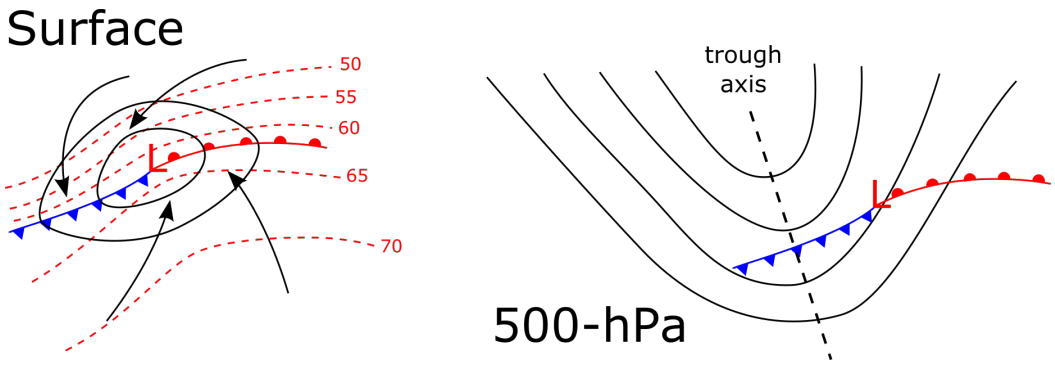


Figure 24: A schematic diagram of a surface cyclone during the cyclogenesis stage of development at the surface and 500 hPa with location of low-level fronts for spatial reference.

As the initial cyclone begins to develop, the trough begins to intensify simultaneously. The intensification results from the different temperature advection surrounding the mid-latitude cyclone. The 500-hPa pattern strengthens (Figure 24) along with the surface cyclone and is evidenced by the deepening of the upper-level trough. Additionally, a typical mid-latitude cyclone exhibits a tilt with height. The typical tilt is where the upper-level trough (e.g., the 500-hPa trough) is located *to the west* of the surface mid-latitude cyclone (Figure 24). This *westward tilt* with height is essential to the continued strengthening of the mid-latitude cyclone.

Mature Stage

The natural tendency for cyclones is to intensify (reduced surface mean sea-level pressure) due to the interaction between low-level temperature advection and upper-level troughs. As the cyclone matures, the temperature gradient will strengthen, the mean sea-level pressure will lower, and wind speeds will in general increase (Figure 25).

During the mature stage our fronts will have the clearest definition through a combination of the temperature structure and wind shift being co-located (Figure 25). The cyclone will be moving roughly along the direction of the isotherms to the east of the cyclone center.

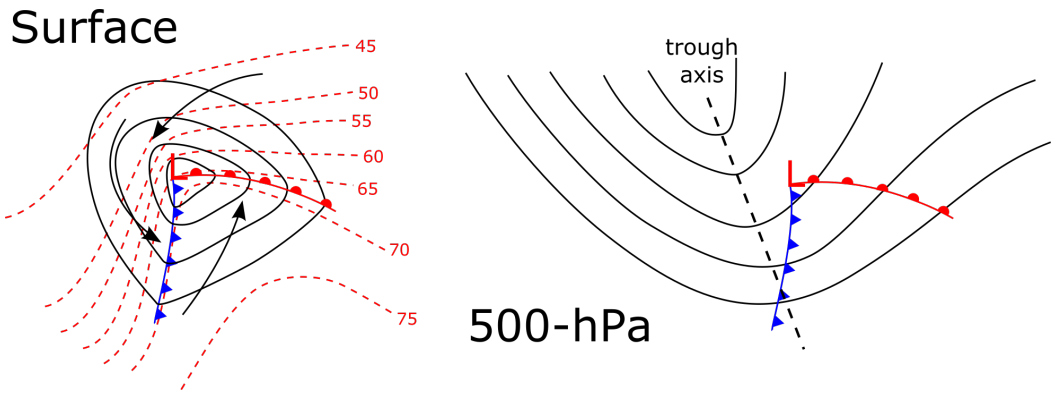


Figure 25: A schematic representation of the mature stage of a mid-latitude cyclone at the surface and 500 hP with location of low-level fronts for spatial reference.

At 500 hPa, the trough has intensified further (Figure 25). This dual intensification of the surface and upper-level pattern is called the *self-development paradigm*. If the surface (or near-surface) cyclone is positioned to the east of the upper-level trough, then the surface cyclone will continue to intensify. The overall tilt has decreased somewhat between the cyclogenesis and mature phase of the cyclone's life cycle. However, some continued development would be expected with this orientation of surface and upper-level patterns.

Occlusion

Near the end of the life cycle of a mid-latitude cyclone the cold air that has been on only the north side of the cyclone will now wrap completely around the center of the circulation (Figure 25). This, in effect, cuts off the cyclone from the moist, tropical airmass that has been bringing energy into the system. This process is called occlusion (e.g., cyclone wrap-up). A result of the occlusion processes the cold front and warm front meet and the remaining wind shift (and possibly slight temperature gradient) can be defined an occluded front. At this point the cyclone may dissipate or form a new low-pressure center at the triple point of the cold front, warm front, and occluded front.

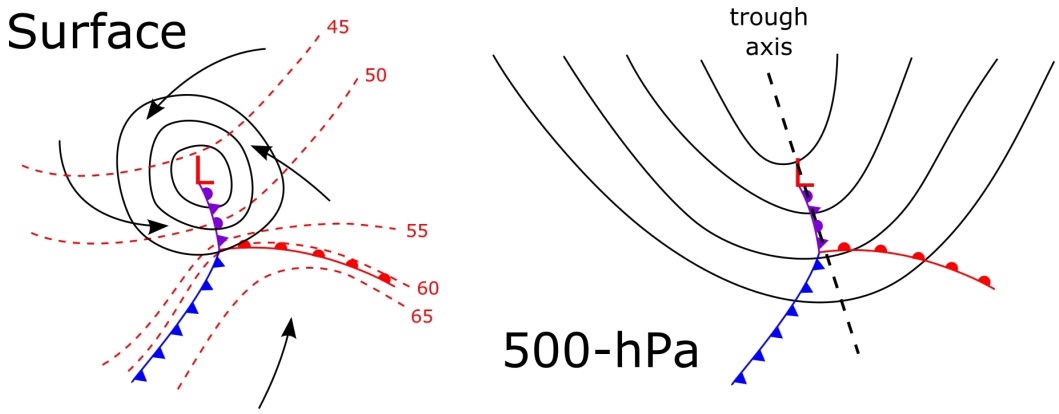


Figure 26: Schematic representation of the occluded stage of a mid-latitude cyclone at the surface and 500 hPa with location of low-level fronts for spatial reference.

Typically, around the time that there is no longer a westward tilt of the cyclone with height the surface system will occlude (Figure 25). The cyclone is said to be vertically stacked at this point. Depending on the orientation of the cyclone, redevelopment may be possible at the triple point of the cold, warm, and occluded fronts. The redevelopment occurs frequently with storm systems, but eventually all systems will dissipate after an occlusion.

Precipitation

Not only does a mid-latitude cyclone potentially bring great changes in temperature for a region, it can also produce copious amounts of precipitation. Depending on your location within the cyclone's path comes a variety of different precipitation characteristics. Not only the amount of precipitation that falls, but the way it falls will determine whether it is beneficial or detrimental to an area.

Before we can get precipitation associated with a mid-latitude cyclone we must first get clouds. There are three main components needed to get cloud formation: (1) cloud condensation nuclei (CCN), (2) a source of moisture, and (3) a source of lift. Generally, there is always plenty of CCN, but without ample moisture, no amount of lift will allow a parcel of air to reach saturation and thus form a cloud.

For precipitation we need these cloud formation processes to continue for some time, such that the droplets (or ice crystals) within our clouds continue to grow. This usually requires added moisture and more lift. The main source of moisture for weather systems in the upper-Midwest is the Gulf of Mexico with the main source of lift associated with our cyclone are the frontal surfaces. The different styles of lift often lead to very different precipitation characteristics.

Warm Front

The gradual slope of the warm front in the lowest layers of the atmosphere enables a slow general lift of airstreams, which sustain a slow, but steady bout of precipitation poleward of the warm front. If the air is cold enough throughout the column, frozen or mixed precipitation is possible (Figure 28). These characteristics hold for any area north of the track of the cyclone.

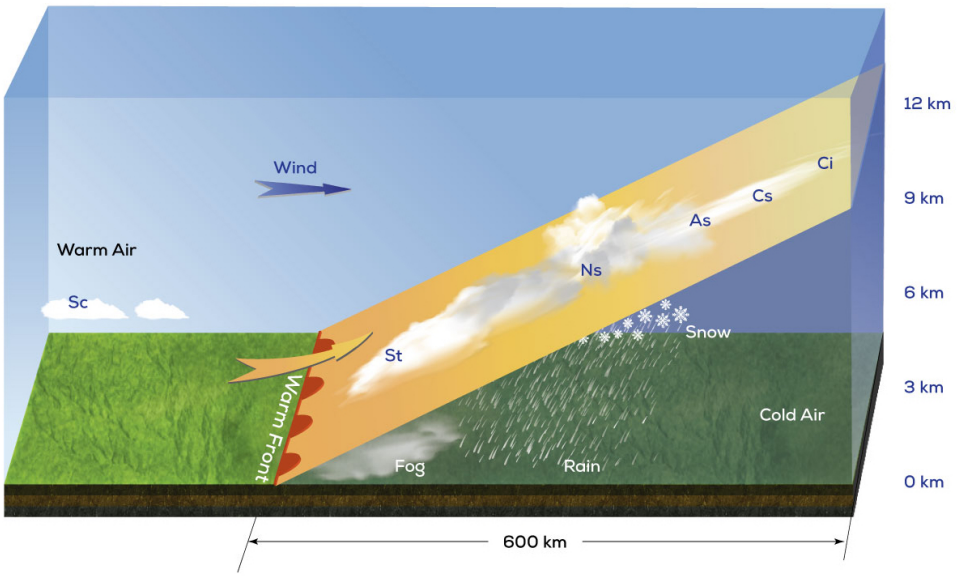


Figure 27: Illustration of a warm front cross-section demonstrating precipitation and cloud structure that could occur poleward its leading edge at the surface.
The cross section through a cold front and warm front image was created by Kh1604 and is licensed under the “Creative Commons Attribution-Share Alike 4.0 International”

Cold Front

The abrupt nose of the cold air in the lowest levels of the atmosphere can act to forcibly lift portions of the warm sector airstreams (Figure 28). This abrupt lift often initiates convective precipitation, which is typically intense. However, due to the generally rapid movement of the front, the heavy precipitation will move through quickly.

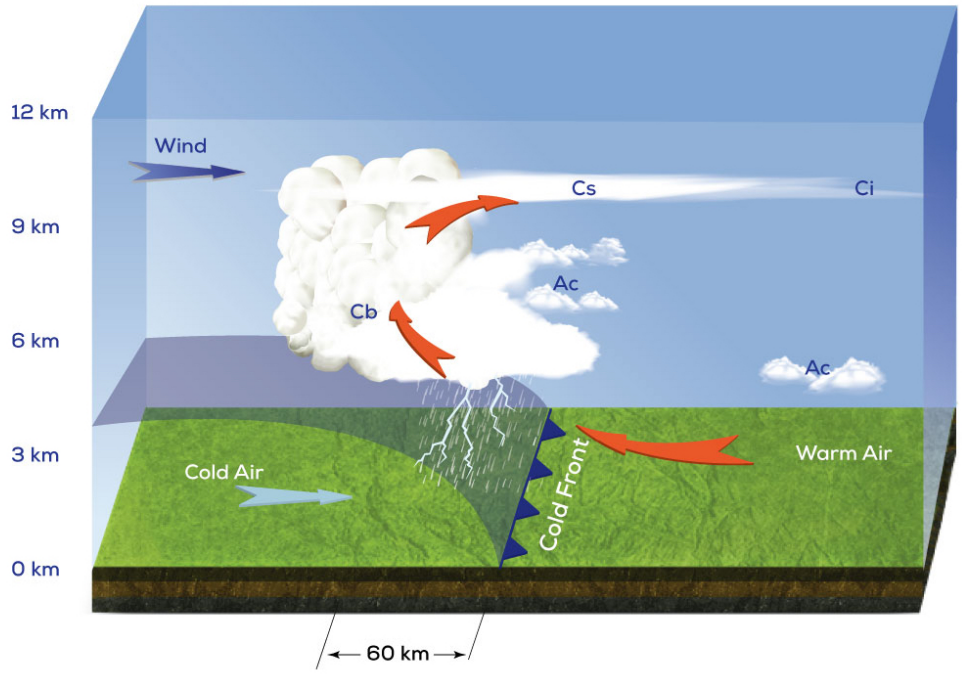


Figure 28: Illustration of a cold front cross-section demonstrating precipitation that could occur along its leading edge.
The cross section through a cold front and warm front image was created by Kh1604 and is licensed under the “Creative Commons Attribution-Share Alike 4.0 International”

Warm Sector Rain

In the warm sector of a cyclone there is typically more than ample moisture for clouds and precipitation, however, the lack of a clear lifting mechanism is often the limiting factor. Depending on the time of year there could be some upper-level support for causing air parcels to rise, thus initiating convective precipitation. Generally, precipitating systems in the warm sector move through areas quickly, possibly inundating an area with heavy rainfall.

Cyclone Tracks

The track of a cyclone is very important as the exact track can make or break a forecast of precipitation. For example, a very powerful mid-latitude cyclone moved up the East coast of the U.S. in March of 1993. Three different weather models had three different tracks and the forecasters had to pick one to determine their forecast. They chose the path that went along the coast, and the storm followed that track. This storm is now known as the Storm of the Century. Had the storm tracked more inland or went further out into the Atlantic Ocean; the storm would not have had the same impact across the U.S., placing the precipitation either more inland or further out to sea.

There are three to four primary tracks of mid-latitude cyclones across the continental U.S. This is not to say that all mid-latitude cyclones will follow only these tracks, but that most cyclones originate and travel well-defined corridors across the country. The three primary tracks are: Alberta cyclones, Denver (Hooker) Cyclones, and East Coast Cyclones (Figure 29).

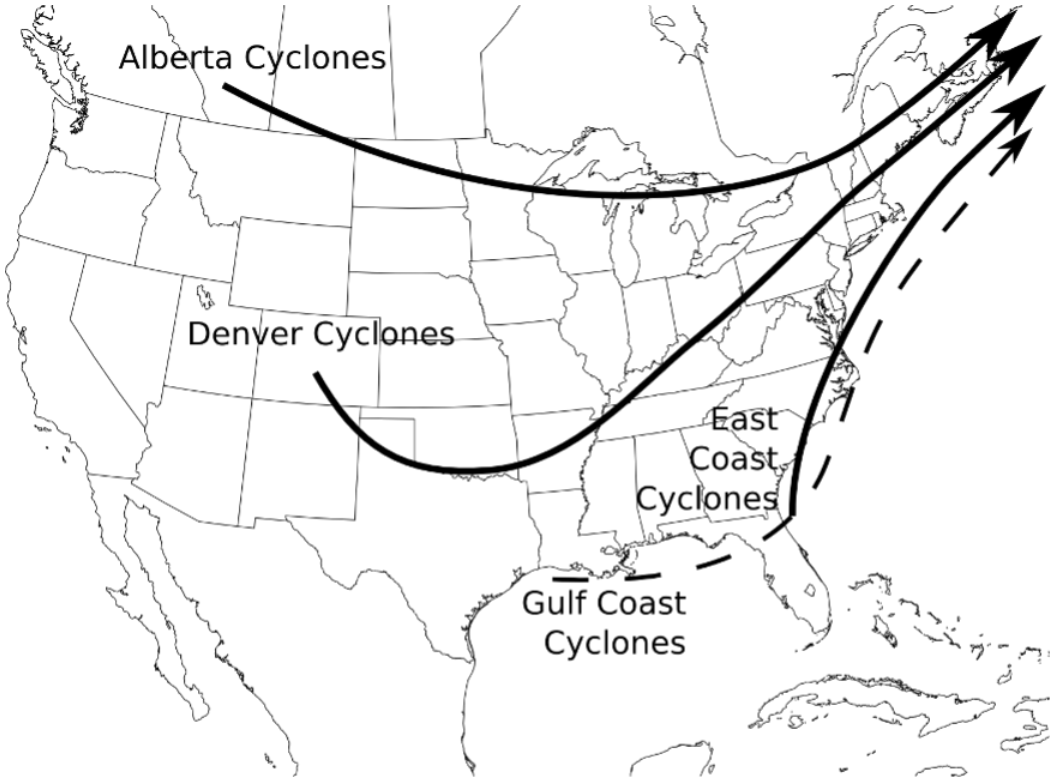


Figure 29: Dominant cyclone tracks across the continental U.S. for typical winter patterns.

So, what's in a track?

The track that a cyclone takes is the first opportunity you have to determine what type of weather a particular area will have as a cyclone moves through. If the track of the center of the storm is going to the north of your location, then at some point that location will be in the warm sector of the storm. This will obviously affect any temperature forecast for the area, but it will likely also play in your precipitation forecast.

Snowstorms For a location to experience a strong snowstorm, the track of the surface low must be to the *south* (but not too far south). For the Valparaiso area, the storm track most likely to allow for a substantial synoptic snowfall would be the Denver cyclone track. This track will typically ingest a lot of moisture from the Gulf of Mexico region, transport it northward and if the cyclone tracks to our south, then we are in the prime territory for snowfall. However, if the track goes through Chicago, then Valparaiso will get into the warm sector and have rainfall as our primary precipitation. Our only hope for snow would be after the cold front passes and the area is under any wrap around precipitation.

Alberta Cyclone Alberta cyclones are generally harbingers of cold to extremely cold air for the upper-Midwest. They can bring snow, although snowfall totals are generally small due to a lack of sufficient moisture and very cold temperatures associated with the cyclone. These storms can produce very strong pressure gradients and thus very strong winds.

Denver Cyclone This is our most likely heavy snow-producing storm. Although the track will tell the tale, if a given location is 100–200 km north of the cyclone track, then it will be in position to receive snowfall. If a location falls to the south of the forecast cyclone track, then any snowfall will likely be minimal.

East Coast or Gulf Coast Cyclones If these cyclones track directly along the coast, they will have an ample supply of moisture that will feed into the storm. Depending on the time of year and exact temperature structure, there may or may not be substantial snow associated with this cyclone. Strongest precipitation is often along the cold front or within 100–200 km of the track of the cyclone (on the poleward side). Gulf coast and East coast cyclones generally do not give any precipitation to the upper-Midwest region; however, cold air will usually come into the region with a strong cyclone following these tracks.

0.3.10 Overlay Observations and Contours

If you desire to plot both observations and computer drawn contours it will be necessary to combine the plotting classes for both observations and the type of gridded display that you desired (e.g., contours). Below is an example of how to create a 500-hPa map with both observations and heights contoured. The overall procedure of this example can be extended to surface maps or other pressure surfaces with one or more contours.

The way to think of these different plot elements as different layers to your picture. Python is generally pretty good about ordering all of the elements in the end to appear correctly on your figure, however, when putting in your plot order in the MapPanel(), put any color-filled plots first and then list the remaining plots after to ensure you can see everything you desire to plot!

Overlay Observations and Contours

Using the declarative syntax to plot upperair observations and overlay contours.

Import Packages Here we'll need to import all of the packages to read both observation data and some form of gridded data. Here we'll plot some upperair maps, so we'll read current observations from Iowa State archive with current GFS model output to have the computer draw contours.

```
from datetime import datetime, timedelta, time, UTC

import cartopy.crs as ccrs
from metpy.io import add_station_lat_lon
from metpy.plots import declarative
from metpy.units import units
import numpy as np
from siphon.simplewebservice.iastate import IAStateUpperAir
import xarray as xr
```

Get Observations Data Here we'll get the observation data from the Iowa State archive, add some latitude/longitude data to those observations and remove an errant data point that got tagged as 'KVER'.

```
# Set the date/time of the model run
# The following code will get you yesterday at 12 UTC
yesterday = datetime.now(UTC) - timedelta(days=1)
date = datetime.combine(yesterday, time(12))

# Request data using Siphon request for data from Iowa State Archive
data = IAStateUpperAir.request_all_data(date)

# Add lat/lon information to dataframe, drop missing station lat/lons
df = add_station_lat_lon(data, data.station.name).dropna(subset=['latitude', 'longitude'])
df = df[df.station != 'KVER']
```

Get Gridded Data Here we'll get the associated gridded output to be able to draw contours from Unidata. Here we use the one degree GFS output.= and subset to being over the CONUS.

```
# Get GFS data for contouring
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib/NCEP/GFS/'
                     f'Global_onedeg_ana/GFS_Global_onedeg_ana_{date:%Y%m%d_%H%M}.grib2')

# Subset data to be just over the CONUS
ds = ds.sel(lat=slice(80, 10), lon=slice(360 -140, 360 -40))
```

Plot Obs and Contours To plot our overlay all we need to do is set up both our attributes for plotting the observations and for drawing contours, then add both pieces and put it on a map figure.

```
# Add point observations
obs = declarative.PlotObs()
obs.data = df
obs.level = 500 * units.hPa
obs.time = date
obs.fields = ['temperature', 'dewpoint', 'height']
obs.locations = ['NW', 'SW', 'ENE']
```

```
obs.vector_field = ['u_wind', 'v_wind']
obs.vector_field_length = 7

# Add contours of geopotential height
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.level = obs.level
cntr.time = date
cntr.field = 'Geopotential_height_isobaric'
cntr.clabels = True
cntr.contours = range(0, 10000, 60)
cntr.smooth_field = 3
cntr.smooth_contour = 5

# Set map panel features
panel = declarative.MapPanel()
panel.projection = 'lcc'
panel.area = [ -124, -72, 22, 53]
panel.layers = ['ocean', 'lakes', 'land', 'states', 'borders', 'coastline']
panel.left_title = f'{obs.level.m} -hPa Observations and Geopotential Height'
panel.right_title = f'Valid Time: {date}'
panel.title_fontsize = 14
panel.plots = [cntr, obs]

# Add map panel to figure
pc = declarative.PanelContainer()
pc.size = (18, 18)
pc.panels = [panel]

pc.show()
```

0.3.11 Numerical Weather Prediction

Meteorology is all about forecasting. Whenever you say that you are a meteorologist or meteorology major, the first thing anyone asks is, "what's it going to be like tomorrow"? So how do we forecast?

Our primary tool in forecasting future weather is to use a numerical weather prediction (NWP) model. These models are finite mathematical representations of the governing equations of the atmosphere that we can arrange to solve for a future time. What is most important is that we get a set of fields (variables) that are valid at future times. Through the diagnoses of the variables at the future time we are able to make a forecast (prediction) of what the most likely state of the weather will be for a given area. But it is only a tool and your own meteorological knowledge and growing expertise will be another critical tool to help you decipher when a NWP model is performing well and when it might be missing something critical to a given forecast.

NWP Overview

There are different ways to set up a model, we'll describe some of the basics of numerical weather prediction using grid cells (Figure 30). In the figure below, a global model representation depicts a a three-dimensional grid that carves up the atmosphere into cubes. The same would be true for a regional model over the limited domain of that model, instead of the full globe.

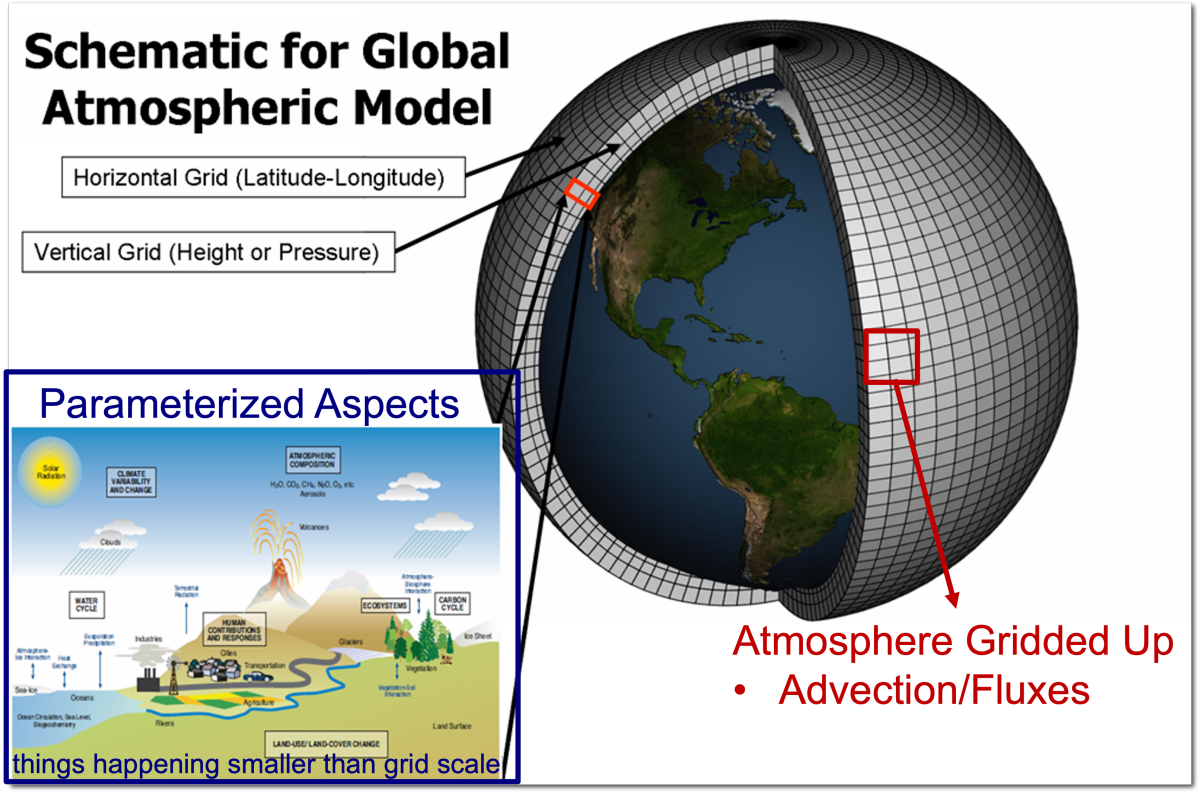


Figure 30: This annotated schematic illustrates different components to a global model. Annotated version of image from https://en.wikipedia.org/wiki/Climate_model#/media/File:Global_Climate_Model.png

Model Resolution The size of the model grid cubes plays a large role in what weather phenomenon will be well represented. Something becomes resolved in a model when the size of the feature is at least eight times bigger than the grid spacing. For example, if the spacing between the grid cubes are 13 km, then only a feature bigger than 104 km would have the chance of being accurately represented within the model. As the grid spacing gets smaller, then a model will be able to more accurately represent smaller features. But there are still lots of things happening in the atmosphere that are smaller, much smaller, than even the grid spacing of any feasible forecasting model. The main aspect that is represented on our grid scale in a model are advections and fluxes.

Parameterization For those elements that would be too small to be represented on a grid, those elements are said to be parameterized. Things like incoming solar radiation, outgoing longwave radiation, cloud microphysics, precipitation, convection (in some instances), turbulence, surface-land interactions, etc. These are still represented mathematically in order to obtain the most accurate representation of the current and future state of the atmosphere.

Convection Allowing Models (CAMs) There are a current suite of mesoscale models that have small enough grid spacings that at least some convection can be resolved on the grid (grid spacings <= 4 km>). In these instances, convection is NOT parameterized, or at least not fully parameterize. Generally, these kinds of models give a better representation of high intensity rainfall that can be

produced by thunderstorms when compared to models that require a parameterization. However, due to the inherent limitations in modeling the future state of the atmosphere, the time, location, and structure of any modeled convection could be wrong.

There is so much more that goes into NWP and being able to produce high quality, accurate forecasts from these tools that go beyond the scope of this text. I encourage you to seek out additional information from other courses, COMET modules, and other resources to understand how and why these models are constructed and work.

Operational NWP Models

There are three primary synoptic models that are used for forecasting across the United States: the Rapid Refresh (RAP), the North American Model (NAM), and the Global Forecast System (GFS) model. All three models have a slightly varying purpose and have different forecast periods, which are summarized below.

	RAP	NAM	GFS
Forecast Hours	48	84	384
Model Type	Grid Point	Grid Point	Finite Volume
Output Time	Every hour	Every three hours	Every six hours
Areal Coverage	Regional (U.S.)	Regional (NA)	Global
Runs	Every hour	00, 06, 12, 18 UTC	00, 06, 12, 18 UTC

For more information on these and other models run by governments across the world check out the Operational Models Encyclopedia.

To access forecast hours from a model file you would simply input how many hours into the future you desire into a timedelta function and add it to your date variable. If model run time is set as,

```
model_run_time = datetime(2020, 2, 20, 12)
```

Then if you wanted the six-hour forecast then,

```
plot_time = model_run_time + timedelta(hours=6)
```

or the 24-hour forecast

```
plot_time = model_run_time + timedelta(hours=24)
```

or the 120-hour forecast

```
plot_time = model_run_time + timedelta(hours=120)
```

0.3.12 Derived Atmospheric Parameters

So far we have focused on plotting variables that exist in a given data file (e.g., temperature, mean sea-level pressure, geopotential heights). However, there may be times when you will want to plot an atmospheric variable (e.g., thickness, temperature advection, relative vorticity, divergence) that is not in the model output file. For that we will have to learn (or remember) a little math and how we can calculate new variables in using MetPy calculations.

In this chapter you will learn about a few of the common derived variables (e.g., atmospheric thickness, vorticity, absolute vorticity, and advection) we can use to describe different aspects of the atmosphere. The following chapter will demonstrate how variables can be easily calculated through either simple mathematical operations or from functions built into the MetPy module. Certain functions will only be able to be used on model output or other ***gridded*** data, and *not* actual surface or upper-air observations. Others will work for point observations only. However, we will start this chapter by describing the importance of some of these important and interesting meteorological parameters.

Atmospheric Thickness

The atmosphere is a compilation of a number of different gases, primarily nitrogen, oxygen, and argon. Atmospheric gases respond to the changing temperature of the atmosphere by expanding or contracting. This expansion or contraction due to a change in temperature is related to the pressure through the ideal gas law, thus the heights of our pressure surfaces will change proportionally to a change in temperature. One way to view the atmosphere then is as the difference of two pressure levels (e.g., 1000 and 500 hPa). The thickness of the layer between the 1000 and 500 hPa surface is proportional to the temperature of that layer. A warm layer will have a larger thickness than a cold layer (Figure 20). The *hypsometic equation* will tell us how "thick" a given portion of the atmosphere will be.

$$Thickness = z_2 - z_1 = \frac{R_d \cdot \bar{T}_v}{g} ln\left(\frac{p_1}{p_2}\right) \tag{18}$$

The thickness between our 1000 and 500 hPa surfaces will simply be the difference between the two geopotential heights of those pressure surfaces (z_1 and z_2). Since the value of gravity (g), the gas constant (R_d), and the pressure levels (p_1, p_2) don't change, the thickness is directly proportional (related) to the average virtual temperature (T_v) of the thickness layer. If we know the height levels of two atmospheric levels, we can directly calculate the thickness.

Note
The use of z_2 would mean the geopotential height from a pressure level that is higher up in the atmosphere (e.g., 500 hPa) compared to the z_1 level (e.g., 1000 hPa).

This can be calculated easily in GEMPAK using the sub function. You can calculate 1000-500 hPa thickness or 850-700 hPa thickness. But you may ask why? Well, since thickness is proportional to average temperature of that layer, thickness can be used as a first guess for precipitation type. If the average temperature of the layer is at or below freezing (273.15 K), then the possibilities of some mix of frozen precipitation increases. For predicting snow, you would need a 1000-500 hPa thickness of no more than 5400 m.

Vorticity

Another variable that meteorologists commonly use is the amount of local spin in the atmosphere or vorticity. There are two primary components of vorticity: relative vorticity and Earth's vorticity. Earth's vorticity we have already talked about in terms of the Coriolis parameter. The relative vorticity is calculated using the following equation in algebraic form:

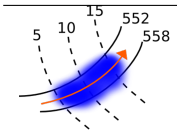
$$\text{vorticity} = \zeta = \frac{\Delta v}{\Delta x} - \frac{\Delta u}{\Delta y} \tag{19}$$

Absolute vorticity is the relative vorticity plus Earth's vorticity.

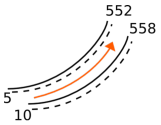
$$\text{Absolute Vorticity} = \eta = \zeta + f \tag{20}$$

Earth's vorticity, f, is simply the Coriolis parameter ($f = 2\Omega \sin \phi$), where ϕ is the latitude.

Displaying the relative or absolute vorticity helps to identify troughs and ridges and will be important to diagnosing vertical motions in dynamic and synoptic meteorology. Many meteorological phenomena are rotating systems, which will all have varying degrees of vorticity.



a) **Max Advection**



b) **No Advection**

Advection

Advection is the horizontal transport of some variable. One of the most common advections that meteorologists investigate is temperature advection. As we saw in the previous section, we can view the change in temperature to be attributable to two different parts, a local time rate of change and an advective component.

$$\frac{\mathrm{DT}}{\mathrm{Dt}} = \frac{\partial T}{\partial t} + \vec{u} \bullet \nabla T = \frac{\Delta T}{\Delta t} + u\frac{\Delta T}{\Delta x} + v\frac{\Delta T}{\Delta y} \tag{21}$$

The second term (boxed) on the right-hand side of the equation is the advective term. This basically states that the wind (u, v) will push the temperature gradient ($\nabla T = \left\langle \frac{\Delta T}{\Delta x}, \frac{\Delta T}{\Delta y} \right\rangle$) around. Depending on the orientation of the temperature gradient and the wind speed and direction we can get warm or cold air advection to occur over a region. Let's look at this schematically to get a better understanding.

If the arrow gives the wind, the left image clearly has cold air advection as the wind is pushing the cold air into the warmer air. The advection of a variable is maximized when the isopleths of the variable being advected is perpendicular to the wind. In the right image, there is no advection since the isotherms are parallel to the wind, thus the wind is not pushing warmer air into colder air or vice versa.

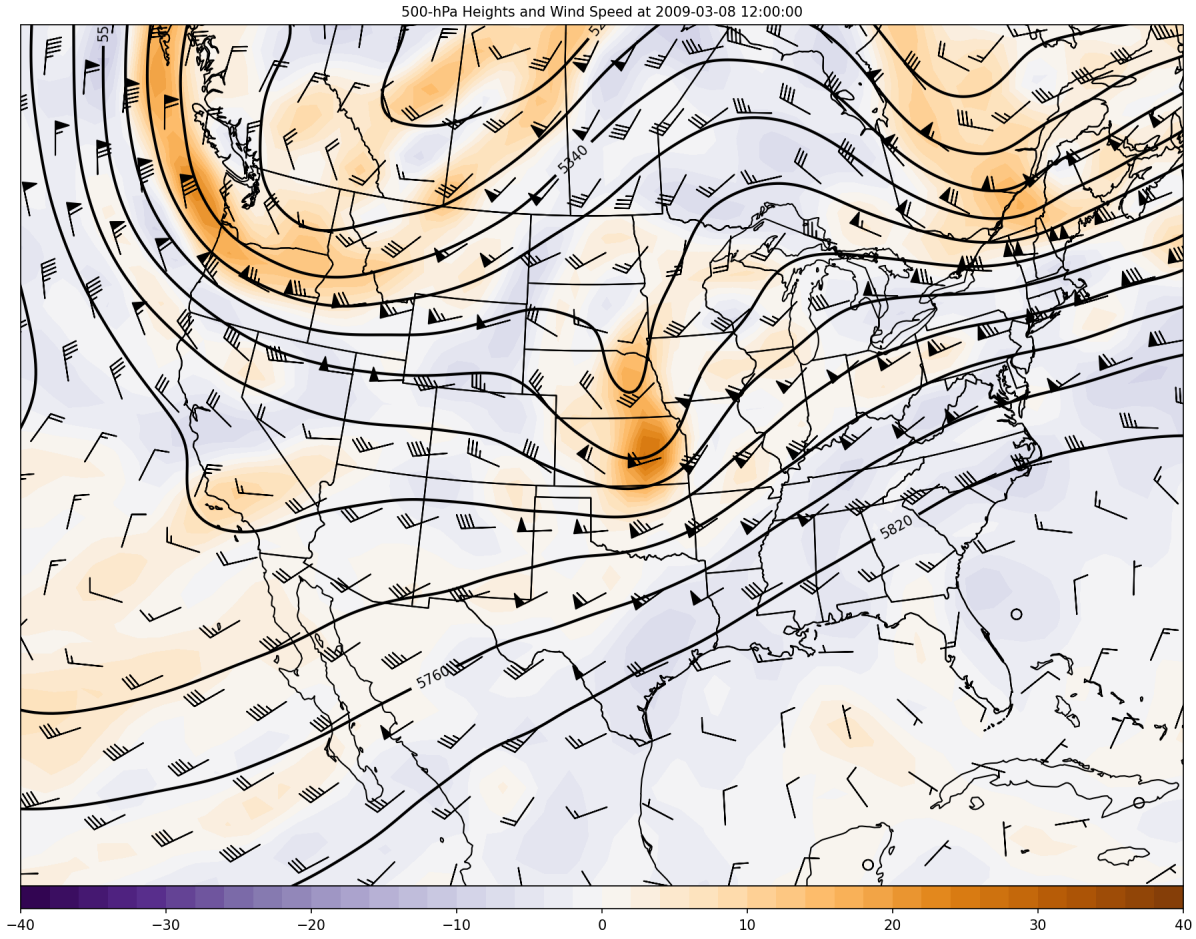
Practice Tasks - Derived Parameters

1. Select one (or more) of the following websites (or choose your own) and navigate through to explore maps on vorticity (typically shown at 500 hPa) and temperature advection (at 850 hPa). Explore patterns relative to features such as upper-level troughs and mid-latitude cyclones. What relationships do you see in real-time forecast models?

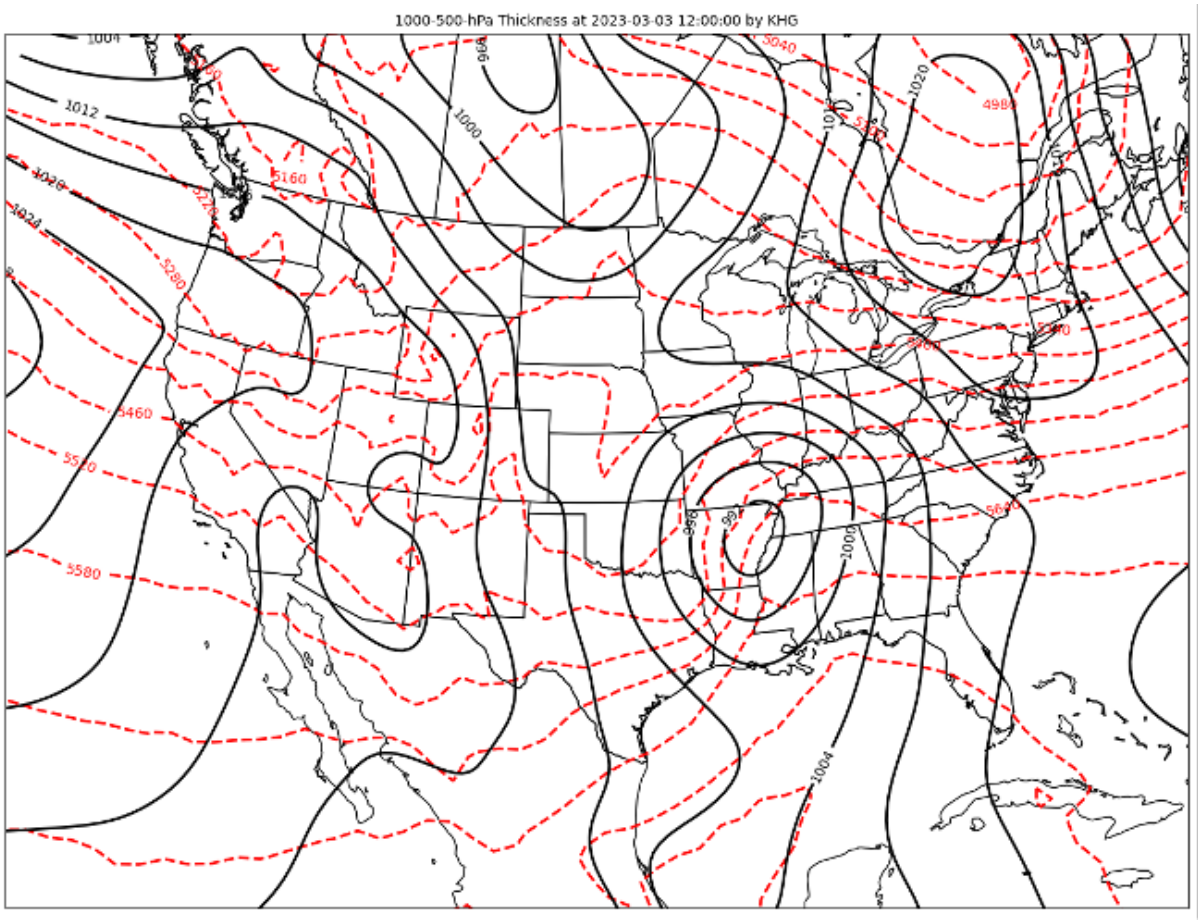
   a. https://weather.cod.edu/forecast/

   b. https://www.pivotalweather.com/model.php

2. Use a map of 500 mb vorticity from the most recent NAM forecast model to practice analyzing areas of *cyclonic* vorticity advection (CVA) as well as *anticyclonic* vorticity advection (AVA). Do you seem to find regions of CVA *ahead of* or *in the wake of* upper-level troughs?



3. Consider the following image – where is the 1000-500 hPa thickness high (and where is it low). How does this coincide with the surface low across the Plains? Where might you expect to find an upper-level trough (and ridge)?


1000-500-hPa Thickness at 2023-03-03 12:00:00 by KHG
1004
1012
1028
1028
5160
5280
5460
5520
5580
996
1000
5040
1020
1020
4980
5640
960
992
1008
1004

0.3.13 Calculations with MetPy

As mentioned in the previous chapter, there may be times when you will want to plot an atmospheric variable (e.g., thickness, temperature advection, relative vorticity, divergence) that is not in the model output file. For that we will have to learn (or remember) a little math and how we can calculate new variables in using MetPy calculations.

Armed with a better understanding of why we care about thickness, the general role of vorticity, and how advection is a fancy term for horizontal transport (of a property) variables can be easily calculated through either simple mathematical operations or from functions built into the MetPy module. Certain functions will only be able to be used on model output or other ***gridded*** data, and *not* actual surface or upper-air observations. Others will work for point observations only. Here we focus on the calculation of, and the methods to display thickness, vorticity, and temp advection.

Some calculations will be simple (e.g., thickness) where you will only have to set up a mathematical difference. Other times there will be more involved calculations that need to be done, which are available as functions in MetPy. The fact that all MetPy gridded calculations produce DataArrays should make them straightforward to use, you'll just need to know what data to feed into the function. That is where looking and reading the documentation for a given function will be important.

Available MetPy Calculation Functions

Simple Calculations

The simple calculations cab be done with the simple mathematical symbols and then add that calculation to a dataset.

```
ds['variable_name'] = <mathematical calculation>
```

A new variable can be added to a dataset (ds) by defining the name within quote marks on the left-hand side of the equal sign in the same way as we did with a Pandas DataFrame earlier in the semester. Then the mathematical calculation can be completed on the right-hand side using at least one variable already in the dataset.

THICKNESS

To understand the function and how it works, let's calculate the geopotential thickness between 500 and 1000-hPa. Recall from previous material:

$$\text{1000-500-hPa Thickness} = z_{500} - z_{1000} \tag{22}$$

So, to compute thickness in Python is a straightforward geopotential height (z) difference between two pressure levels, the most common thickness is the 1000-500-hPa Geopotential Height Thickness. This parameter can be calculated using the following code:

```
hght_500 = ds.Geopotential_height_isobaric.metpy.sel(time=plot_time,
    vertical=500 * units.hPa).metpy.quantify()

hght_1000 = ds.Geopotential_height_isobaric.metpy.sel(time=plot_time,
    vertical=1000 * units.hPa).metpy.quantify()

ds['thickness'] = (hght_500 - hght_1000).metpy.dequantify()
```

This code would add the 1000-500-hPa thickness to the dataset and be able to be readily plotted. The first two lines extract the 500 and 1000-hPa Geopotential heights from our Dataset and quantify them (make them unit arrays). The third line computes the thickness from the variables that we just created, then we dequantify the variable to make it back into a DataArray style for use in our Dataset.

Thickness Calculation

This notebook demonstrates a simple calculation for 1000-500-hPa thickness by adding it to the DataArray.

Import Packages

```
from datetime import datetime

from metpy.plots import declarative
from metpy.units import units
import numpy as np
import xarray as xr
```

Get Data

```
# Set the date/time of the model run
date = datetime(2017, 2, 24, 12)

# Remote access to the dataset from the NCEI Archive
ds = xr.open_dataset('https://tds.gdex.ucar.edu/thredds/dodsC/files/g/d084001/'
                     f'{date:%Y}/{date:%Y%m%d}/gfs.0p25.{date:%Y%m%d%H}.f000.grib2')

# Subset data to be just over the U.S. for plotting purposes
ds = ds.sel(lat=slice(70,10), lon=slice(360 -140, 360 -60))
```

A Simple Calculation 1000-500-hPa Thickness = (Heights at 500 hPa) - (Heights at 1000 hPa)

Easy to do subtraction with a data array and store it directly back into the dataset (ds). In order to do this well, we'll need to quantify the variable (e.g., attach units to the variable we're pulling from our Dataset. Then we'll do the calculation and dequantify it so we are back in a more native xarray DataArray format.

```
hght_500 = ds.Geopotential_height_isobaric.metpy.sel(time=date,
    vertical=500 * units.hPa).metpy.quantify()
hght_1000 = ds.Geopotential_height_isobaric.metpy.sel(time=date,
    vertical=1000 * units.hPa).metpy.quantify()
ds['thickness'] = (hght_500 - hght_1000).metpy.dequantify()
```

> **Note**
> Since we subset by time, as well as by vertical level, when isolating the 500 and 1000-hPa geopotential heights above, we no longer need to subst by those dimensions when plotting. So we'll need to set the *level* and *time* attributes to `None`.

Plot Data

```
# Set attributes for plotting contours
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'thickness'
cntr.level = None
cntr.time = None
cntr.contours = list(range(0, 10000, 60))
cntr.linecolor = 'red'
cntr.linestyle = 'dashed'
```

```
cntr.clabels = True

cntr2 = declarative.ContourPlot()
cntr2.data = ds
cntr2.field = 'Geopotential_height_isobaric'
cntr2.level = 500 * units.hPa
cntr2.time = date
cntr2.contours = range(0, 10000, 60)
cntr2.linecolor = 'black'
cntr2.linestyle = 'solid'
cntr2.clabels = True

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 20, 55]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.left_title = f'1000 -500 -hPa Thickness at {date}'
panel.right_title = f'Valid: {date:%Y -%d -%m %H:%M} UTC'
panel.plots = [cntr, cntr2]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Scaling Parameters

The current GFS output includes the absolute vorticity parameter at a limited number of pressure levels (Absolute_vorticity_isobaric), but typically we plot it at only 500 hPa. However, the values of vorticity are very small, on the order of 10^{-5} s^{-1}. For plotting purposes, we would rather have our data able to plot using whole values (e.g., 1, 2, 3, 4, 5, etc.), so we'll need to scale the values from the model, which is a simple multiplication problem.

To scale the absolute vorticity appropriately we would need to multiply by 1e5 (which is equivalent to $1 * 10^5$). You can use the scale parameter to accomplish this task.

Using the scale parameter will allow you to plot common values of absolute vorticity using a range of -10 to 45.

Scaling Variables

This notebook demonstrates the use of the `scale` parameter to plot the Absolute Vorticity.

Import Packages

```
from datetime import datetime, timedelta

import metpy.calc as mpcalc
from metpy.plots import declarative
from metpy.units import units
import numpy as np
import xarray as xr
```

Get Data

```
# Set the date/time of the model run
date = datetime(2017, 2, 24, 12)

# Remote access to the dataset from the NCEI Archive
ds = xr.open_dataset('https://tds.gdex.ucar.edu/thredds/dodsC/files/g/d084001/'
                     f'{date:%Y}/{date:%Y%m%d}/gfs.0p25.{date:%Y%m%d%H}.f000.grib2')

# Subset data to be just over the U.S. for plotting purposes
ds = ds.sel(lat=slice(70,10), lon=slice(360 -140, 360 -60))
```

Scaling Data Since vorticity values are on the order of 10^{-5} we want to scale them to fit common contouring values of whole numbers, so we want to set the `scale` attribute to be `1e5`.

```
# Set the plot time with forecast hours
plot_time = date + timedelta(hours=0)

# Set attributes for plotting contours
cfill = declarative.FilledContourPlot()
cfill.data = ds
cfill.field = 'Absolute_vorticity_isobaric'
cfill.level = 500 * units.hPa
cfill.time = date
cfill.contours = list(range( -46, 47, 2))
cfill.colormap = 'PuOr_r'
cfill.colorbar = 'horizontal'
cfill.scale = 1e5
cfill.smooth_field = 3

cntr2 = declarative.ContourPlot()
cntr2.data = ds
cntr2.field = 'Geopotential_height_isobaric'
cntr2.level = 500 * units.hPa
cntr2.time = date
cntr2.contours = list(range(0, 10000, 60))
cntr2.linecolor = 'black'
cntr2.linestyle = 'solid'
cntr2.clabels = True
cntr2.smooth_field = 3
```

```
# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 22, 52]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'{cfill.level.m} -hPa Abs. Vorticity Valid at {date}'
panel.plots = [cfill, cntr2]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Practice Tasks - Simple Calculations

1. Create a model representation of a three-hour change in surface (2m) temperature.
2. Create a map with mean sea-level pressure (MSLP) and 1000-500 hPa thickness. Contour MSLP in black and 1000-500 hPa in dashed red.
3. Plot a 500-hPa map for today and plot geopotential height and absolute vorticity from the GFS output.

Wind Speed

A common variable to plot on a 300-hPa or 250-hPa map would be the wind speed. Due to the fact that that wind is a vector quantity and the calculation of wind speed is finding the magnitude of this vector. MetPy has this calculation available (called wind_speed) and the DataArray that comes out of the calculation can be made into a new variable in our Dataset.

Here is code that would accomplish calculating the wind speed from the u- and v-components of the wind and immediately adding new variable called wind_speed to the dataset. First the variables are pulled from the Dataset (and subset if necessary), then used in the function call to calculate wind speed.

```
# Calculate Wind Speed

uwnd = ds['u -component_of_wind_isobaric']

vwnd = ds['v -component_of_wind_isobaric']

ds['wind_speed'] = mpcalc.wind_speed(uwnd, vwnd)
```

MetPy `wind_speed` Documentation

Wind Speed Calculation

This notebook demonstrates using a MetPy calculation function to compute a wind speed, save it as a new varaible, and add it to the Dataset for plotting purposes.

Import Packages

```
from datetime import datetime, timedelta, time, UTC

import metpy.calc as mpcalc
from metpy.plots import declarative
from metpy.units import units
import xarray as xr
```

Get Data

```
# Set the date/time of the model run for yesterday at 00 UTC
yesterday = datetime.now(UTC) - timedelta(days=1)
date = datetime.combine(yesterday, time(0))

# Remote access to the dataset from the UCAR site
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib/NCEP/GFS/'
                     f'Global_0p5deg_ana/GFS_Global_0p5deg_ana_{date:%Y%m%d_%H}00.grib2')

# Set the plot time
plot_time = date + timedelta(hours=0)

# Subset data to be just over the U.S. for plotting purposes and for the plot_time
ds = ds.metpy.sel(lat=slice(70,10), lon=slice(360 -140, 360 -60))
```

Perform Calculation

1. Do the calculation using MetPy (not too hard).

2. Add it to the dataarray.

All MetPy Calculations can be found at https://unidata.github.io/MetPy/latest/api/generated/metpy.calc.htm
Not all calculations work on grids, yet!
Wind Speed Calculation in MetPy
We can add a new variable based on a simialr one, then update the values based on the calculation.
This is easy for wind speed as it is a variant on a single component of the wind.

```
# Isolate needed variables for calculations
uwnd = ds['u -component_of_wind_isobaric'].metpy.sel(time=plot_time)
vwnd = ds['v -component_of_wind_isobaric'].metpy.sel(time=plot_time)

# Calculate Wind Speed, store in Dataset
ds['wind_speed'] = mpcalc.wind_speed(uwnd, vwnd)
```

Plot Data

```
# Set attributes for plotting contours
cfill = declarative.FilledContourPlot()
cfill.data = ds
cfill.field = 'wind_speed'
cfill.level = 300 * units.hPa
```

```
cfill.time = None # We subset for time already
cfill.contours = list(range(10, 201, 20))
cfill.colormap = 'BuPu'
cfill.colorbar = 'horizontal'
cfill.plot_units = 'knot'

cntr2 = declarative.ContourPlot()
cntr2.data = ds
cntr2.field = 'Geopotential_height_isobaric'
cntr2.level = 300 * units.hPa
cntr2.time = plot_time
cntr2.contours = list(range(0, 10000, 120))
cntr2.linecolor = 'black'
cntr2.linestyle = 'solid'
cntr2.clabels = True
cntr2.smooth_field = 4

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 22, 52]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'{cfill.level.m} -hPa Heights and Wind Speed at {plot_time}'
panel.plots = [cfill, cntr2]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Advection

There are a number of different variables that could be advected in the atmosphere including temperature, moisture, vorticity, smoke, etc. Here we'll demonstrate advection using temperature since it is a type of advection that is looked at very frequently and can have a large impact on what we experience as the weather changes. To begin the calculation, first isolate all of the variables you'll need (e.g., temperature, u-component of the wind, and v-component of the wind) for the level and time you are computing. Then feed those variables into the calculation.

```
level = 850 * units.hPa
plot_time = date + timedelta(hours=0)

tmpk = ds.Temperature_isobaric.metpy.sel(vertical=level, time=plot_time)
uwind = ds['u -component_of_wind_isobaric'].metpy.sel(vertical=level, time=plot_time)
vwind = ds['v -component_of_wind_isobaric'].metpy.sel(vertical=level, time=plot_time)

ds['temperature_advection'] = mpcalc.advection(tmpk, uwind, vwind)
```

Advection Calculation Documentation

Advection can be scaled a couple of different ways, but generally we want to put it in terms of temperature per 3 hours. So, scaling by 10$\hat{4}\hat{w}$ould approximately get you there.

> **Note**
> If you subset by time, set the time attribute to None.

Advection Calculation

This notebook demonstrates computing a temperature advection, then plotting it using color-filled contours.

Import Needed Packages

```
from datetime import datetime, timedelta, time, UTC

import metpy.calc as mpcalc
from metpy.plots import declarative
from metpy.units import units
import xarray as xr
```

Get Data

```
# Set the date/time of the model run
yesterday = datetime.now(UTC) - timedelta(days=1)
model_run_time = datetime.combine(yesterday, time(0))

# Remote access to the dataset from the UCAR site
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib'
                     f'/NCEP/GFS/Global_onedeg/GFS_Global_onedeg_'
                     f'{model_run_time:%Y%m%d}_{model_run_time:%H%M}.grib2')

# Subset data to be just over the U.S. for plotting purposes
ds = ds.sel(lat=slice(70,10), lon=slice(360 -150, 360 -55))
```

Calculation

1. Pull out necessary variables from dataset.
2. Do the calculation using MetPy (not too hard).
3. Add it to the dataarray.

All MetPy Calculations can be found at https://unidata.github.io/MetPy/latest/api/generated/metpy.calc.htm

> **Note**
> Not all calculations work on grids, yet!

$$\text{Advection} = -u\frac{\Delta T}{\Delta x} - v\frac{\Delta T}{\Delta y} \tag{23}$$

The minus signs are by convention, which means cold air advection will be negative values and warm air advection will be positive values.

Advection Calculation in MetPy

```
# Select Level and Time
level = 850 * units.hPa
plot_time = model_run_time + timedelta(hours=0)

# Isolate needed variables
tmpk = ds.Temperature_isobaric.metpy.sel(vertical=level, time=plot_time)
```

```
uwind = ds['u -component_of_wind_isobaric'].metpy.sel(vertical=level, time=plot_time)
vwind = ds['v -component_of_wind_isobaric'].metpy.sel(vertical=level, time=plot_time)

# Copmute advection and store in Dataset
ds['temperature_advection'] = mpcalc.advection(tmpk, uwind, vwind)
```

Plot Temperatue Advection Here we plot temperature advection and we want to scale our advection to be in Celsius per 3 hours. We need to change two attributes to get there, first `plot_units` we can set to `degC/hour`, then we set the `scale` attribute to `3` to get our value over a three hour period.

We'll also use the `smooth_contour` attribute to smooth the Geopotential Heights and Temperature variables.

```
# Set attributes for plotting contours
cfill = declarative.FilledContourPlot()
cfill.data = ds
cfill.field = 'temperature_advection'
cfill.level = None # Since already chose level
cfill.time = None # Since already chose time
cfill.contours = list(range( -15, 16, 1))
cfill.colormap = 'coolwarm'
cfill.colorbar = 'horizontal'
cfill.scale = 3
cfill.plot_units = 'degC/hour'

cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'Temperature_isobaric'
cntr.level = level
cntr.time = plot_time
cntr.contours = list(range( -40, 41, 5))
cntr.linecolor = 'red'
cntr.linestyle = 'dashed'
cntr.clabels = True
cntr.plot_units = 'degC'
cntr.smooth_contour = 4

cntr2 = declarative.ContourPlot()
cntr2.data = ds
cntr2.field = 'Geopotential_height_isobaric'
cntr2.level = level
cntr2.time = plot_time
cntr2.contours = list(range(0, 10000, 30))
cntr2.linecolor = 'black'
cntr2.linestyle = 'solid'
cntr2.clabels = True
cntr2.smooth_field = 2
cntr2.smooth_contour = 4

barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = plot_time
barbs.field = ['u -component_of_wind_isobaric', 'v -component_of_wind_isobaric']
barbs.level = level
```

```
barbs.skip = (3, 3)
barbs.plot_units = 'knot'

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 20, 55]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'850 -hPa Temperature Advection at {plot_time} by KHG'
panel.plots = [cfill, cntr, cntr2, barbs]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Relative Vorticity

A similar thing would need to be done for calculating the relative vorticity, where there are multiple pieces of data that must be gathered from the gridded file and used in the final calculation.

```
level = 500 * units.hPa
plot_time = model_run_time + timedelta(hours=0)

uwnd = ds['u -component_of_wind_isobaric'].metpy.sel(vertical=level, time=plot_time)
vwnd = ds['v -component_of_wind_isobaric'].metpy.sel(vertical=level, time=plot_time)

ds['relative_vorticity'] = mpcalc.vorticity(uwnd, vwnd)
```

Relative Vorticity Calculation

Relative vorticity should be scaled in the same manner as the absolute vorticity, so using the scale parameter to multiply by `1e5` would accomplish that goal.

Relative Vorticity Calculation

This notebook demonstrates computing relative vorticity, then plotting it using color-filled contours.

Import Packages

```
from datetime import datetime, timedelta, time, UTC

import metpy.calc as mpcalc
from metpy.plots import declarative
from metpy.units import units
import xarray as xr
```

Get Data

```
# Set the date/time of the model run
yesterday = datetime.now(UTC) - timedelta(days=1)
model_run_time = datetime.combine(yesterday, time(0))

# Remote access to the dataset from the UCAR site
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib'
                     f'/NCEP/GFS/Global_onedeg/GFS_Global_onedeg_'
                     f'{model_run_time:%Y%m%d}_{model_run_time:%H%M}.grib2')

# Subset data to be just over the U.S. for plotting purposes
ds = ds.sel(lat=slice(70,10), lon=slice(360 -150, 360 -55))
```

Calculation

1. Pull out necessary variables from dataset.
2. Do the calculation using MetPy (not too hard).
3. Add it to the dataarray.

All MetPy Calculations can be found at https://unidata.github.io/MetPy/latest/api/generated/metpy.calc.htm

> **Note**
> Not all calculations work on grids, yet!

$$\text{Relative Vorticity} = \zeta = \frac{\Delta V}{\Delta x} - \frac{\Delta U}{\Delta y} \tag{24}$$

$$\tag{25}$$

MetPy Relative Vorticity Calculation

```
# Select Level and Time
level = 500 * units.hPa
plot_time = model_run_time + timedelta(hours=0)

# Isolate needed variables
uwnd = ds['u -component_of_wind_isobaric'].metpy.sel(vertical=level, time=plot_time)
vwnd = ds['v -component_of_wind_isobaric'].metpy.sel(vertical=level, time=plot_time)

# Compute relative vorticity and store in Dataset
ds['relative_vorticity'] = mpcalc.vorticity(uwnd, vwnd)
```

Plot Relative Vorticity In addition to plotting our newly calculated relative vorticity variable, we'll smooth the Geopotential Hieghts using the `smooth_field` attribute.

```
# Set attributes for plotting contours
cfill = declarative.FilledContourPlot()
cfill.data = ds
cfill.field = 'relative_vorticity'
cfill.level = None
cfill.time = None
cfill.contours = list(range( -40, 41, 2))
cfill.colormap = 'PuOr_r'
cfill.colorbar = 'horizontal'
cfill.scale = 1e5

cntr2 = declarative.ContourPlot()
cntr2.data = ds
cntr2.field = 'Geopotential_height_isobaric'
cntr2.level = 500 * units.hPa
cntr2.time = plot_time
cntr2.contours = list(range(0, 10000, 60))
cntr2.linecolor = 'black'
cntr2.linestyle = 'solid'
cntr2.clabels = True
cntr2.smooth_field = 2

barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = plot_time
barbs.field = ['u -component_of_wind_isobaric', 'v -component_of_wind_isobaric']
barbs.level = level
barbs.skip = (3, 3)
barbs.plot_units = 'knot'

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = [ -125, -74, 20, 55]
panel.projection = 'lcc'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'{cntr2.level.m} -hPa Heights and Wind Speed Valid at {plot_time}'
panel.plots = [cfill, cntr2, barbs]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

Practice Tasks - MetPy Calculations

1. Compute the potential temperature on the 850-hPa surface from GFS output, add the dewpoint values to the dataset, and plot over the CONUS.

2. Compute the dewpoint temperature for 850-hPa from GFS output using the temperature and relative humidity, add the dewpoint values to the dataset, and plot over the CONUS.

3. Compute the heat index of the 2-m temperature output from the GFS (you'll need both temperature and relative humidity at 2 m), add it to the dataset, and plot over a local region of your choosing.

0.3.14 Skew-T Log-p Diagram

Taking a deviation away from plotting data on two-dimensional maps, here we look to plot vertical profile data to understand aspects of the atmosphere that are important to stability, precipitation, and precipitation type. There are a number of different thermodynamic diagrams that are used in the sciences and engineering. In meteorology, these diagrams are used to plot a variety of different atmospheric parameters, all in a standard, easily accessible format and are used extensively in forecasting. In a number of ways they can be used to assess the vertical structure and stability of the atmosphere. The vertical profiles of temperature, dew-point temperature, and wind speed and direction can explain a lot about the nature of atmosphere and what types of weather may form in such an environment.

The data displayed in these diagrams are collected by radiosondes, which are instrumented platform with meteorological sensors and a radio transmitter that are carried aloft by a balloon. Radiosondes are released at selected NWS sites across the country at 0000 and 1200 UTC every day, as well as from sites around the world. Sometimes special radiosondes are released at 1800 UTC (or other times) on days when the risk of severe weather is especially high.

Stability is an important concept for understanding many atmospheric processes, but how can we assess the stability? The vertical stability of the atmosphere is determined by the vertical distribution of temperature, pressure, and wind speed and direction. These measurements are obtained by launching radiosondes with a big balloon. In this lab we will first review stability terminology and then discuss how we can plot our variables in a helpful fashion.

Atmospheric Stability

The stability of the atmosphere can be described using the three different conditions of equilibrium: stable, neutral, or unstable (Fig. 1). When a ball is displaced up either side of a valley, gravity will work to bring the ball back to its starting point. This would be described as a stable system. If the ball was on top of a ridge, any displacement would cause the ball would move quickly away from its initial point - this is defined as an unstable system. Finally, when a ball is displaced and does not act to return to its original starting position or move farther away from the original starting point, the stability would be defined as neutral.

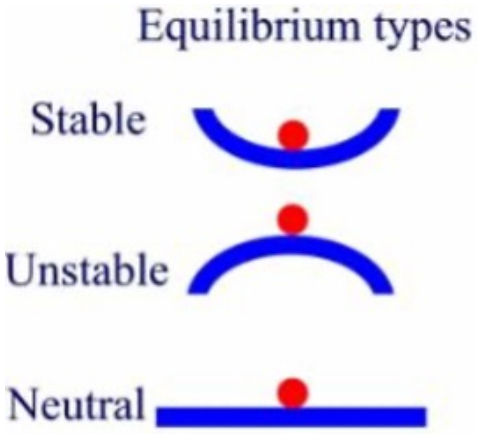


Figure 31: Different conditions of equilibrium related to stability. Image from the Equilibrium, Stability, and Behavior over time and is licensed under the "Creative Commons Attribution-Share Alike 4.0 International". Image source https://serc.carleton.edu/images/introgeo/models/Stability.jpg.

These conditions can also be applied to the atmosphere. For parcels initially displaced upwards, those wanting to rise without any forcing would be *unstable*; those wanting to sink would be *stable*; those that would need more forcing in order to rise freely would be *conditionally unstable*; all others would be *neutral*. We will now apply these equilibrium types to the atmosphere and use parcel theory to determine the stability of the atmosphere.

Parcel Theory In meteorology, parcel theory is used to determine and describe the stability of the Earth-atmosphere system. A parcel is a fictitious blob of air that has similar moisture and temperature characteristics throughout the parcel. Parcel sizes vary from the size of a large playground ball to as big as a parking lot. To envision what parcels may look like you could think of the bubbles in boiling water or the blobs of wax that move up and down in lava lamps. The mechanisms in the atmosphere are similar to both of those examples; a parcel of water or wax is heated from below, just as the sun heats the atmosphere from below.

Whether the atmosphere is stable, unstable, or conditionally unstable, rising motion can occur and lead to the development of different types of clouds. The development of clouds is predicated on the fact that there is sufficient moisture to go along with the rising motion. In stable layers with sufficient moisture and rising motion, sheet like clouds will form (e.g., stratus, altostratus). In unstable or conditionally unstable portions of the atmosphere with sufficient moisture puffy clouds (e.g., cumulus, cumulonimbus) often form.

When air parcels rise, they always, and for all time will, *cool*. Depending on the exact atmospheric conditions, the parcels will cool at one of two lapse rates. If the air is not saturated (<100% RH; unsaturated) then rising parcels will cool at the Dry Adiabatic Lapse Rate (DALR) of 9.8 $K\ km^{-1}$. For parcels with 100% RH, if they continue to rise, will cool at the Wet Adiabatic Lapse Rate (WALR), also known as the Moist Adiabatic Lapse Rate (MALR). The WALR is not constant and varies greatly over the temperature and pressure profile; consult a skew-T/log-p diagram to confirm this fact.

WALR vs. DALR The WALR is actually not an adiabatic process. Adiabatic motions are such that there is no net gain or loss of energy; the change in temperature comes through the expansion

(cooling) or compression (warming) of parcels. If you continue to cool air that is already saturated, according to the Clausius-Clayperon equation, less water vapor can be present in cooler air, so it must be converted from water vapor to liquid water or ice. This is one way in which clouds are formed. So air parcels are still rising and cooling at the DALR, but when the water vapor is converted to liquid water, latent energy is released to the air parcel, which will slightly warm the parcel. This results in a slightly lower cooling rate for a saturated parcel.

Skew-T Log-p Description

In the meteorological field, the two most important thermodynamic diagrams are the psuedoadiabatic diagram (also known as a Stüve diagram) and the skew-T/log-P diagram. In this chapter we will focus on the skew-T/log-P diagram. In practice, this diagram is the most widely used thermodynamic diagram in the meteorological discipline.

Similar to surface observations, upper-air observations are encoded into a special format before being transmitted. Historically these radiosonde data were transmitted in three major blocks: TTAA, TTBB, and PPBB. Each of these three blocks has its own significance. TTAA refers to data on the mandatory levels, TTBB refers to data on significant thermodynamic levels, and PPBB refers to significant wind levels. Now they come across in a binary (computer encoded) format. We'll focus on plotting the vertical profile of the atmosphere that we obtain through the use of radiosondes.

In the Skew-T/Log-p diagram, the logarithm of pressure is used as the vertical (y) axis, with larger values at the bottom. The brown horizontal lines are constant pressure surfaces in units of millibars.

The brown solid lines that tilt toward the upper right are isotherms in řC. Equal distances along the x-axis represent equal differences in temperature, increasing from left to right along the axis. However, the isotherms are tilted toward the upper right rather than running perpendicular to the x-axis. This is why the term "skew" is associated with temperature.

The green dashed lines that tilt with a greater slope to the upper-right are lines of constant mixing ratio in grams of water vapor per kilogram of air. Instead of mixing ratio, some thermodynamic diagrams plot other moisture variables, such as specific humidity. However, in practice, no matter what variable is plotted, the overall meaning and interpretation is the same.

The red solid lines that are oriented perpendicular (or almost perpendicular) to the isotherms are isentropic surfaces, or lines of constant potential temperature in řC. The values of the isentropic surfaces are the same as the temperature lines at 1000 mb. These lines are also known as dry adiabats.

The green curved lines are lines of constant equivalent potential temperature in řC, and are known as pseudoadiabats or moist adiabats. There values are also determined from the temperature at 1000 mb.

Plotting on SkewTs Temperature and dew-point temperature are plotted relative to pressure. For a given pressure level, a temperature and dew-point temperature are plotted separately and a line is drawn for both variables that connect the values between reported pressure levels.

The mixing ratio (r) can be obtained at a given level by interpolating which mixing ratio isopleth intersects the point at which the dew point temperature is plotted.

The saturation mixing (r_s) ratio can be obtained at a given level by interpolating which mixing ratio isopleth intersects the point at which the temperature is plotted.

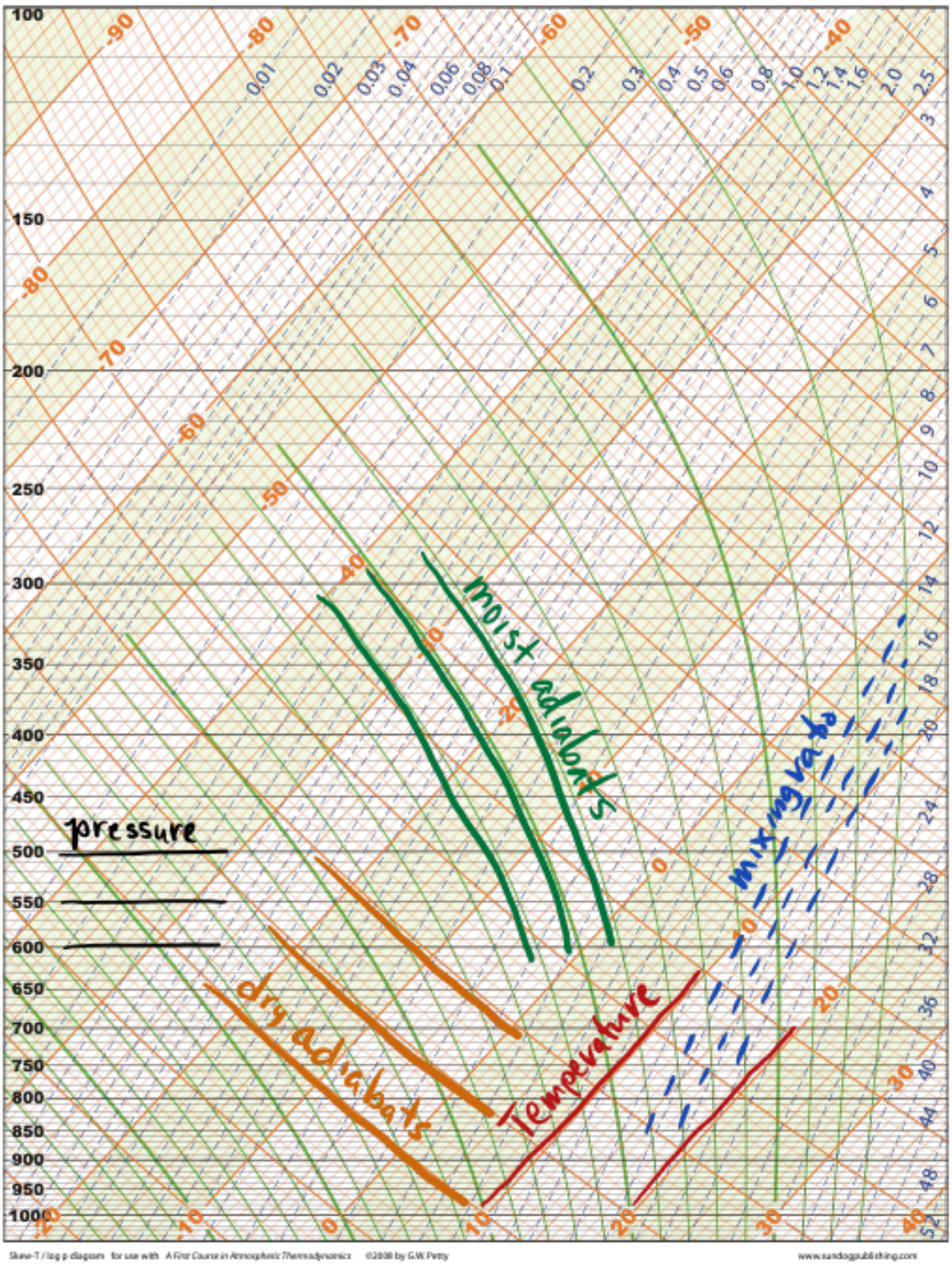


Figure 32: This is an annotated skew-T log-p diagram identifying the important lines that compose the chart.

Sounding attributes

The tropopause is found where there is a distinct change from a positive lapse rate (temperature *decreasing* with height) in the troposphere to a negative lapse rate (temperature *increasing* with height) in the stratosphere. This distinction is not always clear. In general, the height of the tropopause is greater in low latitudes than it is in high latitudes. This is a result of the hypsometric equation, which states mathematically that the thickness of a layer of atmosphere is a function of the average virtual temperature of that layer.

There sometimes are layers below the tropopause where temperatures increase with height in a shallow layer; these are called a temperature "inversion" or simply "inversion". These inversions can be caused by frontal passages, overnight cooling at the surface, or the result of sinking air (*subsidence*). Often accompanying the increase in temperature with height is usually a sharp decrease in dew-point temperature with height, when this happens the inversion is a result of subsidence (sinking), which is a drying process.

The vertical profile of wind speed and direction can be important in atmospheric stability assessment. Winds that are changing direction counterclockwise with height are known as *backing winds*, while those that are changing direction clockwise with height are known as *veering winds*. Backing winds are associated with cold air advection, which tends to promote sinking motion. While, veering winds are associated with warm air advection, which tends to promote rising motion.

Mandatory and Significant Levels The data collected by the radiosonde is encoded for mandatory and significant levels only, even though the instruments are continuously measuring data during the radiosonde flight.

The mandatory levels are: 1000, 925, 850, 700, 500, 400, 300, 250, 200, 150, 100, 70, 50, and 10 (all in mb)

These do not change. Data are always reported from these pressure levels.

Significant levels are altitudes where sensors detect a rapid change in the vertical profile of temperature, moisture, and/or wind speed or direction. The additional handout today is a sampling of decoded data from our recent balloon launch

Practice Tasks - Skew-T Diagrams

1. Why is the first mandatory level (1000 hPa) blank?
2. Plot the sounding on the provided skew-t plot. (This will be part of the official assignment given out at the next class session.)
3. Describe prominent features of the vertical profile of the atmosphere that you observe on your plotted sounding.

SkewTs with Metpy

While plotting a Skew-T plot by hand is useful, it is faster to be able to let a computer plot the data for you. However, there are many different variables that you can specify so that it plots exactly what you want. We can use elements of the MetPy module to create thermodynamic diagrams. Additionally, we can use some simple techniques to further define the stability of the atmosphere by drawing on our skew-t plot.

There is not a declarative syntax for plotting skew-Ts via MetPy, yet. So, the coding that we'll be doing in this section will be slightly different from what has been covered here so far, but hopefully not too much so.

Our ability to analyze the vertical profile of the atmosphere is a critical tool for meteorologists and in Python we can access a single profile from a sounding via Siphon and our remote data access protocols. In this instance we are only going to grab data from one station, instead of all of them as we have been doing over the course of this semester.

> **Note**
> It is important to remember that upper air data are only collected at synoptic times, that is, 0000 and 1200 UTC. Only on special occasions are the sounding data at other times. This usually occurs on the plains during severe weather season in which a sounding may be launched between 18 and 21 UTC. Soundings are only launched at certain places, or stations.

What stations? You can visit http://www.rap.ucar.edu/weather/upper/ to locate the current available station IDs across the U.S. If you are looking for older datasets, visiting the Wyoming Sounding archive (http://weather.uwyo.edu/upperair/sounding.html) webpage will allow you to determine which sounding sites are available during whatever time period you specify.

Visulalizing and Calculating Parameters on a Skew-T

A powerful tool is being developed that will have the same functionality as GEMPAK, but be done in the Python programming langauage. One example of this is the ability to plot Skew-Ts using the Python module MetPy(). However, there is greatly functionality than that contained in GEMPAK, as you can use MetPy to calculate a whole host of variables from the data plotted on the Skew-T (e.g., CAPE, CIN, LCL, LFC, EL). This notebook is a brief introduction to how that can be done.

To plot a skew-T, you need to specify the date and time (in UTC) and choose an upper-air station location. The locations of current sounding sites can be found at http://weather.rap.ucar.edu/upper

Import Packages Here we have a slightly different set of imports to be able to generate a skewT in Python. Here we bring in the `SkewT` class, a unit helper function and Siphon for reading data from the Wyoming archive.

```
from datetime import datetime

import matplotlib.pyplot as plt
from metpy.plots import SkewT
from metpy.units import units, pandas_dataframe_to_unit_arrays
import numpy as np
from siphon.simplewebservice.wyoming import WyomingUpperAir
```

Get Data Here we will obtain data for a single time for a single station from the Wyoming sounding archive.

https://weather.uwyo.edu/upperair/sounding.html

This dataset is prefered over the Iowa State archive due to how the data come in more ideal for plotting a profile.

```
# Set date that you want
# Data goes back to the 1970's
date = datetime(2007, 5, 4, 12)

# Set station ID, there are different stations back in the day
# Current station IDs found at http://weather.rap.ucar.edu/upper
station = 'DDC'

# Use Siphon module to grab data from remote server
df = WyomingUpperAir.request_data(date, station)
```

The data coming from Siphon will be in a Pandas dataframe format, but to use this data with MetPy calculations, we'll want them as unit arrays. Luckily for us, there is a simple function that we can feed our data object into to get back a dictionary of unit arrays. Below we do that and then subsequently pull out our needed data into separate variables.

```
# Create dictionary of unit arrays
data = pandas_dataframe_to_unit_arrays(df)

# Isolate united arrays from dictionary to individual variables
p = data['pressure']
T = data['temperature']
Td = data['dewpoint']
u = data['u_wind']
v = data['v_wind']
```

Now we are ready to use the data for calculation and/or plotting purposes. To begin, we'll just use the data to make a simple skew-T/log-p diagram plot.

Plot Skew-T Our plotting is a little different from the syntax we have been using to this point; however, what I hope is that it is pretty simple to understand. First, we create out figure instance, second set up our plot axis (`skew`), and then we plot on that axis (`skew.plot` and `skew.plot_barbs`).

MetPy Skew-T Documentation

```
# Plot a skew -T image
fig = plt.figure(figsize=(10, 10))

# Set up the skewT axes
skew = SkewT(fig, rotation=45)

# Plot the data using normal plotting functions, in this case using
# log scaling in Y, as dictated by the typical meteorological plot
skew.plot(p, T, 'r')
skew.plot(p, Td, 'g')

# Plot wind barbs, skipping every other one
skew.plot_barbs(p[::2], u[::2], v[::2], y_clip_radius=0.03)

# Set sensible axis limits
skew.ax.set_ylim(1000, 100)
skew.ax.set_xlim( -40, 50)

# Add the relevant special lines
skew.plot_dry_adiabats(t0=np.arange(233,555,10)*units.K, alpha=0.25)
skew.plot_moist_adiabats(colors='tab:green', alpha=0.25)
skew.plot_mixing_lines(colors='tab:blue', linestyle='dotted')

# Plot some titles
plt.title(f'Station: K{station}', loc='left')
plt.title('Skew -T/Log -p', loc='center')
plt.title(f'Valid: {date} UTC', loc='right')

# Show the plot
plt.show()

# Save the plot by uncommenting the following line
#plt.savefig('skewt_image.png', bbox_inches='tight', dpi=150)
```

This is a bit more of "raw" coding with setting up the figure, axis, and title, but not too much of a stretch from what we were doing before. To save a figure now you use `plt.savefig()`, but you use the same settings as before.

Practice Tasks - Basic MetPy SkewTs

1. Make a sounding from a station near to where you grew up on your tenth birthday.

2. Alter the figure by using different settings for the diagrams' special lines (i.e., dry adiabats, moist adiabats, and mixing lines).

3. Plot the morning and afternoon sounding for a particular station. You'll want the temperature and dewpoint lines to look different depending on what time they are associated with. Note: you'll need two different instances of reading from the archive or using some advanced scripting to loop through the data calls and plotting.

Skew-T Parcel Path

Skew-T diagrams are filled with lots of information; this part will introduce you to a few concepts related to stability that can be analyzed from an atmospheric sounding plotted on a skew-T diagram.

Drawing a Parcel Path To draw a parcel path, you must first determine the lifting condensation level (LCL). In previous courses you did this through assuming a DALR and WALR, now we will use the skew-T/log-p diagram. The LCL is where cloud bases will form and thus the point of initial saturation in the atmospheric column.

Lifting Condensation Level (LCL) Definition: The height at which a parcel of air lifted dry adiabatically would become saturated. The LCL is always found at or below the CCL.

Procedure: Follow the saturation mixing ratio line from the surface dew-point and the dry adiabat from the surface temperature upward until they intersect.

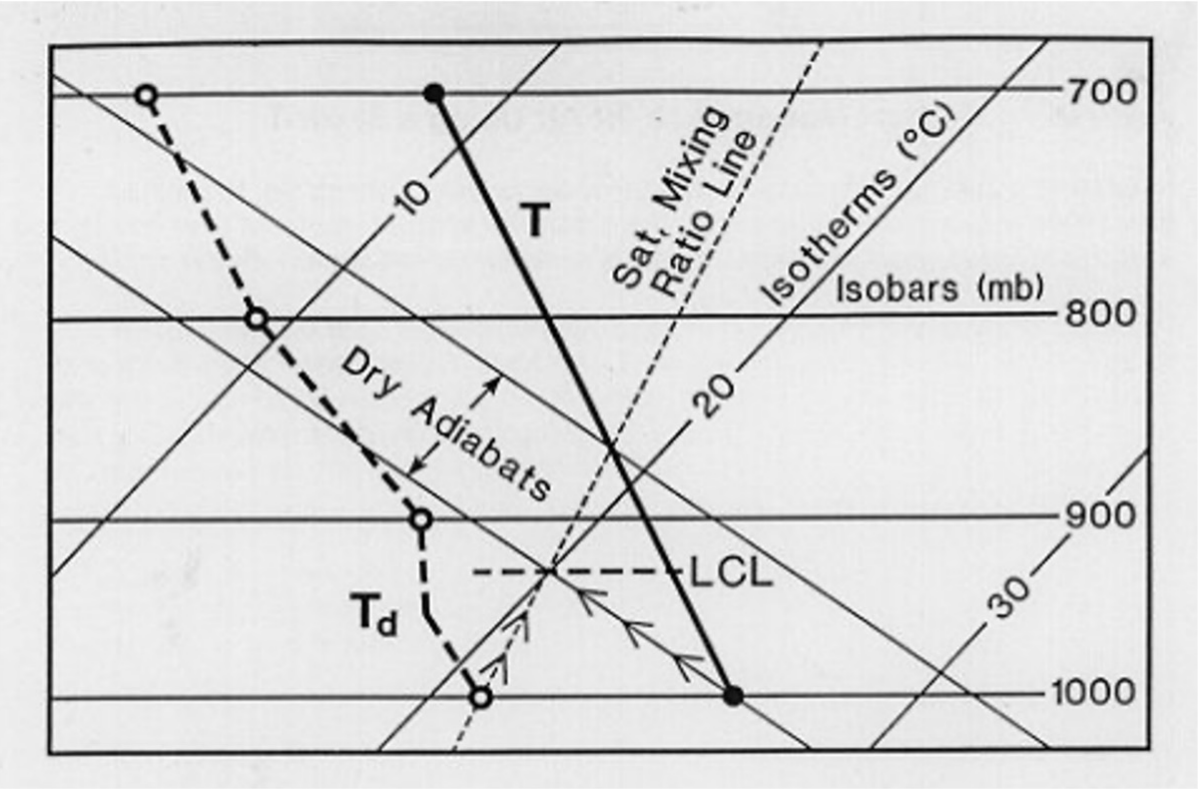


Figure 33: This is a diagram to illustrate the finding of the Lifiting Condensation Level (LCL). Source: *The Use of The Skew T, Log P Diagram in Analysis and Forecasting* AWS/TR-79/006

Once the LCL is determined, then the parcel path can be drawn. Starting the surface parcel at the temperature of the surface and following the dry adiabat line up until you reach the LCL, then lift the parcel via the moist adiabatic line. (If you are not directly on a dry or moist adiabat, then follow parallel to the lines on either side of the parcel path.)

As the result of drawing in the parcel path two other important levels are identified. The Level of Free Convection (LFC) and the Equilibrium Level (EL).

Level of Free Convection (LFC) Definition: The level at which a parcel of air lifted dry adiabatically until saturated (LCL), then moist adiabatically becomes *warmer* than its environment.

Procedure: Find the LCL and then follow the moist adiabat upward until it intersects the temperature sounding. This is the LFC.

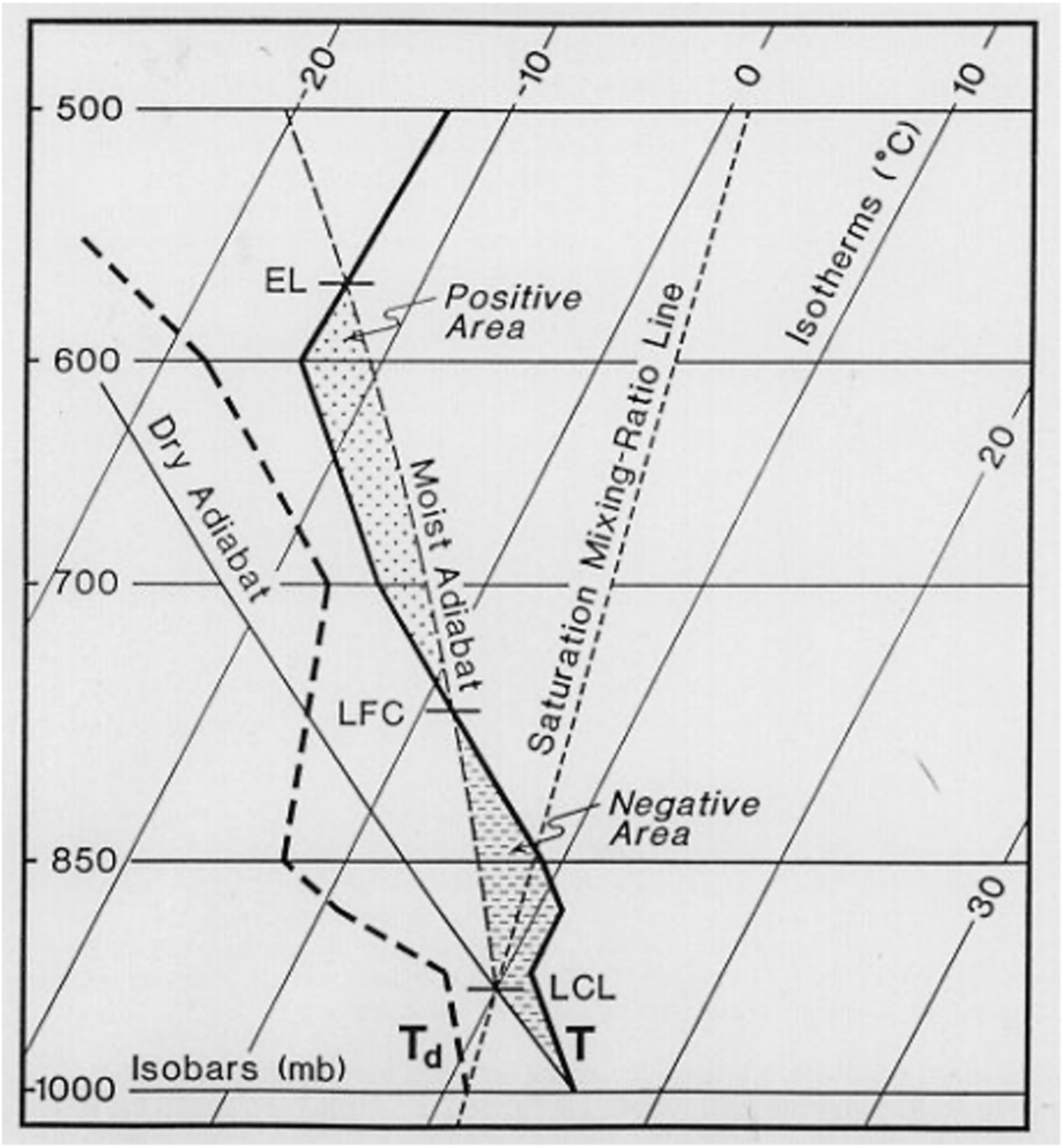


Figure 34: This is a diagram to illustrate of plotting the parcel path using the lifting condentation level (LCL).
Source: *The Use of The Skew T, Log P Diagram in Analysis and Forecasting* AWS/TR-79/006

Note
Be careful when the LFC is below a sharp inversion. Ideally, one should continue to follow the moist adiabat through the inversion to determine the LFC.

Equilibrium Level (EL) Definition: The level at which a parcel of air lifted moist adiabatically from the LFC becomes *cooler* than its environment. This is an estimate of the cloud height of convective type clouds.

Procedure: Find the LFC, then follow the moist adiabat upward until it intersects the temperature sounding. This is the EL.

When the parcel path is to the right of the environmental temperature profile, the parcel is unstable and would want to rise. When it is to the left of the environmental temperature profile the parcel is stable and would want to sink. If the parcel is stable in the lowest layers (i.e., the level below the LFC) then the atmosphere must force parcels to rise in order for the parcel to become unstable and realize any instability that might be present in the Earth-atmosphere system.

Stability Indices

To quickly assess the stability of the atmosphere, many different stability indices have been developed to give a quick, one value snapshot of the vertical stability. Some of the most common measures are: Lifted Index (LI) and Convectively Available Potential Energy (CAPE). We will review the basics of these two parameters, how they are calculated, and what they imply about the stability of the atmosphere.

Lifted Index The Lifted Index (LI) is calculated from the difference between the environmental temperature at 500 hPa and the surface parcel temperature at 500 hPa. Mathematically LI can be described as,

$LI = T_{ENV_{500}} - T_{PARCEL_{500}}$

Typical values for LI and likely weather associated with those values:

LI Value	
>+2	No convective activity
0 – +2	Showers probable, isolated T-storms possible
-2 – 0	Thunderstorms probable
-4 – -2	Severe T-storms possible
<-4	Severe T-storms probable, tornadoes possible

The associated weather is if the atmosphere is otherwise conditioned for precipitation and convective weather. One major drawback of using LI as a forecast tool is that it is only a look at the stability of one level of the atmosphere, which may or may not play an important role in a particular forecast.

CAPE The Convectively Available Potential Energy (CAPE) is defined as the 'positive' area on a skew-t chart between the environmental temperature profile and the parcel path, and furthermore between the LFC and the Equilibrium Level. Mathematically CAPE can be described by,

$$CAPE = R_d \int_{LFC}^{EL} \frac{T(p) - T_a(p)}{p} dp \tag{26}$$

Where EL is the equilibrium level, R_d is the gas constant, LFC is the level of free convection, $T(p)$ is the parcel temperature at pressure level p in Kelvin, and $T_a(p)$ is the environmental temperature at pressure p in Kelvin.

CAPE Value	
<1000 J kg$\hat{\{}$-1}	Weak instability
1000 – 2500	Moderate instability
2500 – 5000	Strong instability
>5000	Extreme instability

Once major advantage of CAPE is that it is an integrated value through the atmosphere considering the changing parcel profile and environmental profile.

Advanced Skew-T Calculations and Plotting

This notebook demonstrates calculating a number of common sounding parameters and plotting some of them on a skew-T diagram.

Import Packages

```
from datetime import datetime

import matplotlib.pyplot as plt
import metpy.calc as mpcalc
from metpy.plots import SkewT
from metpy.units import units, pandas_dataframe_to_unit_arrays
import numpy as np
from siphon.simplewebservice.wyoming import WyomingUpperAir
```

Get Data

```
# Set date that you want
# Data goes back to the 1970's
date = datetime(2007, 5, 4, 18)

# Set station ID, there are different stations back in the day
# Current station IDs found at http://weather.rap.ucar.edu/upper
station = 'DDC'

# Use Siphon module to grab data from remote server
df = WyomingUpperAir.request_data(date, station)
```

Subsetting Data When performing calculations on radiosonde data it is best to subset the data to only use the data needed, which is typically pressures >= 100 hPa. This can be accomplished via code by creating a Boolean (true/false) array from a simple inequality and then using that array to subset all of the variable arrays.

```
# Create dictionary of unit arrays
data = pandas_dataframe_to_unit_arrays(df)

# Isolate united pressure array from dictionary
p = data['pressure']

# Setting a subset to only use the data needed
idx = p.m > 99

# Isolate united arrays and subset from dictionary
p = p[idx]
T = data['temperature'][idx]
Td = data['dewpoint'][idx]
u = data['u_wind'][idx]
v = data['v_wind'][idx]
```

Calculation of Sounding Parameters Once we have our data, we can calculate any of the skew-T variables we desire, such as LCL using the functions available in MetPy (https://unidata.github.io/MetPy/latest/api/generated/metpy.calc.html#soundings).

Lifting Condensation Level This is the level at which a lifted parcel will become saturated (e.g., have 100% RH). The function in MetPy is called lcl and takes inputs of pressure, temperature, and dewpoint.

MetPy LCL Documentation

This function will output two values, the first is the pressure level of the LCL and the second is the temperature of the LCL.

```
# Calculate LCL pressure and temperature
# Note: since lcl function returns two values, we can have
# two variables to the left of the equal sign

lcl_pressure, lcl_temperature = mpcalc.lcl(p[0], T[0], Td[0])
print(f'LCL: {lcl_pressure:.2f}')
```

Parcel Profile The parcel path is essential the temperature that a parcel would have if lifted from a given level. This is an essential component to determining the stability of the atmosphere (a la introduction to meteorology) and determining the available energy for thunderstorm production.

MetPy Parcel Profile Documentation

The input would be all of the pressures from the sounding and the starting temperature and dewpoint values for where the parcel is being lifted from. Typically, this is the zeroth element since we want to lift a surface parcel.

```
# Calculate full surface parcel profile and add to plot as black line
prof = mpcalc.parcel_profile(p, T[0], Td[0]).to('degC')
```

CAPE/CIN Often, we want to know how much energy might be available for use in the generation of thunderstorms, so we want the ability to calculate CAPE. This is actually not a trivial calculation due to any number of details that could be considered, but it is not too hard to do a basic estimation that works well for forecasting purposes. Along with CAPE, we get CIN or convective inhibition, a measure of the excess amount of energy we need to be able to realize any CAPE that might be present in a profile. MetPy has a function that computes both of these values using a trapezoid integration scheme (and you thought you would never hear about estimating an integral outside of calculus!?!).

MetPy CAPE/CIN Documentation

Here our inputs expand to four items, pressure, temperature, dewpoint, and the parcel profile. Each variable must contain the same number of points corresponding to all of the pressure levels given in the dataset.

```
# Calculate the CAPE and CIN of the profile
cape, cin = mpcalc.cape_cin(p, T, Td, prof)

print(f'CAPE: {cape:.2f}')
print(f'CIN: {cin:.2f}')
```

Lifted Index The LI is the difference between the parcel temperature and environmental temperature at 500 hPa. If you have calculated the parcel profile, then this becomes a simple difference of two values once you have found both values at 500 hPa.

MetPy Lifted Index Documentation

Here we need to feed the function the pressure, temperature, and parcel profile temperature.

```
# Compute LI and print result
LI = mpcalc.lifted_index(p, T, prof)
print(f'LI: {LI[0].m:.02f} C')
```

Plot Skew-T with Parcel Path A common thing to plot on a skew-T image is the parcel path, this allows a meteorologist to visually estimate the amount of CAPE and CIN since the area on a skew-T is directly proportional to energy. The key area being between the environmental temperature and the parcel profile.

Plotting the parcel profile is pretty easy, we can use the same skew.plot to plot on the same axis and use our pressure and profile temperature to plot the parcel profile line. In the code below we are plotting the parcel path in black with a thicker line than normal.

The LCL is also highlighted with a straight line at the pressure level at which our parcel becomes saturated.

```
# Plot a skew -T image
fig = plt.figure(figsize=(10, 10))

# Set up the skewT axes
skew = SkewT(fig,  rotation=45)

# Plot the data using normal plotting functions, in this case using
# log scaling in Y, as dictated by the typical meteorological plot
skew.plot(p, T, 'r')
skew.plot(p, Td, 'g')

# Plot barbs skipping to every other barb
skew.plot_barbs(p[::2], u[::2], v[::2], y_clip_radius=0.03)

# Set sensible axis limits
skew.ax.set_ylim(1000, 100)
skew.ax.set_xlim( -40, 50)

# Plot the skew -T parcel temperature profile
skew.plot(p, prof, 'k', linewidth=2)

# Plot a line marker at the LCL level
skew.plot(lcl_pressure, lcl_temperature, '_', color='black', markersize=24,
          markeredgewidth=3, markerfacecolor='black')

# Add the relevant special lines
skew.plot_dry_adiabats(t0=np.arange(233,555,10)*units.K, alpha=0.25)
skew.plot_moist_adiabats(colors='tab:green', alpha=0.25)
skew.plot_mixing_lines(colors='tab:blue', linestyle='dotted')

# Plot some titles
plt.title(f'Station: K{station}', loc='left')
plt.title('Skew -T/Log -p', loc='center')
plt.title(f'{date} UTC', loc='right')

# Show the plot
plt.show()

# Save the plot by uncommenting the following line
#plt.savefig('advanced_skewt_image.png', bbox_inches='tight', dpi=150)
```

Practice Tasks - MetPy Sounding Calculations

1. Plot a sounding with a surface parcel profile for a different day. Note: most sites only have soundings available at 00 and 12 UTC, when radiosondes are regularly launched. However, there are times when special launches are conducted on interesting weather days and you might have an 18 or 21 UTC sounding, but those are rare.

2. Calculate the level of free convection (LFC) and equilibrium level (EL) and plot on a skew-T.

3. Shade CAPE and CIN areas; see the example from MetPy for help https://unidata.github.io/MetPy/latest/examples/Advanced_Sounding.html#sphx-glr-examples-advanced-sounding-py

0.3.15 Satellite Observations

So far we have been building our Python skill set by plotting surface data, upper-air data, model output, and Skew-Ts. We have created new variables to plot and plotted variables already in the data files. Now we have the opportunity two explore some other types of weather data, such as soundings, radar, and satellite-derived variables. In this chapter we'll review some aspects about satellite orbits, types of satellite, and then expand your knowledge about the use of satellite imagery involving the analysis of atmospheric phenomena, such as jet streams. In a separate lab, you'll learn more about creating satellite images using Python.

Types of Satellites

In April of 1960, the meteorological world was turned upside down with the launch of the first successful meteorological satellite. For the first time, meteorologists could get an idea of what the "big picture" looked like in an instant. These first satellites were nothing more than television cameras launched into orbit. Since then satellite technology has increased to the point that a continuous stream of information is sent back to earth 24 hours a day, 7 days a week. With the advancements in technology, we now have different kinds of satellites observing weather conditions, each focusing on different regions of earth.

While there are numerous types of observations that are collected by satellites, the most widely known and used to observe mid-latitude cyclones are visible, infrared, and water vapor imagery. These three types of imagery are discussed next.

Geostationary Earth Orbiting (GEO) One kind of satellite in use is known as a geostationary earth orbiting (GEO) satellite. These satellites are launched to an altitude of 19,312 nautical miles (22,224 miles) where their forward speed matches that of earth's rotation. What does this mean? These satellites appear to be stationary in the sky, if observed from earth. These satellites always observe the same portion of the atmosphere each and every day.

There are two operational GEO satellites positioned over the Continental U.S. they are referred to as the Geostationary Operational Environment Satellite (GOES) 16 and 18. GOES-16 is centered over the western Atlantic Ocean (75W) and observes the central and eastern portions of the United States and has been operational since 2017. GOES-18 is centered of the eastern Pacific Ocean and observes the western portion of the United States since late 2022. These are the primary satellites currently used in forecasting synoptic scale weather patterns.

Low Earth Orbiting (LEO) Another kind of satellite in use today is known as a low earth orbiting (LEO) satellite. These satellites are only launched to an altitude of around 500 nautical miles (575 miles). At an altitude of this height, these satellites are not in a geostationary orbit. Instead, these satellites appear to be synchronized with the sun, meaning that if viewed from the surface of the sun, these satellites would be stationary. These satellites complete a revolution about the earth in about 1.5 hours. Thus, it takes about 1.5 hours between images from a LEO satellite in the polar regions. Because these satellites make frequent passes over the Polar Regions, the term Polar Orbiter Environmental Satellite (POES) is given to these satellites launched by the United States. It should be noted that the resolution of POES satellites infrared imagery is about 4 times that of the imagery from GOES.

Satellite Channels

There are sixteen (16) channels that are part of the advanced baseline imager (ABI) that is currently deployed on the geostationary series of satellites currently residing over the United States. Here we'll cover the most commonly used channels for basic atmospheric analysis, red channel visible, clear IR, and midlevel water vapor. For more detail and information about all available channels can be found here.

Visible Imagery Visible satellite imagery is probably the most often thought of and easily understood of the various satellite products. In essence, visible satellite imagery collects light (or radiation) from the visible spectrum that has been reflected back into space. You can think of this type of satellite imagery as a giant photograph of Earth from space.

Visible imagery is very useful for forecasters, especially in convective weather situations. The spatial resolution allows for forecasters to watch for development of fine details in cloud structure, for example, cumulus clouds in regions where thunderstorm development is possible. Thus, visible satellite imagery provides short term monitoring capabilities of thunderstorm and general cloud development, before they are observed on radar. Additionally, the low-level wind field can be determined in severe weather environments but examining the orientation of the cumulus field.

There is one major drawback to using visible imagery – nighttime. Visible imagery uses visible light, so when the sun sets on a particular area, visible imagery quickly becomes useless. Thus we have other channels to use...

"Long-wave" Infrared Radiation Imagery As previously mentioned, visible satellite observations are only useful during daylight hours. However, weather doesn't stop when the sun sets. It is just for this purpose that we have the long-wave infrared radiation channel, or IR, for short. IR satellite observations measure the amount of long-wave radiation that has been *emitted* by earth, clouds, water, etc. The amount of IR radiation that is emitted by an object is proportional to the temperature of that object (Stefan-Boltzmann Law). The more IR radiation that is emitted by an object, the warmer that object is; the less IR radiation being emitted, the colder an object is. Knowing this, it is possible to determine the temperature of the object emitting the radiation.

Typically, the color scheme used is for warmer temperatures to be represented as darker areas and colder temperatures to be represented as whiter areas on IR imagery. However, you can enhance the satellite image by using color scales.

One caveat that may arise when using IR imagery is that it is based on temperature. Thus, in the winter during severe cold outbreaks, the northern United States and Canada will appear to be completely covered by clouds, when in reality, the ground is so cold that the IR imagery is detecting the cold ground.

Water Vapor Imagery One of the crucial ingredients in the development of clouds and various other weather features is water – specifically, midlevel water vapor (WV). This is where water vapor imagery figures into the equation. Water vapor in the middle atmosphere tends to absorb radiation emitted from earth's surface. Thus, while the name "water vapor" suggests that the satellite is looking for regions of moisture in the atmosphere, standard water vapor imagery is actually looking for regions where the radiation may have been absorbed by moisture in the atmosphere. This channel of the satellite works in the middle atmosphere between 650 mb and 350 mb. Thus, the water vapor image may show the atmosphere is very moist even though the low levels are extremely dry and vice-versa. Water vapor imagery is very useful for determining midlevel flow patterns as well as trough and ridges. The way water vapor imagery is interpreted is as follows:

> A dark pixel indicates that radiation was absorbed by the satellite. This is indicative of dry air in the middle portion of the atmosphere.
>
> A bright pixel indicates that radiation was NOT absorbed by the satellite. This means one of two things: the radiation was absorbed by moisture, or that the ground didn't emit a large amount of radiation.

> A band of very dark pixels poleward of a band of very bright pixels may indicate a jet stream. The jet stream or jet streak would be found along the area where the bright and dark bands meet.

Satellite Imagery

To plot satellite imagery, we can use the same plotting techniques we have been using all semester to plot gridded data. Satellite data is similar to model output in that each pixel of the image is evenly spaced and can be plotted using the `ImagePlot()` class from the declarative syntax in MetPy.

Import Packages Similar to accessing the Iowa State/Wyoming Upperair archive we can use the Python module Siphon to help gain access to the satellite data.

```
from datetime import datetime, UTC

from metpy.plots import declarative
from metpy.units import units
import numpy as np
from siphon.catalog import TDSCatalog
import xarray as xr
```

Get Data We use a set of variables to help construct the web URL including, region, channel, and satellite. This will help us when we simply want to view a slightly different satellite product. After constructing the data_url, then we can use the helper function to find us the right file and use Siphon to grab the correct link to the dataset on the UCAR server. Then we'll bring that dataset into our notebook and made to look like an xarray data object, which is what we have been working with through all of our model output.

```
# Set the datetime from the computer
date = datetime.now(UTC).replace(tzinfo=None)

# Create variables for URL generation
region = 'CONUS'
channel = 2
satellite = 'east'

# We want to match something like:
# https://thredds -test.unidata.ucar.edu/thredds/catalog/satellite/goes16/GOES16/Mesoscale -1/

# Construct the data_url string
data_url = (f'https://thredds.ucar.edu/thredds/catalog/satellite/goes/{satellite}/'
            f'products/CloudAndMoistureImagery/{region}/Channel{channel:02d}/'
            f'{date:%Y%m%d}/catalog.xml')

# Get list of files available for particular day
cat = TDSCatalog(data_url)

# Grab dataset for desired time
dataset = cat.datasets.filter_time_nearest(date, regex=r'_s(?P<strptime>\d{13})', strptime='%Y

# Open most recent file available
ds = dataset.remote_access(service='OPENDAP', use_xarray=True)

# Apply a square root correction for visible imagery only
if channel == 2:
    ds['Sectorized_CMI'].values = np.sqrt(ds['Sectorized_CMI'].values)

# Grab time from file and make into datetime object for plotting and later data access
vtime = ds.time.values.astype('datetime64[ms]').astype('O')
```

Plot Imagery From here, plotting becomes relatively routine, expect we'll want to use the `ImagePlot()` class since we are plotting a satellite *image*. We are going to use the time form the actual file for plotting purposes since we don't know the exact time of the data we got back from the server.

The files that we are using are called Cloud and Moisture Imagery files. That means that the data have been converted to a format that is more amenable for plotting (either reflectance or brightness temperature). So, the name of the variable we want to plot is `'Sectorized_CMI'`.

A simple map can then be generated via the following settings using a grey scale image.

```
# Declare the data we wish to plot
img = declarative.ImagePlot()
img.data = ds
img.field = 'Sectorized_CMI'
img.colormap = 'Greys_r'

# Plot the data on a map
panel = declarative.MapPanel()
panel.layers = ['coastline', 'borders', 'states']
panel.left_title = f'GOES East VIS Channel {channel:02d}'
panel.right_title = f'Valid Time: {vtime} UTC'
panel.plots = [img]

# Place the map on a figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]
pc.show()
```

Satellite Feature Identification

The entire atmosphere is connected through the temperature and pressure distribution. Changes in the local distribution of temperature can lead to dramatic changes in the structure of the atmosphere, including where we observe strong winds aloft. These areas of strong wind aloft are known as *jet streams* and they arise due to the temperature differences created between tropical and polar regions in each hemisphere (thermal wind balance). These fast-moving regions of air are a large driver of the speed and motion of mid-latitude cyclones. Therefore, it is important to be able to adequately identify jet streams on upper-level maps as well as satellite images. This section will introduce you to the basic concepts behind jet streams (and fronts to some extent) and how to identify them using satellite imagery.

Jet Streams There are two main jet streams that affect the North American continent, the polar and subtropical jet streams. The polar jet stream is the northern branch of strong westerly winds aloft and the subtropical jet stream is semi-constant jet stream that straddles with hemisphere around 25–30řN/S. Our main focus will be on the development and identification of the polar jet stream.

A jet stream is the result of the integrated temperature gradient of the atmosphere. Where we observe strong gradients (differences) in temperature at lower-levels of the atmosphere, there will be an area of strong winds aloft. This is a result of the atmosphere being maintained in *thermal wind balance*, and thus is the explanation for why we have strong winds aloft. Thermal wind balance states that where there are strong temperature gradients in the lower atmosphere, winds will increase with height up to the top of the troposphere (i.e., the tropopause). Inherently this means that where there are jet streams aloft, there are likely fronts at the surface (where the strong temperature gradients are). This links our jets to being key in the evolution of our mid-latitude cyclones.

Jet streams travel with the location of temperature gradients from winter to summer and back again. The strongest temperature gradients over North America occur during the winter months, December, January, and February. The temperature gradient is also at its southernmost extent. Therefore, the jet stream is at its strongest and located on average over the southern half of the United States. As the atmosphere transitions from winter to spring and then to summer, the temperature gradient treks northward and decreases in intensity. This moves the jet stream northward and weakens the overall speed, on average.

On an upper-level map (e.g., 300 hPa) it is rather easy to identify the region of strongest wind by inspecting the plotted wind barbs or contours of wind speed at the level. However, the Earth is primarily covered by our vast oceans where we do not regularly obtain *in situ* upper-level or surface observations, therefore we must use other tools for identifying the regions of strong winds aloft.

Jet Stream Identification The most useful satellite image for identifying upper-level flow patterns is the water vapor imagery. Water vapor is ubiquitous throughout the atmosphere, meaning that there is some quantity of water vapor at all times in all places in the atmosphere. This water vapor can then be traced as it circulates within our weather systems around the globe. For jet streams there is a distinct pattern that occurs within water vapor imagery that makes it possible to identify the location of intense jet streams.

Jet streams are located where there is a strong gradient (difference) in moisture on water vapor imagery (Figure 35). This boundary is a result of circulations associated with the jet itself. The jet stream is located parallel to the axis of the boundary as illustrated in Figure 35.

A real example of a jet stream as observed in water vapor imagery is given in Figure 36. The main part of the jet stream is located across central California into central Nevada. The sharp boundary in the water vapor imagery indicates southern boundary of the strongest part of the jet stream. Note the contours of wind speed and how they are oriented parallel to that boundary. This real-life example is a cross between the schematic image in Figure 35 and Figure 37.

Additionally, a mature mid-latitude cyclone will exhibit the shape of a comma (Figure 37). This is the result of airflow through a mid-latitude cyclone. Identifying the stage of cyclone development is key to being able to forecast well in a data sparse region. The use of satellites to identify important features about the mid-latitude cyclone will aid in making a good forecast.

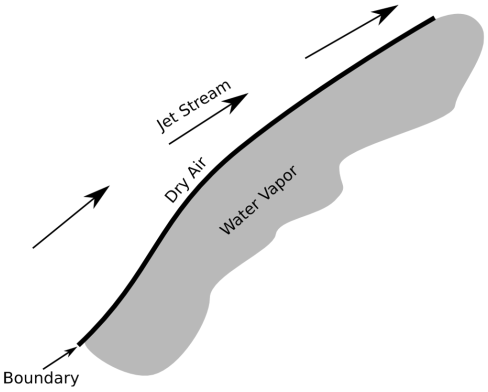


Figure 35: Schematic diagram of the water vapor boundary and location of jet streams in relation to those boundaries.

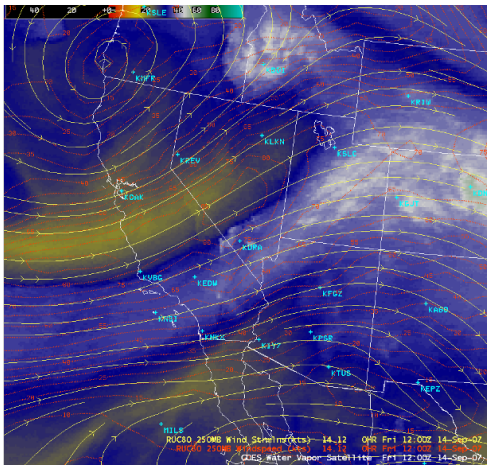


Figure 36: Enhanced water vapor imagery with 250 hPa wind speed analyzed in (kts). Image courtesy of CIMSS at https://cimss.ssec.wisc.edu/satellite-blog/wp-content/uploads/sites/5/2007/09/us_water_vapor_20070914_1200.png

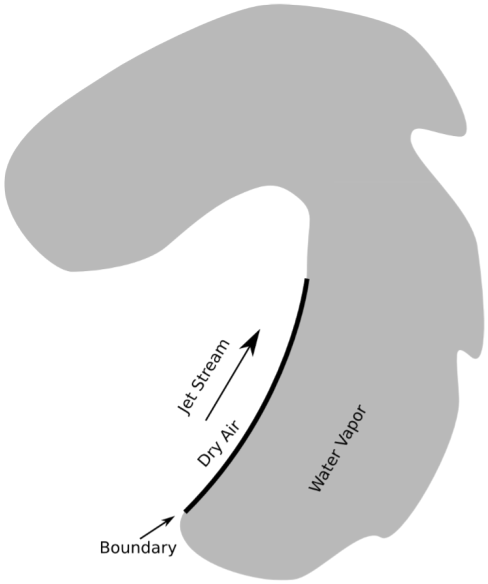


Figure 37: Schematic diagram of a developing mid-latitude cyclone in water vapor imagery with a jet stream.

Wave Pattern The upper atmosphere is also marked by the wavy pattern of troughs and ridges that is the result of a differentially heated, rotating planet. These waves (a trough/ridge combination) is called a Rossby wave, after the 20t̂hĉentury atmospheric scientist Carl Gustav-Rossby who identified the reason for the wavy pattern. Water vapor can also be used to identify these troughs and ridges by detecting cyclonic (counter-clockwise in the Northern Hemisphere) and anticyclonic (clockwise in the Northern Hemisphere) spin (vorticity) within the motions of the water vapor at upper-levels (Fig. 4).

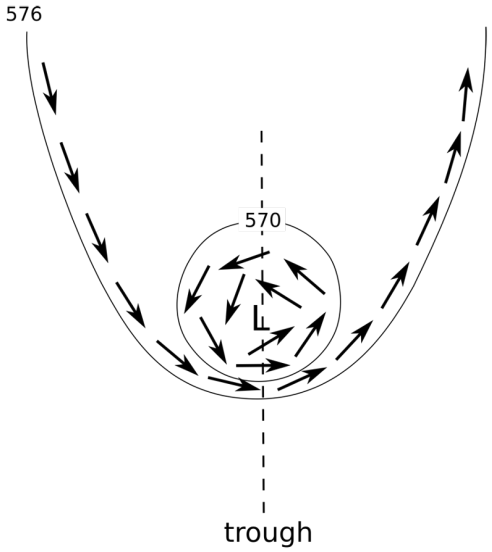


Figure 38: Schematic of a trough aloft and the arrows indicate how the water vapor will rotate around a trough in the Northern Hemisphere.

Color-Enhanced Satellite Imagery

To plot satellite imagery we can use data that we bring in through our local data feed or remote access from UCAR. Satellite data are stored in `/data/ldmdata/satellite` for both GOES-16 and GOES-17 or at `https://thredds.ucar.edu/thredds/idd/satellite.html`. We'll be using the remote data source in this notebook. There are a couple of different sectors that we can view the data from, but the most common for synoptic-dynamic purposes would be the CONUS projection. Data are available every 5 min.

For this plotting we are doing to use the declarative plotting syntax using Python and the MetPy module.

Import Packages

```
from datetime import datetime, UTC

import metpy.calc as mpcalc
from metpy.plots import declarative
from metpy.units import units
import numpy as np
from siphon.catalog import TDSCatalog
import xarray as xr
```

Get Data

```
date = datetime.now(UTC).replace(tzinfo=None)

# Create variables for URL generation
region = 'CONUS'
channel = 9
satellite = 'east'

# We want to match something like:
# https://thredds -test.unidata.ucar.edu/thredds/catalog/satellite/goes16/GOES16/Mesoscale -1/

# Construct the data_url string
data_url = (f'https://thredds.ucar.edu/thredds/catalog/satellite/goes/{satellite}/'
            f'products/CloudAndMoistureImagery/{region}/Channel{channel:02d}/'
            f'{date:%Y%m%d}/catalog.xml')

# Get list of files available for particular day
cat = TDSCatalog(data_url)

# Grab dataset for desired time
dataset = cat.datasets.filter_time_nearest(date, regex=r'_s(?P<strptime>\d{13})', strptime='%Y

# Open most recent file available
ds = dataset.remote_access(service='OPENDAP', use_xarray=True)

# Apply a square root correction for visible imagery only
if channel == 2:
    ds['Sectorized_CMI'].values = np.sqrt(ds['Sectorized_CMI'].values)

# Grab time from file and make into datetime object for plotting and later data access
vtime = ds.time.values.astype('datetime64[ms]').astype('O')
```

Plot Imagery Color enhancement colormaps are available from MetPy and can be found at: https://unidata.github.io/MetPy/latest/api/generated/metpy.plots.ctables.html

Water Vapor Colormaps:

- WVCIMSS_r
- wv_tpc_r

Infrared Colormaps:

- ir_drgb
- ir_tmpc
- ir_rgbv
- ir_bd

> **Note**
> The `_r` can be added to any colormap name to reverse the colormap. Depending on the data you may have to switch between reveresed and non-reveresed colormaps.

```
# Declare the data we wish to plot
img = declarative.ImagePlot()
img.data = ds
img.field = 'Sectorized_CMI'
img.colormap = 'WVCIMSS_r'
img.colorbar = 'horizontal'
img.image_range = (180, 280)

# Plot the data on a map
panel = declarative.MapPanel()
panel.layers = ['coastline', 'borders', 'states']
panel.left_title = f'GOES East WV Channel {channel:02d}'
panel.right_title = f'Valid Time: {vtime} UTC'
panel.plots = [img]

# Place the map on a figure
pc = declarative.PanelContainer()
pc.size = (20, 16)
pc.panels = [panel]
#pc.save(f'GOES_East_{vtime:%Y%m%d_%H%M}_colorenhanced.png', bbox_inches='tight', dpi=150)
pc.show()
```

0.4 Appendix

0.4.1 Common Errors and Solutions

Reading error messages can be challenging since it may be hard to exactly decipher what might have gone wrong. In this appendix some of the most common errors and their solutions are presented. This is by no means an exhaustive list and through making more maps and solving the errors that come up, it will get easier.

Subsetting Errors

One of the most common errors when plotting a map of contours, color-filled contours, etc. the data must be two-dimensional (i.e., 2D). When reading in gridded data it is common for it to have four dimensions: time, vertical level, x (longitude), and y (latitude). From there it will need to be subset for a particular time and vertical level to be able to plot the map. A common error if there was not enough subsetting was completed might be:

```
ValueError: Must provide a combination of subsetting values to give either 2D data or
3D data subset for plotting with third dimension size 3 or 4
```

In reading the error above, the critical piece is the last part where the error message indicates that the dimension size is 3 or 4, which means not enough subsetting has been done. The solution is that you likely need to subset using the `time` or `level` attribute to get to a two-dimensional array for plotting.

```
AttributeError: vertical attribute is not available.
```

The error above indicates that an attempt to subset the vertical level attribute via the declarative syntax cannot be completed because there is no vertical dimension that has not already been subset (or exists if the variable doesn't have any vertical dimension). The solution is to set the `level` attribute to `None`.

```
AttributeError: time attribute is not available.
```

Similar to the vertical attribute error, this message appears when time has already been subset and therefore cannot be subset again. The solution is to set the `time` attribute to `None`.

No Contours

If you get no error, but a blank map there are a couple of different things that could be going on with the settings.

- Check the setting on contours to ensure it is set appropriately for what you are attempting to plot.
- Consider what units the data are in as that may affect the contours setting.
- Did you subset the data for a particular geographic region but are now plotting a different region through the `area` attribute?

Missing Data

If you get no error, but some data are plotted but other seem to be missing.

- If using gridded data, double check the subsetting of the x (longitude) and y (latitude) dimensions as you may need to expand the bounds of subsetting to get your desired results.
- If using point data (e.g., surface observations or upperair observations) ensure you that have have requested and have data for the time that you desire. It is possible that the data you read in defaulted to a different time that is outside the bounds of the date you are using for plotting.

Missing Wind Barbs

When plotting multiple parameters there are times when the overlay of certain variables will cover up other types of plots. This most commonly occurs with wind barbs. To ensure the best visualization of your data it is best to put any `ImagePlot()`, `RasterPlot()`, or `FilledContourPlot()` as the first thing in your plot list and have wind barbs last.

There can be other instances where the zorder (the Pythonic word for the a graphic layer) can cause unusual layering of data and other map features. Currently there are no straight forward ways to overcome these issues, but should be possible in the near future.

0.4.2 Accessing Surface Data

There are a number of different methods for accessing current (and archived) surface observations. The first first access point is only accessible within the Valparaiso University domain. The second two are accessible to anyone on the web.

Valpo Local Data Access

Local Access Current Data: `/data/ldmdata/surface/sao`
The current data are roughly the past two weeks worth of surface data.

```
from metpy.io import metar

data = '/data/ldmdata/surface/sao/20220114_sao.wmo'

df = metar.parse_metar_file(data, year=2021, month=1)
```

Valpo Remote Data Access

Remote Access Current Data (Valpo: http://bergeron.valpo.edu/current_surface_data)
The current data are roughly the past two weeks worth of surface data.

```
from io import StringIO
from urllib.request import urlopen

from metpy.io import metar

data = StringIO(urlopen('http://bergeron.valpo.edu/current_surface_data/'
                        '20201221_sao.wmo').read().decode('utf -8', 'backslashreplace'))

df = metar.parse_metar_file(data, year=2020, month=12)
```

Iowa State Current and Archive Remote Data Access

Remote Access Current and Archived Data (Iowa State Archive: 1950-current):

```
from io import StringIO
from urllib.request import urlopen

from metpy.io import metar
import pandas as pd

# Set a date and time for the observations you want
date = datetime(1993, 3, 13, 12)

# Set a time window to look for observations
dt = timedelta(minutes=15)

# Use datetime and time window to set final start/end times
sdate = date - dt
edate = date + dt

# Use Pandas remote csv reading capability to grab desired data
# Make Missing values (\'M\') into NaNs
# Replace Trace precip with very small float value
```

```
data = pd.read_csv('http://mesonet.agron.iastate.edu/'
                   'cgi -bin/request/asos.py?data=all&tz=Etc/UTC&format=comma&'
                   f'latlon=yes&elev=yes&year1={sdate.year}&month1={sdate.month}&'
                   f'day1={sdate.day}&hour1={sdate.hour}&minute1={sdate.minute}&'
                   f'year2={edate.year}&month2={edate.month}&day2={edate.day}&'
                   f'hour2={edate.hour}&minute2={edate.minute}', skiprows=5,
                   na_values=['M'],
                   low_memory=False).replace('T', 0.00001).groupby('station').tail(1)

# Parse METAR observations using MetPy
df = metar.parse_metar_file(StringIO('\n'.join(val for val in data.metar)),
                            year=date.year, month=date.month)
```

0.4.3 Accessing Upperair Data

There are currently two different data sources that provide access to current and archived upperair observations: Iowa State and University of Wyoming.

Iowa State Upperair Data Access

This dataset is best for requests needing access to a single level of data for multiple upperair stations at a single time period. For example, when requesting 850-hPa data to plot an 850-hPa map across the continental United States.

```
from datetime import datetime

from metpy.io import add_station_lat_lon
from siphon.simplewebservice.iastate import IAStateUpperAir

# Set date for desired UPA data
date = datetime(2021, 12, 31, 0)

# Request data using Siphon request for data from Iowa State Archive
df = IAStateUpperAir.request_all_data(date)

# Add lat/long data and drop stations for which we don't have lat/lons
df = add_station_lat_lon(df, 'station').dropna(subset=['latitude', 'longitude'])

# Eliminate the station KVER
df = df[df.station != 'KVER']
```

The code above will gather all of the available upperair observations (for North America) for the given date, then add station latitude and longitude information for each station and get rid of any that were left with N/A locations. Additionally, the final line will remove the 'KVER' station as MetPy currently gives the station information for a surface observation site in the state of Missouri as opposed to the actual location in Mexico. The data retrieved will have ALL of the pressure levels reported for each station and you can subset to a particular desired level for plotting purposes within the declarative syntax.

Wyoming Sounding Data Access

This dataset is best used for requests for the entire vertical profile for a single station at a single time. For example, if you are attempting to plot a skew-T log-p diagram, this would be an optimal data source.

```
from datetime import datetime

from siphon.simplewebservice.wyoming import WyomingUpperAir

# Set date that you want
# Data goes back to the 1970's
date = datetime(2007, 5, 4, 12)

# Set station ID, there are different stations back in the day
# Current station IDs found at http://weather.rap.ucar.edu/upper
station = 'DDC'

# Use Siphon module to grab data from remote server
df = WyomingUpperAir.request_data(date, station)
```

Here the above code will use the combination of the station string and the date to obtain a single vertical profile for the station at the given time.

Note
Station locations and names have changed with time, so you may have to look up historic three letter identifiers to obtain the data you want, especially if you are accessing data from prior to the year 2000.

0.4.4 Access Gridded Archive Data

There are many times when we might want to access past (archive) datasets to analyze and visualize meteorological information. There are a host of different gridded data that are available through various THREDDS servers that allow simple remote access to the requested data. Unfortunately, these data are not all located in the same place or use the same conventions for storing the data. Data availability is also highly contigent on the date/time. Models get updated, new ones come along, and old ones are retired. Additionally, some datasets only contain the best analysis time (e.g., F000) for a given model run (at least 00, 06, 12, and 18), whereas others contain all of the forecast hours. Therefore you'll want to look carefully at the data avalability times and the full scope of the available data to help you choose the best data access method for the question(s) you are attempting to answer.

```
from datetime import datetime

import metpy.calc as mpcalc
import xarray as xr
```

RDA GFS Analysis Archive

The NCAR Research Data Archive (RDA) is a great resource for gridded data that can be easily utilized in the declarative syntax within MetPy. Depending on the dates desired, there are different end points to point to gain access to the desired data. This archive contains all of the forecast hours in addition to the analysis (e.g., F000) times.

GFS Data from 1 January 2009 - near current

GFS One Degree Archive - Older Data 18 UTC 30 July 1999 through 06 UTC 6 December 7

```
# Data available July 30, 1999 at 18 UTC through December 6, 2007 at 06 UTC
ds = xr.open_dataset('https://tds.gdex.ucar.edu/thredds/dodsC/aggregations/g/d083002/1/TP')

print(ds.Geopotential_height_isobaric)
```

GFS One Degree Archive - to current 12 UTC 6 December 2007 to current

```
# Data Available December 6, 2007 at 06 UTC through October 2025
ds = xr.open_dataset('https://tds.gdex.ucar.edu/thredds/dodsC/aggregations/g/d083002/2/TP')

print(ds.Geopotential_height_isobaric)
```

GFS 0.25 Degree Archive - to current 12 UTC 6 December 2007 to present

```
date = datetime(2019, 3, 12, 0)

# Current data from January 15, 2015 at 00 UTC through present
# Direct file access
ds = xr.open_dataset('https://tds.gdex.ucar.edu/thredds/dodsC/files/g/d084001/'
                     f'{date:%Y}/{date:%Y%m%d}/gfs.0p25.{date:%Y%m%d%H}.f000.grib2')

print(ds.Geopotential_height_isobaric)
```

North American Regional Reanalysis

Another set of archive gridded data are those available from NOAA NCEI. This data source has been more tempermental and is not always available. This is a reanalysis dataset that was stored in this location from September 2014. The data are on the NAM grid.

NARR Data from January 1979 - September 2014

```
date = datetime(1999, 10, 15, 0)

# North American Regional Reanalysis Analysis back to 0 UTC 1 January 1979 to 30 September 202
# Note: The 2009 months of March through December wer re -run and they are in the year -month
# with rerun added to the end of it (e.g., 200903rerun) - you'll need to modify the second lin
# open_dataset call to be f'{date:%Y%m}rerun/merged_AWIP32.{date:%Y%m%d}.tar'
ds = xr.open_dataset('https://tds.gdex.ucar.edu/thredds/dodsC/files/g/d608000/NARRraw/'
                     f'{date:%Y%m}/merged_AWIP32.{date:%Y%m%d}.tar')

print(ds.Geopotential_height_isobaric)
```

0.4.5 Working with NAM Data

This notebook demonstrates plotting gridded NAM data, which has the complication of having winds that are in grid relative coordinates and NOT earth relative. This will entail the need to use some MetPy functionality to easy the plotting of NAM winds.

```
from datetime import datetime, timedelta, time, UTC

from metpy.plots import declarative
from metpy.units import units
import xarray as xr
```

Get Data

Here we are using current data to demonstrate contour plotting with wind barbs. If you have a different data source you would like to use, simply change out the call to remote data in the `open_dataset` function to be appropriate for the local or remote data you would like to access. Future chapters will demonstrate different data sources that can be commonly used. Here we'll use the NAM data from the Unidata THREDDS server.

```
# Set the date/time of the model run
# The following code will get you yesterday at 12 UTC
yesterday = datetime.now(UTC) - timedelta(days=1)
model_run_date = datetime.combine(yesterday, time(12))

# Remote access to the dataset from the UCAR site
ds = xr.open_dataset('https://thredds.ucar.edu/thredds/dodsC/grib'
                     '/NCEP/NAM/CONUS_12km/NAM_CONUS_12km_'
                     f'{model_run_date:%Y%m%d}_{model_run_date:%H%M}.grib2')
```

Initial Plot

Here let's plot the data that we get from the NAM. Pay particular attention to the winds barbs relationship to the geopotential height contours.

```
# Set the plot time with forecast hours
plot_time = model_run_date + timedelta(hours=0)

# Set attributes for plotting contours
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'Geopotential_height_isobaric'
cntr.level = 300 * units.hPa
cntr.time = plot_time
cntr.contours = range(0, 10000, 120)
cntr.clabels = True

# Add wind barbds
barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = plot_time
barbs.field = ['u -component_of_wind_isobaric',
               'v -component_of_wind_isobaric']
barbs.level = 300 * units.hPa
barbs.skip = (25, 25)
```

```
barbs.plot_units = 'knot'

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = 'awips'
panel.layers = ['states', 'coastline', 'borders']
panel.title = f'300 -hPa Geopotential Heights at {plot_time}'
panel.plots = [cntr, barbs]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (12, 12)
pc.panels = [panel]

# Show the figure
pc.show()
```

Sanity Check Here we are plotting 300-hPa geopotential heights and winds. At that level, the winds should be geostrophic, meaning that the winds barbs should be parallel to the contours of geopotential height. When we look toward the edge of the map this is where we see that the wind barbs are not clearly conforming to geostrophic balance. The winds are crossing the geopotential height contours at an angle that they shouldn't be. This behaviour is due to the curvature of the projection not aligning with the assumptions made to plot the wind barbs.

Luckily we have an easy solution built in to the MetPy declarative syntax to convert the stored grid relative winds and make them earth relative for plotting. The attribute to set as part of the `BarbPlot()` class is `earth_relative`. We need to tell MetPy that our winds are NOT earth relative. This is a boolean setting, such that for our NAM winds `earth_relative` should be set to `False`.

Corrected NAM Winds

Let's try it out and compare the NAM wind barbs that are grid-relative to our recomputed earth relative wind barbs using the MetPy attribute.

```
# Set the plot time with forecast hours
plot_time = model_run_date + timedelta(hours=0)

# Set attributes for plotting contours
cntr = declarative.ContourPlot()
cntr.data = ds
cntr.field = 'Geopotential_height_isobaric'
cntr.level = 300 * units.hPa
cntr.time = plot_time
cntr.contours = range(0, 10000, 120)
cntr.clabels = True

# Add wind barbds
barbs = declarative.BarbPlot()
barbs.data = ds
barbs.time = plot_time
barbs.field = ['u -component_of_wind_isobaric',
               'v -component_of_wind_isobaric']
barbs.level = 300 * units.hPa
```

```
barbs.skip = (25, 25)
barbs.plot_units = 'knot'

# Add wind barbds
barbs2 = declarative.BarbPlot()
barbs2.data = ds
barbs2.time = plot_time
barbs2.field = ['u -component_of_wind_isobaric',
                'v -component_of_wind_isobaric']
barbs2.level = 300 * units.hPa
barbs2.skip = (25, 25)
barbs2.plot_units = 'knot'
barbs2.color = 'tab:red'
barbs2.earth_relative = False

# Set the attributes for the map
# and put the contours on the map
panel = declarative.MapPanel()
panel.area = 'awips'
panel.layers = ['states', 'coastline', 'borders']
panel.title = (f'300 -hPa Geopotential Heights at {plot_time}\n'
               'grid -relative wind barbs (black) and earth -relative wind barbs (red)')
panel.plots = [cntr, barbs, barbs2]

# Set the attributes for the panel
# and put the panel in the figure
pc = declarative.PanelContainer()
pc.size = (15, 15)
pc.panels = [panel]

# Show the figure
pc.show()
```

In the above graphic we can observe the corrected winds (in red) and notice that the largest differences are noted on the edges of the map where the curvature of the projection is notably different between the projection of the winds and the projection of the plot. The only change that was needed to get the winds in the correct orientation was to set the attribute `earth_relative` to `False` since the NAM output stores wind components in a grid-relative format.

0.4.6 MetPy Declarative Syntax

In addition to the great documentation contained within the code itself, the following pages summarize the most commonly used elements of the declarative syntax and all of the attributes that can be set as a quick reference guide. An additional resource to use is the MetPy Declarative Syntax Tutorial.

MetPy - PlotObs()

To make a plot of observations requires the use of the `PlotObs` module from the MetPy declarative syntax. This is a Class of the declarative syntax and note that Classes contain MiXeD case names. This class contains a number of attributes that can be set to specify what and how to plot the data in a figure. The descriptions of some of the basic elements of each element are given below.

`PlotObs()`

Attribute	Description
colors	List of the scalar and symbol field colors.
data	Pandas DataFrame that contains the fields to be plotted.
fields	Name of the scalar or symbol fields to be plotted.
fontsize	An integer value to set the font size of station plots.
formats	List of the scalar, symbol, and text field data formats.
level	The level of the field to be plotted.
locations	List of strings for scalar or symbol field plotting locations.
plot_units	A list of the desired units to plot the fields in.
reduce_points	Float to reduce number of points plotted.
time	Set the valid time to be plotted as a datetime object.
time_window	Set a range to look for data to plot as a timedelta object.
vector_field	List of the vector field to be plotted.
vector_field_color	String color name to plot the vector.
vector_field_length	Integer value to set the length of the plotted vector.
vector_plot_units	The desired units to plot the vector field in.
mpl_args	Supply a dictionary of valid Matplotlib keyword arguments to modify how the plot variable is drawn.

Attribute Descriptions

data

- This attribute needs to be set to the DataFrame variable containing the fields that you desire to plot.

fields

- This attribute is a list of variable names from your DataFrame that you desire to plot at the given locations around the station model.

level

- For a surface plot this needs to be set to None.

time

- This attribute needs to be set to subset your data attribute for the time of the observations to be plotted. This needs to be a datetime object.

locations (optional)
- This attribute sets the location of the fields to be plotted around the surface station model. The default location is center ('C')

time_range (optional)
- This attribute allows you to define a window for valid observations (e.g., 15 minutes on either side of the datetime object setting). This is important for surface data since actual observed times are not all exactly on the hour. If multiple observations exist in the defined window, the most recent observations is retained for plotting purposes.

formats (optional)
- This attribute sets a formatter for text or plotting symbols around the station model. For example, plotting mean sea-level pressure is done in a three-digit code and a formatter can be used to achieve that on the station plot.
  MSLP Formatter: lambda v: format(10 * v, '.0f')[-3:]
  For plotting symbols use the available MetPy options through their name. Valid symbol formats are current_weather, sky_cover, low_clouds, mid_clouds, high_clouds, and pressure_tendency.

colors (optional)
- This attribute can change the color of the plotted observation. Default is 'black'. Acceptable colors are those available through Matplotlib: https://matplotlib.org/stable/_images/sphx_glr_named_colors_003.png

plot_units (optional)
- A list of the desired units to plot the fields in. Will only work if data have units attached.

vector_field (optional)
- This attribute can be set to a list of wind component values for plotting (e.g., ['uwind', 'vwind'])

vector_field_color (optional)
- Same as colors except only controls the color of the wind barbs. Default is 'black'.

vector_field_length (optional)
- Set an integer value for the length of the plotted vector. Default is 6.

vector_plot_units (optional)
- The desired units you want the vector field plotted in. Will only work if data have units attached.

fontsize (optional)
- An integer value to set the font size of station plots. Default is 10 pt.

reduce_points (optional)
- This attribute can be set to a real number to reduce the number of stations that are plotted. Default value is zero (e.g., no points are removed from the plot).

fontsize (optional)
- This attribute can be set to an integer value to set the font size of station plots. Default is 10 pt.

mpl_args

- This attribute can be set to a dictionary containing keyword arguments (kwarg) valid for the various Matplotlib plotting functions to enhance how a plot is created. The kwargs are those to specify the StationPlot object. For example, you can use this attribute to set the `zorder` kwarg to alter what aspects of the your plot are on top of one another. NOTE: Setting the mpl_args trait will override any other trait that corresponds to a specific kwarg for the particular plot type (e.g., linecolor, linewidth).

MetPy - ContourPlot()

This class is designed to plot contours of gridded data, most commonly model output from the GFS, NAM, or RAP.

`ContourPlot()`

Attribute	Description
clabels	A boolean (True/False) on whether to plot tour labels.
contours	A list of values to contour or an integer nu of contour levels.
data	Xarray dataset that contains the field to be ted.
field	Name of the field to be plotted.
label_fontsize	An integer, float, or string value to set the size of labels for contours.
latitude	The latitude coordinate of the field to be plo
level	The level of the field to be plotted.
linecolor	A string value to set the color of plotted tours; default is black.
linestyle	A string value to set the linestyle (e.g., dash or *None*; default is *None*, which, when u monochrome line colors, uses solid lines for itive values and dashed lines for negative va
linewidth	An integer value to set the width of plotted tours; default value is 2.
longitude	The longitude coordinate of the field to be ted.
name	Generate a name for the plot.
plot_units	The desired units to plot the field in.
scale	Scale the field to be plotted by the value gi
smooth_contour	Spline interpolation to smooth contours.
smooth_field	Number of smoothing passes using smoother.
time	Set the valid time to be plotted as a date object.
mpl_args	Supply a dictionary of valid Matplotlib key arguments to modify how the plot variab drawn.

Attribute Descriptions

data

- This attribute must be set with the variable name that contains the xarray dataset. (Typically this is the variable ds)

field

- This attribute must be set with the name of the variable that you want to contour. For example, to plot the heights of pressure surfaces from the GFS you would use the name 'Geopotential_height_isobaric'

level
- This attribute sets the level of the data you wish to plot. If it is a pressure level, then it must be set to a unit bearing value (e.g., 500*units.hPa). If the variable does not have any vertical levels (e.g., mean sea-level pressure), then the level attribute must be set to None.

time
- This attribute must be set with a datetime object, just as with the PlotObs() class. To get a forecast hour, you can use the timedelta function from datetime to add the number of hours into the future you wish to plot. For example, if you wanted the six hour forecast from the 00 UTC 2 February 2020 model run, then you would set the attribute with:
  datetime(2020, 2, 2, 0) + timedelta(hours=6)

contours
- This attribute sets the contour values to be plotted with a list. This can be set manually with a list of integers in square brackets (e.g., [5400, 5460, 5520, 5580, 5640, 5700]) or programmatically (e.g., `range(0, 10000, 60)`). The second method is a way to easily set a contour interval (in this case 60).

clabels
- This attribute can be set to True if you desire to have your contours labeled.

linestyle
- This attribute can be set to make the contours 'solid', 'dashed', 'dotted', or 'dashdot'. Default is 'solid'.

linewidth
- This attribute alters the width of the contours (defaults to 1). Setting the value greater than 1 will yield a thicker contour line.

linecolor
- This attribute sets the color of the contour lines. Default is 'black'. All colors from matplotlib are valid.

plot_units
- If you want to change the units for plotting purposes, add the string value of the units desired. For example, if you want to plot temperature in Celsius, then set this attribute to 'degC'.

smooth_contour
- This attribute is best used to smooth contours when contouring a sparse grid (e.g., when your data has a large grid spacing). This will produce smoothed geopotential heights for all levels available in the DataArray using a spline interpolation.

smooth_field
- This attribute is best used to smooth contours when contouring a higher resolution (smaller grid spacing with lots of grid points) field. This will produce a smoothed field using the nine-point smoother twice.

mpl_args
- This attribute can be set to a dictionary containing keyword arguments (kwarg) valid for the various Matplotlib plotting functions to enhance how a plot is created. For example, you can use this attribute to set the `zorder` kwarg to alter what aspects of the your plot are on top of one another. NOTE: Setting the mpl_args trait will override any other trait that corresponds to a specific kwarg for the particular plot type (e.g., linecolor, linewidth).

MetPy - FilledContourPlot()

Make color-filled contours plots by defining appropriate traits.

`FilledContourPlot()`

Attribute	Description
clabels	A boolean (True/False) on whether tour labels.
colorbar	A string (horizontal/vertical) on wh a colorbar to the plot.
colorbar_fontsize	An integer, float, or string value to size of labels for the colorbar.
colormap	The string name for a Matplotlib or ormap.
contours	A list of values to contour or an inte of contour levels.
data	Xarray dataset that contains the fiel ted.
field	Name of the field to be plotted.
image_range	A tuple of min and max values tha the range of values to color the raste
label_fontsize	An integer, float, or string value to size of labels for contours.
level	The level of the field to be plotted.
plot_units	The desired units to plot the field in
scale	Scale the field to be plotted by the v
smooth_contour	Spline interpolation to smooth conto
smooth_field	Number of smoothing passes smoother.
time	Set the valid time to be plotted as object.
mpl_args	Supply a dictionary of valid Matplot arguments to modify how the plot drawn.

Attribute Descriptions

data

- This attribute must be set with the variable name that contains the xarray dataset. (Typically this is the variable ds)

field

- This attribute must be set with the name of the variable that you want to contour. For example, to plot the heights of pressure surfaces from the GFS you would use the name 'Geopotential_height_isobaric'

level

- This attribute sets the level of the data you wish to plot. If it is a pressure level, then it must be set to a unit bearing value (e.g., 500*units.hPa). If the variable does not have any vertical levels (e.g., mean sea-level pressure), then the level attribute must be set to None.

time
- This attribute must be set with a datetime object, just as with the PlotObs() class. To get a forecast hour, you can use the timedelta function from datetime to add the number of hours into the future you wish to plot. For example, if you wanted the six hour forecast from the 00 UTC 2 February 2020 model run, then you would set the attribute with:
  datetime(2020, 2, 2, 0) + timedelta(hours=6)

contours
- This attribute sets the contour values to be plotted with a list. This can be set manually with a list of integers in square brackets (e.g., [5400, 5460, 5520, 5580, 5640, 5700]) or programmatically (e.g., `range(0, 10000, 60)`). The second method is a way to easily set a contour interval (in this case 60).

colormap
- This attribute is used to set a valid colormap from either Matplotlib or MetPy:
  Matplotlib Colormaps
  MetPy Colormaps

colorbar
- This attribute can be set to 'vertical' or 'horizontal', which is the location the colorbar will be plotted on the panel.

image_range
- A set of values indicating the minimum and maximum for the data being plotted. This attribute should be set as (min_value, max_value), where min_value and max_value are numeric values.

plot_units
- If you want to change the units for plotting purposes, add the string value of the units desired. For example, if you want to plot temperature in Celsius, then set this attribute to 'degC'.

smooth_contour
- This attribute is best used to smooth contours when contouring a sparse grid (e.g., when your data has a large grid spacing). This will produce smoothed geopotential heights for all levels available in the DataArray using a spline interpolation.

smooth_field
- This attribute is best used to smooth contours when contouring a higher resolution (smaller grid spacing with lots of grid points) field. This will produce a smoothed field using the nine-point smoother twice.

mpl_args
- This attribute can be set to a dictionary containing keyword arguments (kwarg) valid for the various Matplotlib plotting functions to enhance how a plot is created. For example, you can use this attribute to set the `zorder` kwarg to alter what aspects of the your plot are on top of one another. NOTE: Setting the mpl_args trait will override any other trait that corresponds to a specific kwarg for the particular plot type (e.g., linecolor, linewidth).

MetPy - RasterPlot()

Make raster plots by defining relevant traits.

`RasterPlot()`

Attribute	Description
colorbar	A string (horizontal/vertical) on whether to a a colorbar to the plot.
colorbar_fontsize	An integer, float, or string value to set the f size of labels for the colorbar.
colormap	The string name for a Matplotlib or MetPy ormap.
data	Xarray dataset that contains the field to be p ted.
field	Name of the field to be plotted.
image_range	A tuple of min and max values that repres the range of values to color the rasterized ima
level	The level of the field to be plotted.
plot_units	The desired units to plot the field in.
scale	Scale the field to be plotted by the value giv
smooth_contour	Spline interpolation to smooth contours.
smooth_field	Number of smoothing passes using 9 smoother.
time	Set the valid time to be plotted as a datet object.
mpl_args	Supply a dictionary of valid Matplotlib keyw arguments to modify how the plot variabl drawn.

Attribute Descriptions

colorbar

- A string (horizontal/vertical) on whether to add a colorbar to the plot.
  To add a colorbar associated with the plot, set the trait to `horizontal` or `vertical`, specifying the orientation of the produced colorbar. The default value is `None`.

colorbar_fontsize

- An integer, float, or string value to set the font size of labels for the colorbar.
  This trait sets the font size of labels for the colorbar. Accepts size in points or relative size. Allowed relative sizes are those of Matplotlib: 'xx-small', 'x-small', 'small', 'medium', 'large', 'x-large', 'xx-large'.

colormap

- The string name for a Matplotlib or MetPy colormap.
  For example, the Blue-Purple colormap from Matplotlib can be accessed using 'BuPu'.

data

- Xarray dataset that contains the field to be plotted.

field

- Name of the field to be plotted.
  This is the name of the variable from the dataset that is to be plotted. An example, from a model grid file that uses the THREDDS convention for naming would be *Geopotential_height_isobaric* or *Temperature_isobaric*. For GOES-16/17 satellite data it might be *Sectorized_CMI*. To check for the variables available within a dataset, list the variables with the following command assuming the dataset was read using xarray as *ds*, *list(ds)*

image_range

- A tuple of min and max values that represent the range of values to color the rasterized image.
  The min and max values entered as a tuple will be converted to a `matplotlib.colors.Normalize`instance for plotting.

level

- The level of the field to be plotted.
  This is a value with units to choose a desired plot level. For example, selecting the 850-hPa level, set this parameter to `850 * units.hPa`. Note that this requires your data to have a vertical dimension coordinate.

plot_units

- The desired units to plot the field in.
  Setting this attribute will convert the units of the field variable to the given units for plotting using the MetPy Units module.

scale

- Scale the field to be plotted by the value given.
  This attribute will scale the field by multiplying by the scale. For example, to scale vorticity to be whole values for contouring you could set the scale to `1e5`, such that the data values will be multiplied by $10\hat{5}$.

smooth_contour

- Spline interpolation to smooth contours.
  This attribute requires settings for the `metpy.calc.zoom_xarray` function, which will produce a spline interpolation given an integer zoom factor. Either a single integer specifying the zoom factor (e.g., 4) or a tuple containing two integers for the zoom factor and the spline interpolation order can be used. The default spline interpolation order is 3.
  This is best used to smooth contours when contouring a sparse grid (e.g., when your data has a large grid spacing).

smooth_field

- Number of smoothing passes using 9-pt smoother.
  By setting this parameter with an integer value it will call the MetPy 9-pt smoother and provide a smoothed field for plotting. It is best to use this smoothing for data with finer resolutions (e.g., smaller grid spacings with a lot of grid points).

time

- Set the valid time to be plotted as a datetime object.
  If a forecast hour is to be plotted the time should be set to the valid future time, which can be done using the `datetime` and `timedelta` objects from the Python standard library. Note that this requires your data to have a time dimension coordinate.

mpl_args

- This attribute can be set to a dictionary containing keyword arguments (kwarg) valid for the various Matplotlib plotting functions to enhance how a plot is created. For example, you can use this attribute to set the `zorder` kwarg to alter what aspects of the your plot are on top of one another. NOTE: Setting the mpl_args trait will override any other trait that corresponds to a specific kwarg for the particular plot type (e.g., linecolor, linewidth).

MetPy - ImagePlot()

Make image plots by defining relevant traits.

`ImagePlot()`

Attribute	Description
colorbar	A string (horizontal/vertical) on whether to a a colorbar to the plot.
colorbar_fontsize	An integer, float, or string value to set the fo size of labels for the colorbar.
colormap	The string name for a Matplotlib or MetPy c ormap.
data	Xarray dataset that contains the field to be plo ted.
field	Name of the field to be plotted.
image_range	A tuple of min and max values that represe the range of values to color the rasterized imag
level	The level of the field to be plotted.
plot_units	The desired units to plot the field in.
scale	Scale the field to be plotted by the value give
smooth_contour	Spline interpolation to smooth contours.
smooth_field	Number of smoothing passes using 9- smoother.
time	Set the valid time to be plotted as a datetim object.
mpl_args	Supply a dictionary of valid Matplotlib keywo arguments to modify how the plot variable drawn.

Attribute Descriptions

colorbar

- A string (horizontal/vertical) on whether to add a colorbar to the plot.
  To add a colorbar associated with the plot, set the trait to `horizontal` or `vertical`, specifying the orientation of the produced colorbar. The default value is `None`.

colorbar_fontsize

- An integer, float, or string value to set the font size of labels for the colorbar.
  This trait sets the font size of labels for the colorbar. Accepts size in points or relative size. Allowed relative sizes are those of Matplotlib: 'xx-small', 'x-small', 'small', 'medium', 'large', 'x-large', 'xx-large'.

colormap

- The string name for a Matplotlib or MetPy colormap.
  For example, the Blue-Purple colormap from Matplotlib can be accessed using 'BuPu'.

data

- Xarray dataset that contains the field to be plotted.

field

- Name of the field to be plotted.
  This is the name of the variable from the dataset that is to be plotted. An example, from a model grid file that uses the THREDDS convention for naming would be *Geopotential_height_isobaric* or *Temperature_isobaric*. For GOES-16/17 satellite data it might be *Sectorized_CMI*. To check for the variables available within a dataset, list the variables with the following command assuming the dataset was read using xarray as *ds*, *list(ds)*

image_range

- A tuple of min and max values that represent the range of values to color the rasterized image.
  The min and max values entered as a tuple will be converted to a `matplotlib.colors.Normalize`instance for plotting.

level

- The level of the field to be plotted.
  This is a value with units to choose a desired plot level. For example, selecting the 850-hPa level, set this parameter to `850 * units.hPa`. Note that this requires your data to have a vertical dimension coordinate.

plot_units

- The desired units to plot the field in.
  Setting this attribute will convert the units of the field variable to the given units for plotting using the MetPy Units module.

scale

- Scale the field to be plotted by the value given.
  This attribute will scale the field by multiplying by the scale. For example, to scale vorticity to be whole values for contouring you could set the scale to `1e5`, such that the data values will be multiplied by $10\hat{5}$.

smooth_contour

- Spline interpolation to smooth contours.
  This attribute requires settings for the `metpy.calc.zoom_xarray` function, which will produce a spline interpolation given an integer zoom factor. Either a single integer specifying the zoom factor (e.g., 4) or a tuple containing two integers for the zoom factor and the spline interpolation order can be used. The default spline interpolation order is 3.
  This is best used to smooth contours when contouring a sparse grid (e.g., when your data has a large grid spacing).

smooth_field

- Number of smoothing passes using 9-pt smoother.
  By setting this parameter with an integer value it will call the MetPy 9-pt smoother and provide a smoothed field for plotting. It is best to use this smoothing for data with finer resolutions (e.g., smaller grid spacings with a lot of grid points).

time

- Set the valid time to be plotted as a datetime object.
  If a forecast hour is to be plotted the time should be set to the valid future time, which can be done using the `datetime` and `timedelta` objects from the Python standard library. Note that this requires your data to have a time dimension coordinate.

mpl_args

- This attribute can be set to a dictionary containing keyword arguments (kwarg) valid for the various Matplotlib plotting functions to enhance how a plot is created. For example, you can use this attribute to set the `zorder` kwarg to alter what aspects of the your plot are on top of one another. NOTE: Setting the mpl_args trait will override any other trait that corresponds to a specific kwarg for the particular plot type (e.g., linecolor, linewidth).

MetPy - BarbPlot()

This class is designed to plot wind barbs from gridded data, most commonly model output from the GFS, NAM, or RAP.

`BarbPlot()`

Attribute	Description
barblength	A float value that changes the length of the win barbs.
color	A string value that controls the color of the vec tors.
data	Xarray dataset that contains the field to be plot ted.
earth_relative	A boolean value to indicate whether the vecto to be plotted is earth- or grid-relative.
field	A tuple containing the two components of th vector field from the dataset in the form (east west component, north-south component).
level	The level of the field to be plotted.
pivot	A string setting the pivot point of the vector.
plot_units	The desired units to plot the field in.
scale	Scale the field to be plotted by the value given
skip	A tuple of integers to indicate the number of gri points to skip between plotting vectors.
time	Set the valid time to be plotted as a datetim object.
mpl_args	Supply a dictionary of valid Matplotlib keywor arguments to modify how the plot variable i drawn.

Attribute Descriptions

data
- This attribute must be set to the variable that contains the vector components to be plotted.

field
- This attribute is a list of the vector components to be plotted. For the typical meteorological case it would be the ['u-component', 'v-component'].

time
- This attribute should be set to a datetime object, the same as for all other declarative classes.

skip
- Set this to an integer value to skip the plotting of wind barbs in the x and y directions to thin the number of wind barbs plotted.

barblength
- This attribute sets the length of the wind barbs. The default value is based on the font size.

color
- This attribute sets the color of the wind barbs, which can be any Matplotlib color. Default color is 'black'.

earth_relative

- This attribute can be set to False if the vector components are grid relative (e.g., for NAM or NARR output)

pivot

- This attribute can be set to a string value about where the wind barb will pivot relative to the grid point. Possible values include 'tip' or 'middle'. Default is 'middle'.

mpl_args

- This attribute can be set to a dictionary containing keyword arguments (kwarg) valid for the various Matplotlib plotting functions to enhance how a plot is created. For example, you can use this attribute to set the `zorder` kwarg to alter what aspects of the your plot are on top of one another. NOTE: Setting the mpl_args trait will override any other trait that corresponds to a specific kwarg for the particular plot type (e.g., linecolor, linewidth).

MetPy - ArrowPlot()

Make plots of wind barbs on a map with traits to refine the look of plotted elements.

`ArrowPlot()`

Attribute	Description
arrowkey	Set the characteristics of an arrow key using tuple of values representing (value, xloc, ylo position, string).
arrowscale	Number of data units per arrow length unit, e. m/s per plot width; a smaller scale paramet makes the arrow longer.
color	A string value that controls the color of the ve tors.
data	Xarray dataset that contains the field to be plo ted.
earth_relative	A boolean value to indicate whether the vect to be plotted is earth- or grid-relative.
field	A tuple containing the two components of t vector field from the dataset in the form (eas west component, north-south component).
level	The level of the field to be plotted.
pivot	A string setting the pivot point of the vector.
plot_units	The desired units to plot the field in.
plotdata	Return the data for plotting.
scale	Scale the field to be plotted by the value give
skip	A tuple of integers to indicate the number of gr points to skip between plotting vectors.
time	Set the valid time to be plotted as a datetir object.
mpl_args	Supply a dictionary of valid Matplotlib keywo arguments to modify how the plot variable drawn.

Attribute Descriptions

arrowkey

- Set the characteristics of an arrow key using a tuple of values representing (value, xloc, yloc, position, string). Default is *None*.
  If *None*, no vector key will be plotted.
  value default is 100 xloc default is 0.85 yloc default is 1.02 position default is 'E' (options are 'N', 'S', 'E', 'W') label default is an empty string
  If you wish to change a characteristic of the arrowkey you'll need to have a tuple of five elements, fill in the full tuple using *None* for those characteristics you wish to use the default value and put in the new values for the other elements. This trait corresponds to the keyword length in `matplotlib.pyplot.quiverkey`.

arrowscale

- Number of data units per arrow length unit, e.g., m/s per plot width; a smaller scale parameter makes the arrow longer. Default is *None*.
  If *None*, a simple autoscaling algorithm is used, based on the average vector length and the number of vectors. The arrow length unit is given by the *key_length* attribute.
  This trait corresponds to the keyword length in `matplotlib.pyplot.quiver`.

color

- A string value that controls the color of the vectors. Default value is black.
  This trait can be set to any named color from *Matplotlibs Colors*

data

- Xarray dataset that contains the field to be plotted.

earth_relative

- A boolean value to indicate whether the vector to be plotted is earth- or grid-relative. Default value is *True*, indicating that vectors are earth-relative.

Common gridded meteorological datasets including GFS and NARR output contain wind components that are earth-relative. The primary exception is NAM output with wind components that are grid-relative. For any grid-relative vectors set this trait to *False*. This value is ignored for 2D vector fields not in the plane of the plot (e.g., cross sections).

field

- A tuple containing the two components of the vector field from the dataset in the form (east-west component, north-south component).
  For a wind barb plot each component of the wind must be specified and should be of the form (u-wind, v-wind).

level

- The level of the field to be plotted.
  This is a value with units to choose a desired plot level. For example, selecting the 850-hPa level, set this parameter to `850 * units.hPa`. Note that this requires your data to have a vertical dimension coordinate.

pivot

- A string setting the pivot point of the vector. Default value is 'middle'.
  This trait takes the values of the keyword argument from *matplotlin.pyplot.barbs*: 'tip' or 'middle'.

plot_units

- The desired units to plot the field in.
  Setting this attribute will convert the units of the field variable to the given units for plotting using the MetPy Units module.

scale

- Scale the field to be plotted by the value given.
  This attribute will scale the field by multiplying by the scale. For example, to scale vorticity to be whole values for contouring you could set the scale to 1e5, such that the data values will be multiplied by $10\hat{5}$.

skip

- A tuple of integers to indicate the number of grid points to skip between plotting vectors. Default is (1, 1).
  This trait is to be used to reduce the number of vectors plotted in the (east-west, north-south) components. The two values can be set to the same or different integer values depending on what is desired.

time

- Set the valid time to be plotted as a datetime object.
  If a forecast hour is to be plotted the time should be set to the valid future time, which can be done using the `datetime` and `timedelta` objects from the Python standard library. Note that this requires your data to have a time dimension coordinate.

mpl_args

- This attribute can be set to a dictionary containing keyword arguments (kwarg) valid for the various Matplotlib plotting functions to enhance how a plot is created. For example, you can use this attribute to set the `zorder` kwarg to alter what aspects of the your plot are on top of one another. NOTE: Setting the mpl_args trait will override any other trait that corresponds to a specific kwarg for the particular plot type (e.g., linecolor, linewidth).

MetPy - PlotGeometry()

Make raster plots by defining relevant traits.

`PlotGeometry()`

Attribute	Description
fill	Fill color(s) for polygons and points.Fill co for polygons and points.
geometry	A collection of Shapely objects to plot.
label_edgecolor	Outline color(s) for labels.
label_facecolor	Font color(s) for labels.
label_fontsize	An integer or string value for the font s labels.
labels	A collection of labels corresponding to p geometry.
marker	Symbol used to denote points.
level	Supply a dictionary of valid Matplotlib ke arguments to modify how the plot varia drawn.
stroke	Stroke color(s) for polygons and line color lines.
stroke_width	Stroke width(s) for polygons and lines.

Attribute Descriptions

fill

- Fill color(s) for polygons and points.
  A single string (color name or hex code) or collection of strings with which to fill polygons and points. If a collection, the first color corresponds to the first Shapely object in `geometry`, the second color corresponds to the second Shapely object, and so on. If `fill` is shorter than `geometry`, `fill` cycles back to the beginning, repeating the sequence of colors as needed. Default value is lightgray.

geometry

- A collection of Shapely objects to plot.
  A collection of Shapely objects, such as the 'geometry' column from a `geopandas.GeoDataFrame`. Acceptable Shapely objects are `shapely.MultiPolygon`, `shapely.Polygon`, `shapely.MultiLineString`, `shapely.LineString`, `shapely.MultiPoint`, and `shapely.Point`.

label_edgecolor

- Outline color(s) for labels.
  A single string (color name or hex code) or collection of strings for the outline color of labels. If a collection, the first color corresponds to the label of the first Shapely object in `geometry`, the second color corresponds to the label of the second Shapely object, and so on. Default value is `fill`.

label_fontcolor

- Font color(s) for labels.
  A single string (color name or hex code) or collection of strings for the font color of labels. If a collection, the first color corresponds to the label of the first Shapely object in `geometry`, the second color corresponds to the label of the second Shapely object, and so on. Default value is `fill`.

label_fontsize

- An integer or string value for the font size of labels.
  Accepts size in points or relative size. Allowed relative sizes are those of Matplotlib: 'xx-small', 'x-small', 'small', 'medium', 'large', 'x-large', 'xx-large'.

labels

- A collection of labels corresponding to plotted geometry.
  A collection of strings to use as labels for geometry, such as a column from a `Geopandas.GeoDataFrame`. The first label corresponds to the first Shapely object in `geometry`, the second label corresponds to the second Shapely object, and so on. The length of `labels` must be equal to the length of `geometry`. Labels are positioned along the edge of polygons, and below lines and points. No labels are plotted if this attribute is left undefined, or set equal to None.

marker

- Symbol used to denote points.
  Accepts any matplotlib marker. Default value is '.', which plots a dot at each point.

mpl_args

- This attribute can be set to a dictionary containing keyword arguments (kwarg) valid for the various Matplotlib plotting functions to enhance how a plot is created. For example, you can use this attribute to set the `zorder` kwarg to alter what aspects of the your plot are on top of one another. NOTE: Setting the mpl_args trait will override any other trait that corresponds to a specific kwarg for the particular plot type (e.g., linecolor, linewidth).

stroke

- Stroke color(s) for polygons and line color(s) for lines.
  A single string (color name or hex code) or collection of strings with which to outline polygons and color lines. If a collection, the first color corresponds to the first Shapely object in `geometry`, the second color corresponds to the second Shapely object, and so on. If `stroke` is shorter than `geometry`, `stroke` cycles back to the beginning, repeating the sequence of colors as needed. Default value is black.

stroke_width

- Stroke width(s) for polygons and lines.
  A single integer or floating point value or collection of values representing the size of the stroke width. If a collection, the first value corresponds to the first Shapely object in `geometry`, the second value corresponds to the second Shapely object, and so on. If `stroke_width` is shorter than `geometry`, `stroke_width` cycles back to the beginning, repeating the sequence of values as needed. Default value is 1.

MetPy - MapPanel()

`MapPanel()`

Attribute	Description
layers	A list of strings for a pre-defined feature layer o a Cartopy Feature object.
layers_alpha	A list of values defining the alpha for a layer.
layers_edgecolor	A list of strings for a pre-defined edgecolor for layer.
layers_linestyle	A list of string values defining the linestyle for layer or None.
layers_linewidth	A list of values defining the linewidth for a layer
layers_zorder	A list of values defining the zorder for a layer.
layout	A tuple that contains the description (nrows ncols, index) of the panel position; default valu is (1, 1, 1).
mpl_args	Supply a dictionary of valid Matplotlib keywor arguments to modify how the plot variable i drawn.
projection	A string for a pre-defined projection or a Cartop projection object.
left_title	A string to set a title for the figure with th location on the top left of the figure.
right_title	A string to set a title for the figure with th location on the top right of the figure.
title	A string to set a title for the figure.
title_fontsize	An integer or string value for the font size of th title of the figure.
plots	A list of handles that represent the plot (e.g., **PlotObs**, **ContourPlot**, **FilledCon tourPlot**, **ImagePlot**) to put on a given panel

Attribute Descriptions

layout
- The Matplotlib layout of the figure. For a single panel figure the setting should be (1, 1, 1)

projection
- The projection can be set with the name of a default projection ('lcc', 'mer', or 'ps') or it can be set to a Cartopy projection.

layers
- This attribute will add map layers to identify boundaries or features to plot on the map. Valid layers are borders, coastline, states, lakes, land, ocean, rivers, counties.

layers_alpha
- This attribute will set the alpha setting for those values listed in the layers attribute. Values should be given as floats between 0 and 1.

layers_edgecolor
- This attribute will set the edgecolor for those values listed in the layers attribute. Valid colors correspond to valid Matplotlib named colors.

layers_linestyle
- This attribute will set the linestyle for those values listed in the layers attribute. The valid values are those from Matplotlib and include 'dashed', 'dash-dot', 'dotted', or 'solid'.

layers_linewidth
- This attribute will set the linewidth (as a float) for those values listed in the layers attribute.

layers_zorder
- This attribute will set the zorder (as a float) for those values listed in the layers attribute and can be used to raise or lower layer features.

area
- This attribute sets the geographical area of the panel. This can be set with a predefined name of an area including all US state postal abbreviations (e.g., 'us', 'natl', 'in', 'il', 'wi', 'mi', etc.) or a tuple value that corresponds to longitude/latitude box based on the projection of the map with the format (west-most longitude, east-most longitude, south-most latitude, north-most latitude). This tuple defines a box from the lower-left to the upper-right corner.

mpl_args
- This attribute can be set to a dictionary containing keyword arguments (kwarg) valid for the various Matplotlib plotting functions to enhance how a plot is created. For example, you can use this attribute to set the `zorder` kwarg to alter what aspects of the your plot are on top of one another.

title
- This attribute sets a title for the panel in the center of the figure.

left_title
- This attribute sets a title for the panel at the left edge of the figure.

right_title
- This attribute sets a title for the panel at the right edge of the figure.

plots
- A list collecting the observations to be plotted in the panel.

MetPy - PanelContainer()

`PanelContainer()`

Attribute	Description
panels	A list of panels to plot on the figure.
size	This trait takes a tuple of (width, heigh the size of the figure.

Attribute Descriptions

size
- The size of the figure in inches (e.g., (10, 8))

panels
- A list collecting the panels to be plotted in the figure.

show
- Calling this will show the image.

save
- Setting this attribute will allow you to save the figure to a specified filename given in a string. Additional arguments can be passed in accordance with those from `matplotlib.savefig`.

Bibliography

J. Bjerknes and H. S. Solberg. Life cycle of cyclones and the polar front theory of atmospheric circulation. *Geofys. Publ.*, 3:1–18, 1922.

F. Sanders and C. A. Doswell. A case for detailed surface analysis. *Bull. Amer. Meteor. Soc.*, 76: 505–521, 1995.